Laborator 01: Divide et Impera

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Autori:

- Radu Vișan (2018) [mailto:visanr95@gmail.com]
- Cristian Banu (2018) [mailto:cristb@gmail.com]
- Darius Neațu (2018) [mailto:neatudarius@gmail.com]

Obiective laborator

- Înțelegerea conceptului teoretic din spatele descompunerii unei probleme
- Rezolvarea de probleme abordabile folosind conceptul de Divide et Impera

Precizari initiale

Toate exemplele de cod se gasesc in demo-lab01.zip.

Exemplele de cod apar incorporate si in textul laboratorului pentru a facilita parcurgerea cursiva a acestuia.

- Toate bucatile de cod prezentate in partea introductiva a laboratorului (inainte de exercitii) au fost testate. Cu toate acestea, este posibil ca din cauza mai multor factori (formatare, caractere invizibile puse de browser etc) un simplu copy-paste sa nu fie de ajuns pentru a compila codul.
- Va rugam sa incercati si codul din arhiva **demo-lab01.zip**, inainte de a raporta ca ceva nu merge. :D
- Pentru orice problema legata de continutul acestei pagini, va rugam sa dati email unuia dintre responsabili.

Importanţă – aplicaţii practice

Paradigma Divide et Impera stă la baza construirii de algoritmi eficienți pentru diverse probleme:

- Sortări (ex: MergeSort [1] [http://www.sorting-algorithms.com/merge-sort], QuickSort [2] [http://www.sorting-algorithms.com/quick-sort])
- Înmulțirea numerelor mari (ex: Karatsuba [3] [http://en.wikipedia.org/wiki/Karatsuba_algorithm])
- Analiza sintactică (ex: parsere top-down [4] [http://en.wikipedia.org/wiki/Top-down_parser])
- Calcularea transformatei Fourier discretă (ex: FFT [5] [http://en.wikipedia.org/wiki/Fast_Fourier_transform])

Un alt domeniu de utilizare a tehnicii divide et impera este programarea paralelă pe mai multe procesoare, sub-problemele fiind executate pe mașini diferite.

Prezentarea generală a problemei

O descriere a tehnicii D&I: "Divide and Conquer algorithms break the problem into several sub-problems that are similar to the original problem but smaller in size, solve the sub-problems recursively, and then combine these solutions to create a solution to the original problem." [7]

Deci un algoritm D&I **împarte problema** în mai multe subprobleme similare cu problema inițială şi de dimensiuni mai mici, **rezolva subproblemele** recursiv şi apoi **combina soluțiile** obţinute pentru a obține

soluția problemei inițiale.

Sunt trei pași pentru aplicarea algoritmului D&I:

- **Divide**: împarte problema în una sau mai multe *probleme similare de dimensiuni mai mici*.
- **Impera** (stăpânește): rezolva subprobleme recursiv; dacă dimensiunea sub-problemelor este mica se rezolva iterativ.
- **Combină**: combină soluțiile subproblemelor pentru a obține soluția problemei inițiale.

Complexitatea algoritmilor D&I se calculează după formula:

$T(n) = D(n) + S(n) + C(n)$,

unde $D(n)$, $S(n)$ şi $C(n)$ reprezintă complexitățile celor 3 pași descriși mai sus: divide, stăpânește, respectiv combină.

Probleme clasice

MergeSort

Enunt

Sortarea prin interclasare (MergeSort [1] [http://www.sorting-algorithms.com/merge-sort]) este un algoritm de sortare de vectori ce folosește paradigma D&I:

- **Divide**: împarte vectorul inițial în doi sub-vectori de dimensiune n/2.
- **Stăpânește**: sortează cei doi sub-vectori recursiv folosind sortarea prin interclasare; recursivitatea se oprește când dimensiunea unui sub-vector este 1 (deja sortat).
- **Combina**: Interclasează cei doi sub-vectori sortați pentru a obține vectorul inițial sortat.

Pseudocod

Mai jos gasiti algoritmul MergeSort scris in pseudocod.

```
MergeSort(v, start, end)                // v – vector, start – limită inferioră, end – limită superioară
        if (start == end) return;       // condiția de oprire
        mid = (start + end) / 2;        // etapa divide
        MergeSort(v, start, mid);       // etapa stăpânește
        MergeSort(v, mid + 1, end);
        Merge(v, start, end);           // etapa combină

Merge(v, start, end)                    // interclasare sub-vectori
        // indecsi
        mid = (start + end) / 2;
        i = start;
        j = mid + 1;
        k = 1;

        tmp = buffer temporar in care incap (end - start + 1) numere
        while (i <= mid && j <= end)
                if (v[i] <= v[j]) tmp[k++] = v[i++];
                             else tmp[k++] = v[j++];

        while (i <= mid)
                tmp[k++] = v[i++];

        while (j <= end)
                tmp[k++] = v[j++];

        copy(v[start..end], tmp[1..k-1]);
```

Mai jos puteti gasi o implementare in C++.

```
#include <bits/stdc++.h>
using namespace std;

vector<int> v;

void merge_halves(int left, int right) {
        int mid = (left + right) / 2;
        vector<int> aux;
        int i = left, j = mid + 1;

        while (i <= mid && j <= right) {
                if (v[i] <= v[j]) {
                        aux.push_back(v[i]);
                        i++;
                } else {
                        aux.push_back(v[j]);
                        j++;
                }
        }

        while (i <= mid) {
                aux.push_back(v[i]);
                i++;
        }

        while (j <= right) {
                aux.push_back(v[j]);
                j++;
        }

        for (int k = left; k <= right; k++) {
                v[k] = aux[k - left];
        }
}


void merge_sort(int left, int right) {
        if (left >= right) return ;
        int mid = (left + right) / 2;

        merge_sort(left, mid);
        merge_sort(mid + 1, right);
        merge_halves(left, right);
}

int main() {
        random_device rd;
        for (int i = 0; i < 10; i++) {
                v.push_back(rd() % 100);
        }

        cout << "Vectorul initial: ";
        for (int i = 0; i < v.size(); i++) {
                cout << v[i] << " ";
        }
        cout << "\n";

        merge_sort(0, v.size() - 1);

        cout << "Vectorul sortat: ";
        for (int i = 0; i < v.size(); i++) {
                cout << v[i] << " ";
        }
        cout << "\n";

        return 0;
}
```

Complexitate

Complexitatea algoritmului este dată de formula: $T(n) = D(n) + S(n) + C(n)$, unde $D(n) = O(1)$, $S(n) = 2 * T(n/2)$ și $C(n) = O(n)$, rezulta $T(n) = 2 * T(n/2) + O(n)$.

Folosind teorema Master [8] [http://people.csail.mit.edu/thies/6.046-web/master.pdf] găsim complexitatea algoritmului: $T(n) = O(n * log(n))$.

- **complexitate temporala** : $T = O(n * log(n))$
  - Ce inseamna aceasta metrica? Masoara efectiv **timpul de executie** al algoritmului (nu include citiri, afisari etc).
- **complexitate spatiala** : $S = O(n)$
  - Ce inseamna aceasta metrica? Masoara efectiv **memoria suplimentara** folosita de algoritm (in acest caz ne referim strict la buffer-ul temporar).

Retineti cele doua conventii despre complexitati de mai sus. Le vom folosi pentru restul semestrului.

Binary Search

Enunt

Se dă un **vector sortat crescător** ($v[1]$, $v[2]$, ..., $v[n]$) ce conține valori reale distincte și o valoare x.

Sa se găsească la ce **poziție** apare x în vectorul dat.

Rezolvare

BinarySearch (Cautare Binara), se rezolva cu un algoritm D&I:

- **Divide**: împărțim vectorul în doi sub-vectori de dimensiune n/2.
- **Stăpânește**: aplicăm algoritmul de căutare binară pe sub-vectorul care conține valoarea căutată.
- **Combină**: soluția sub-problemei devine soluția problemei inițiale, motiv pentru care nu mai este nevoie de etapa de combinare.

Pseudocod

```
BinarySearch(v, left, right, x) {             // functia returneaza pozitia x pe care se afla numarul x (oricare pozitie) sa
       // vom cauta cat timp intervalul de cautare nu a fost inca epuizat (are cel putin un element)
       while (left <= right) {
            mid = (left + right) / 2          // mijlocul intervalului de cautare

            if (x == v[mid]) return mid;      // elementul cautat este cel din mijloc
            if (x < v[mid])  right = mid - 1; // elementul cautat este mai mic decat cel din mijloc, ne mutam in prima jumatate
            if (x > v[mid])  left  = mid + 1; // elementul cautat este mai mare decat cel din mijloc, ne mutam in a doua jumatate
        }

        return -1;                            // in acest punct ajungem daca si numai daca x nu a fost gasit
}
```

Complexitate

- **complexitate temporala** : $T = O(log(n))$
  - se deduce din recurenta $T(n) = T(n/2) + O(1)$
- * **complexitate spatiala** : $S = O(1)$
  - nu avem structuri de date complexe auxiliare
  - atragem atentia ca acest algoritm se poate implementa si **recursiv**, caz in care complexitatea spatiala devine $O(log(n))$, intrucat salvam pe stiva $O(log(n))$ parametri (intregi, referinte)

Turnurile din Hanoi

Enunt

Se considera 3 tije S (**sursa**), D (**destinatie**), aux (**auxiliar**) şi n discuri de dimensiuni distincte ($1, 2, \ldots, n$ - ordinea crescătoare a dimensiunilor) situate iniţial toate pe tija S în ordinea $1, 2, \ldots, n$ (de la vârf către baza).

Singura operaţie care se poate efectua este de a selecta un disc ce se află în vârful unei tije şi plasarea lui în vârful altei tije astfel încât să fie aşezat deasupra unui disc de dimensiune mai mare decât a sa.

Sa se găsească un algoritm prin care se mută toate discurile de pe tija S pe tija D (problema turnurilor din Hanoi).

Solutie

Pentru rezolvarea problemei folosim următoarea strategie [9] [http://www.mathcs.org/java/programs/Hanoi/index.html]:

- mutam primele $n-1$ discuri de pe tija S pe tija aux folosindu-ne de tija D.
- mutam discul n pe tija D.
- mutam apoi cele $n-1$ discuri de pe tija aux pe tija D folosindu-ne de tija S.

Ideea din spate este ca avem mereu o singura sursa si o singura destinatie sa atingem un scop. Intotdeauna a 3-a tija va fi considerata auxiliara si poate fi folosita pentru a atinge scopul propus.

Algoritm

```
// muta n discuri de pe tija S pe tija D folosind tija aux
Hanoi(n, S, D, aux)  {
    if (n >= 1) {
        Hanoi(n - 1, S, aux, D);   // mut n-1 discuri de pe sursa (S) pe auxiliar (aux)
                                   // in aceasta subproblema sursa este S, destinatia este aux, intermediarul este D

        Muta_disc(S, D);           // acum pot muta direct discul n de pe sursa (S) pe destinatie (D)

        Hanoi(n - 1, aux, D, S);   // mut n-1 discuri de pe sursa (aux, aici sunt ele momentan) pe destinatie (D - scop final)
                                   // in aceasta subproblema, S este auxiliar, intrucat este tija libera
    }
}
```

Complexitate

- **complexitate temporala** : $T(n) = O(2^n)$
  - se deduce din recurenta $T(n) = 2 * T(n-1) + O(1)$
- * **complexitate spatiala** : $S(n) = O(n)$
  - la un moment dat, nivelul maxim de recursivitate este n

ZParcurgere

Enunt

Gigel are o tabla patratica de dimensiuni $2^n * 2^n$. Ar vrea sa scrie pe patratelele tablei numere naturale cuprinse intre 1 si $2^n * 2^n$ conform unei parcurgeri mai deosebite pe care o numeste **Z-parcurgere**.

O Z-parcurgere viziteaza recursiv cele patru cadrane ale tablei in ordinea: **stanga-sus**, **dreapta-sus**, **stanga-jos**, **dreapta-jos**.

La un moment dat Gigel ar vrea sa stie ce **numar de ordine** trebuie sa scrie conform Z-parcurgerii pe anumite patratele date prin coordonatele lor (x, y). Gigel incepe umplerea tablei **intotdeauna** din coltul din stanga-sus.

$n = 1$ si $(x, y) = (2, 2)$

Raspuns: 4

Explicatie: Matricea arata ca in exemplul urmator.

1	2
3	4

$n = 2$ si $(x, y) = (3, 3)$

Raspuns: 13

Explicatie: Matricea arata ca in exemplul urmator.

1	2	5	6
3	4	7	8
9	10	13	14
11	12	15	16

Solutie

Analizand modul in care se **completeaza** tabloul/matricea din enunt, observam ca la fiecare etapa impartim matricea (**problema**) in 4 submatrici (**4 subprobleme**). De asemenea, sirul de numere pe care dorim sa il punem in matrice se imparte in 4 secvente, fiecare corespunzand unei submatrici.

Observam astfel ca problema suporta **descompunerea** in **subprobleme disjuncte** si cu **structura similara**, ceea ce ne face sa ne gandim la o solutie cu Divide et Impera.

Complexitate

- **complexitate temporala** : $T = O(n)$
  - $\log_4(2^n) = \frac{1}{2}\log_2(2^n) = \frac{1}{2}n$
- **complexitate spatiala** : $S = O(n)$
  - stocam parametri pentru recursivitate
  - solutia se poate implementa si iterativ, caz in care $S = O(1)$; deoarece dimensinile spatiului de cautare sunt $2^n * 2^n$, n este foarte mic ($n <= 15$), de aceea o solutie iterativa nu aduce nici un castig **efectiv** in aceasta situatie

Concluzii

Divide et impera este o tehnică folosită pentru a realiza solutii pentru o anumita clasa de probleme: acestea contin **subprobleme disjuncte** si cu **structura similara**. În cadrul acestei tehnici se disting trei etape: divide, stăpânește și combină.

Mai multe exemple de algoritmi care folosesc tehnica divide et impera puteți găsi la [11] [http://www.cs.berkeley.edu/~vazirani/algorithms/chap2.pdf].

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab01.zip.

Count occurrences

Se da un sir sortat **v** cu **n** elemente. Gasiti numarul de elemente egale cu **x** din sir.

$n = 6$ si $x = 10$

i	1	2	3	4	5	6
v	1	2	4	10	10	20

Raspuns: 2

Explicatie: 10 apare de 2 ori in sir.

Task-uri:

- Aceasta problema este deja rezolvata. Pentru a va acomoda cu scheletul, va trebui sa faceti cativa pasi:
  - Rulati comanda $./check.sh$ si cititi cum se foloseste checker-ul.
  - Rulati comanda necesara pentru a rula task-ul 1. Sursa nu implementeaza corect algoritmul si returneaza valori default. Din acest motiv primiti mesajul **WRONG ANSWER**.
  - Copiati urmatoarea sursa in folderul corespunzator. Rulati comanda anterioara. Observati mesajele afisate cand ati rezolvat corect un task.

Sursa **main.cpp** asociata cu task 1 este mai jos.

```
#include <bits/stdc++.h>
using namespace std;

class Task {
public:
        void solve() {
                read_input();
                print_output(get_result());
        }

private:
        void read_input() {
                ifstream fin("in");
                fin >> n;
                for (int i = 0, e; i < n; i++) {
                        fin >> e;
                        v.push_back(e);
                }
                fin >> x;
                fin.close();
        }

        int find_first() {
                int left = 0, right = n - 1, mid, res = -1;
                while (left <= right) {
                        mid = (left + right) / 2;
                        if (v[mid] >= x) {
                                res = mid;
                                right = mid - 1;
                        } else {
                                left = mid + 1;
                        }
                }
                return res;
        }

        int find_last() {
                int left = 0, right = n - 1, mid, res = -1;
                while (left <= right) {
                        mid = (left + right) / 2;
                        if (v[mid] <= x) {
                                res = mid;
                                left = mid + 1;
                        } else {
                                right = mid - 1;
                        }
                }
                return res;
        }

        int get_result() {
                int first = find_first();
                int last = find_last();
                if (first == -1 || last == -1) {
                        return 0;
                }
```

```
                return last - first + 1;
        }

        void print_output(int result) {
                ofstream fout("out");
                fout << result;
                fout.close();
        }

        int n, x;
        vector<int> v;
};

int main() {
        Task task;
        task.solve();
        return 0;
}
```

Sursa **Main.java** asociata cu task 1 este mai jos.

```
import java.io.File;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;

public class Main {
        static class Task {
                public final static String INPUT_FILE = "in";
                public final static String OUTPUT_FILE = "out";

                int n;
                int[] v;
                int x;

                private void readInput() {
                        try {
                                Scanner sc = new Scanner(new File(INPUT_FILE));
                                n = sc.nextInt();
                                v = new int[n];
                                for (int i = 0; i < n; i++) {
                                        v[i] = sc.nextInt();
                                }
                                x = sc.nextInt();
                                sc.close();
                        } catch (IOException e) {
                                throw new RuntimeException(e);
                        }
                }

                private void writeOutput(int count) {
                        try {
                                PrintWriter pw = new PrintWriter(new File(OUTPUT_FILE));
                                pw.printf("%d\n", count);
                                pw.close();
                        } catch (IOException e) {
                                throw new RuntimeException(e);
                        }
                }

                private int findFirst() {
                        int left = 0, right = n - 1, mid, res = -1;
                        while (left <= right) {
                                mid = (left + right) / 2;
                                if (v[mid] >= x) {
                                        res = mid;
                                        right = mid - 1;
                                } else {
                                        left = mid + 1;
                                }
                        }
                        return res;
                }

                private int findLast() {
```

```
                        int left = 0, right = n - 1, mid, res = -1;
                        while (left <= right) {
                                mid = (left + right) / 2;
                                if (v[mid] <= x) {
                                        res = mid;
                                        left = mid + 1;
                                } else {
                                        right = mid - 1;
                                }
                        }
                        return res;
                }

                private int getAnswer() {
                        int first = findFirst();
                        int last = findLast();
                        if (first == -1 || last == -1) {
                                return 0;
                        }
                        return last - first + 1;
                }

                public void solve() {
                        readInput();
                        writeOutput(getAnswer());
                }
        }

        public static void main(String[] args) {
                new Task().solve();
        }
}
```

- Intelegeti solutia oferita impreuna cu asistentul vostru.
- Care este complexitatea solutiei (timp + spatiu)? De ce?

SQRT

Se da un numar real **n**. Scrieti un algoritm de complexitate **O(log n)** care sa calculeze $sqrt(n)$ cu o precizie de 0.001.

$n = 0.25$

Raspuns: orice valoare intre 0.499 si 0.501 (inclusiv)

Pentru a putea trece testele, trebuie sa afisati rezultatul cu **cel putin** 4 zecimale.

ZParcurgere

Rezolvati problema ZParcurgere folosind scheletul pus la dispozitie. Enuntul si explicatiile le gasiti in partea de seminar.

Exponentiere logaritmica

Se dau doua numere naturale **base** si **exponent**. Scrieti un algoritm de complexitate $O(log(exponent))$ care sa calculeze $base^{exponent} \ \% \ MOD$.

Intrucat expresia $base^{exponent}$ este foarte mare, dorim sa aflam doar **restul** impartirii lui la un numar **MOD**.

Proprietati matematice necesare:

- $(a + b) \ \% \ MOD = ((a \ \% \ MOD) + (b \ \% \ MOD)) \ \% \ MOD$
- $(a \ * b) \ \% \ MOD = ((a \ \% \ MOD) \ * (b \ \% \ MOD)) \ \% \ MOD$

Atentie la inmultire! Rezultatul **temporar** poate provoca un overflow. Solutii:

- **C++**: $a * b \Rightarrow 1LL * a * b$
- **Java**: $a * b \Rightarrow 1L * a * b$

$base = 2$, $exponent = 10$, $MOD = 5$

Raspuns: 4

Explicatie: $2^{10} \ \% \ 5 = 4$

Bonus

Se da un sir S de n numere intregi. Sa se detemine cate inversiuni sunt in sirul dat. Numim inversiune o pereche de indici $1 <= i < j <= n$ astfel incat $S[i] > S[j]$

Exemplu: in sirul `{0 1 9 4 5 7 6 8 2}` sunt 12 inversiuni.

Aceasta problema nu are schelet.

Testați soluția voastră de la **Exponentiere logaritmica** pe infoarena, la problema ClassicTask [https://infoarena.ro/problema/classictask] (trebuie să modificați numele fișierelor).

Identificați problema și modificați sursa astfel încât să luați punctaj maxim.

Extra

Se dă un vector de numere întregi neordonate. Scriind o funcție de partitionare, folosiți **Divide et Impera** pentru

- a determina a k-lea element ca mărime din vector
- a sorta vectorii prin QuickSort

Puteti testa problema partition aici [https://leetcode.com/problems/kth-largest-element-in-an-array/description/]. Problema QuickSort [https://en.wikipedia.org/wiki/Quicksort] (chiar si MergeSort) poate fi testata aici [https://infoarena.ro/problema/algsort].

Exemplu: pentru vectorul `{0 1 2 4 5 7 6 8 9}`, al 3-lea element ca ordine este 2, iar vectorul sortat este {0 1 2 4 5 6 7 8 9}

Se dau $n - 1$ numere naturale distincte intre 0 si $n - 1$. Scriind o functie de partitionare, determinati numarul lipsa.

Exemplu: pentru $n = 9$ si vectorul 019457682, numarul lipsa este 3.

Puteti rezolva aceasta problema pe infoarena [https://infoarena.ro/problema/fractal].

Puteti rezolva aceasta problema de aici [https://ocw.cs.pub.ro/courses/pa/laboratoare/laborator-03#ssm].

Pentru problema SSM vom studia o solutie potrivita in lab03. Puteti sa incercati sa o rezolvati cu Divide et Impera pentru a exersa cele invatate astazi.

Dându-se N numere întregi sub forma unei secvenţe de numere strict crescătoare, care se continuă cu o secvenţă de întregi strict descrescătoare, se doreşte determinarea punctului din întregul şir înaintea căruia toate elementele sunt strict crescătoare, şi dupa care, toate elementele sunt strict descrescătoare. Considerăm

evident faptul că acest punct nu există dacă cele N numere sunt dispuse într-un şir fie doar strict crescător, fie doar strict descrescător.

pa/laboratoare/laborator-01.txt · Last modified: 2020/03/03 21:52 by darius.neatu

Laborator 02 : Greedy

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Autori:

- Alex Rotaru (2018) [mailto:rotarualexandruandrei94@gmail.com]
- Darius Neațu (2018) [mailto:neatudarius@gmail.com]
- Radu Vișan (2018) [mailto:visanr95@gmail.com]
- Cristian Banu (2018) [mailto:cristb@gmail.com]

Obiective laborator

- Înțelegerea noțiunilor de bază legate de tehnica greedy
- Însușirea abilităților de implementare a algoritmilor bazați pe greedy

Precizari initiale

Toate exemplele de cod se gasesc in arhiva demo-lab02.zip.

Acestea apar incorporate si in textul laboratorului pentru a facilita parcurgerea cursiva a laboratorului.

- Toate bucatile de cod prezentate in partea introductiva a laboratorului (inainte de exercitii) au fost testate. Cu toate acestea, este posibil ca din cauza mai multor factori (formatare, caractere invizibile puse de browser etc) un simplu copy-paste sa nu fie de ajuns pentru a compila codul.
- Va rugam sa incercati si codul din arhiva **demo-lab02.zip**, inainte de a raporta ca ceva nu merge. :D
- Pentru orice problema legata de continutul acestei pagini, va rugam sa dati email unuia dintre responsabili.

Importanță – aplicații practice

În general tehnicile de tip Greedy sau Programare Dinamică (lab04) sunt folosite pentru rezolvarea problemelor de optimizare. Acestea pot adresa probleme în sine sau pot fi subprobleme dintr-un algoritm mai mare. De exemplu, algoritmul Dijkstra pentru determinarea drumului minim pe un graf alege la fiecare pas un nod nou urmărind algoritmul greedy.

Exista însă probleme care ne pot induce în eroare. Astfel, există probleme în care urmărind criteriul Greedy nu ajungem la soluția optimă. Este foarte important să identificăm cazurile când se poate aplica Greedy și cazurile când este nevoie de altceva. Alteori această soluție neoptimă este o aproximare suficientă pentru ce avem nevoie. Problemele NP-complete necesita multă putere de calcul pentru a găsi optimul absolut. Pentru a optimiza aceste calcule mulți algoritmi folosesc decizii Greedy și găsesc un optim foarte aproape de cel absolut.

Greedy

"greedy" = "lacom". Algoritmii de tip greedy vor să construiască într-un mod cât mai rapid soluția unei probleme. Ei se caracterizează prin luarea unor decizii rapide care duc la găsirea unei soluții potențiale a problemei. Nu întotdeauna asemenea decizii rapide duc la o soluție optimă; astfel ne vom concentra atenția pe identificarea acelor anumite tipuri de probleme pentru care se pot obține soluții optime. În general exista mai multe soluții posibile ale problemei. Dintre acestea se pot selecta doar anumite soluții optime, conform unor anumite criterii. Algoritmii greedy se numără printre cei mai direcți algoritmi posibili. Ideea de bază este simplă: având o problema de optimizare, de calcul al unui cost minim sau maxim, se va alege la fiecare pas decizia cea mai favorabilă, fără a evalua global eficiența soluţiei. Scopul este de a găsi una dintre acestea sau dacă nu este posibil, atunci o soluție cât mai apropiată, conform criteriului optimal impus.

Trebuie înțeles faptul ca rezultatul obținut este optim doar dacă un optim local conduce la un optim global. În cazul în care deciziile de la un pas influențează lista de decizii de la pasul următor, este posibila obținerea unei valori neoptimale. În astfel de cazuri, pentru găsirea unui optim absolut se ajunge la soluții supra-polinomiale. De aceea, dacă se optează pentru o astfel de soluție, algoritmul trebuie însoțit de o demonstrație de corectitudine. Descrierea formală a unui algoritm greedy este următoarea:

```
// C este mulțimea candidaților
function greedy(C) {
    S ← ∅ // în S construim soluția

    while !solutie(C) and C ≠ ∅
        x ← un element din C care minimizează/maximizează select(x)
        C ← C \ {x}
        if fezabil( S ∪ {x}) then S ← S∪{x}

    return S
}
```

Este ușor de înțeles acum de ce acest algoritm se numește "greedy": la fiecare pas se alege cel mai bun candidat de la momentul respectiv, fără a studia alternativele disponibile în moment respectiv şi viabilitatea acestora în timp.

Dacă un candidat este inclus în soluție, rămâne acolo, fără a putea fi modificat, iar dacă este exclus din soluție, nu va mai putea fi niciodată selectat drept un potențial candidat.

Exemple

Simple task

Enunt

Fie un șir de N numere pentru care se cere determinarea unui subșir de numere cu suma maximă. Un subșir al unui șir este format din elemente (nu neapărat consecutive) ale șirului respectiv, în ordinea în care acestea apar în șir.

Pentru numerele $1, -5, 6, 2, -2, 4$ răspunsul este $1, 6, 2, 4$ (suma 13).

Solutie

Se observa ca tot ce avem de făcut este sa verificam fiecare număr dacă este pozitiv sau nu. În cazul pozitiv, îl introducem în subșirul soluție.

Daca toate numerele sunt negative, solutia este data de cel mai mare numar negativ (cel mai mic in modul).

Problema spectacolelor

Enunt

Se dau mai multe spectacole, prin timpii de start și timpii de final. Se cere o planificare astfel încât o persoană să poată vedea cât mai multe spectacole.

Solutie

Rezolvarea constă în **sortarea** spectacolelor crescător după timpii de final, apoi la fiecare pas se alege **primul spectacol** care are timpul de start mai mare decât ultimul timp de final. Timpul inițial de final este inițializat la $-\inf$ (spectacolul care se termină cel mai devreme va fi mereu selectat, având timp de start mai mare decât timpul inițial).

Implementare

```
bool end_hour_comp (pair<int, int>& e1, pair<int, int>& e2) {
    // comparam doar dupa ora de sfarsit
    return (e1.second < e2.second);
}

vector<pair<int, int>> plan(vector<pair<int, int> >& intervals) {
    vector<pair<int, int>> plan;
    // se sorteaza intervalele pe baza orei de sfarsit a spectacolelor
    sort(intervals.begin(), intervals.end(), end_hour_comp);

    // se ia ultimul spectacol ca terminat la -oo pt a putea incepe cu
    // cel mai devreme
    int last_end = INT_MIN; // -oo a.k.a -infinit
    for (auto interval : intervals) {
        // daca inceputul intervalului curent este dupa sfarsitul ultimului
        // spectacol (last_end) il adaugam in lista de spectacole la care
        // se participa
        if (interval.first >= last_end)
        {
            plan.push_back(interval);
            // dupa ce am adaugat un spectacol, updatam ultimul sfarsit de spectacol
            last_end = interval.second;
        }
    }
    return plan;
}
```

Complexitate

Solutia va avea urmatoarele complexitati:

- **complexitate temporala** : $T(n) = O(n * log(n))$
  - explicatie
    - sortarea are $O(n * log(n))$
    - facem inca o parcurgere in $O(n)$
- **complexitate spatiala** : depinde de algoritmul de sortare folosit.

Problema florarului

Enunt

Se da un grup de k oameni care vor sa cumpere impreuna n flori. Fiecare floare are un pret de baza, insa pretul cu care este cumparata variaza in functie de numarul de flori cumparate anterior de persoana respectiva. De exemplu daca George a cumparat 3 flori (diferite) si vrea sa cumpere o floare cu pretul 2, el

va plati $(3 + 1) * 2 = 8$. Practic el va plati un pret proportional cu numarul flori cumparate pana atunci tot de el.

Cerinta: Se cere pentru un numar k de oameni si n flori sa se determine care este costul minim cu care grupul poate sa achizitioneze toate cele n flori o singura data.

Observatie: Un tip de floare se cumpara o singura data. O persoana poate cumpara mai multe tipuri de flori. In final in grup va exista un singur exemplar din fiecare tip de floare.

Formal avem k numar de oameni, n numar de flori, $c[i]$ = pretul florii de tip i, costul de cumparare i va fi $(x + 1) * c[i]$, unde x este numarul de flori cumparate anterior de persoana respectiva.

n=3 k=3 c=[2 5 6]

Cost minim = 13

Explicatie: Fiecare individ cumpara cate o floare si deci acestea se cumpara la pretul nominal.

Solutie

Se observa ca pretul efectiv de cumpare va fi mai mare cu cat cumparam acea floare mai tarziu. Daca consideram cazul in care avem o singura persoana in grup observam ca are sens sa cumparam obiectele in ordine descrescatoare(deoarece vrem sa minimizam costul fiecarui tip de flori si aceste creste cu cat cumparam floarea mai tarziu).

De aici, gandindu-ne la versiunea cu k persoane, observam ca ar fi mai ieftin daca am repartiza urmatoarea cea mai scumpa floare la alt individ. Deci impartim florile sortate descrescator dupa pret in grupuri de cate k, fiecare individ luand o floare din acest grup si ne asiguram ca pretul va creste doar in functie de numarul de grupuri anterioare.

Implementare

```
struct greater_comparator
{
    template<class T>
    bool operator()(T const &a, T const &b) const { return a > b; }
};

int minimum_cost(int k, vector<int>& costs) {
    // sortam vectorul de preturi in ordine descrescatoare
    sort(costs.begin(), costs.end(), greater_comparator());

    // numarul de flori cumparate de fiecare individ din grup la un moment dat
    int x = 0;
    // o varianta mai putin eficienta spatial ar fi fost sa retinem pt fiecare
    // individ din grup numarul de flori cumparate intr-un hashmap
    // costul total
    int total_cost = 0;

    // parcurgem fiecare pret de floare si o "asignam" unui individ din grup
    // pretul acesteia fiind proportional cu numarul de achizitii facut pana acum
    // de acesta (x)
    for (int idx = 0; idx < costs.size(); idx++) {
        int customer_idx = idx % k;
        total_cost += (x + 1) * costs[idx];
        // in momentul in care ultimul individ a cumparat o floare din grupul curent
        // incrementam numarul de flori achizitionate de fiecare
        if (customer_idx == k - 1) {
            x += 1;
        }
    }
    return total_cost;
}
```

Complexitate

Solutia va avea urmatoarele complexitati:

- **complexitate temporala** : $T(n) = O(n * log(n))$
  - explicatie
    - sortarea are $O(n * log(n))$
    - facem inca o parcurgere in $O(n)$
- **complexitate spatiala** : depinde de algoritmul de sortare folosit. Fara partea de sortare, spatiul este constant (nu se ia in considerare vectorul de elemente).

Problema cuielor

Enunt

Fie N scânduri de lemn, descrise ca niște **intervale închise** cu capete reale. Găsiți o mulțime **minimă** de cuie astfel încât fiecare scândură să fie bătută de cel puțin un cui. Se cere poziția cuielor.

Formulat matematic: găsiți o mulțime de puncte de cardinal **minim** M astfel încât pentru orice interval [a_i, b_i] din cele N, să existe un punct x din M care să aparțină intervalului [a_i, b_i].

- intrare: N = 5, intervalele: [0, 2], [1, 7], [2, 6], [5, 14], [8, 16]
- ieșire: M = {2, 14}
- explicație: punctul 2 se afla în primele 3 intervale, iar punctul 14 în ultimele 2

Solutie

Se observa că dacă x este un punct din M care nu este capăt dreapta al nici unui interval, o translație a lui x la dreapta care îl duce în capătul dreapta cel mai apropiat nu va schimba intervalele care conțin punctul. Prin urmare, exista o mulțime de cardinal minim M pentru care toate punctele x sunt capete dreapta.

Astfel, vom crea mulțimea M folosind numai capete dreapta în felul următor:

- sortăm intervalele dupa capatul dreapta
- iterăm prin fiecare interval și dacă intervalul curent nu conține ultimul punct introdus în mulțime atunci îl adăugam pe acesta la mulțime

Implementare

```
bool point_in_interval(const pair<int, int>&interval, int point) {
    return point >= interval.first && point <= interval.second;
}

bool right_edge_comparator (pair<int, int>& e1, pair<int, int>& e2) {
    // comparam scandurile dupa capatul drepata
    return (e1.second < e2.second);
}

vector<int> cover_intervals_greedy(vector<pair<int, int>>& intervals) {
    vector<int> nails; // pozitiile cuielor, a.k.a multimea M
    // ultimul punct inserat
    int last_point = INT_MIN;

    //sortam invervalele dupa capatul drepata
    sort(intervals.begin(), intervals.end(), right_edge_comparator);
```

```
    for (auto interval : intervals) {
        // daca intervalul nu contine ultimul punct adaugat
        if (!point_in_interval(interval, last_point)) {
            // il adaugam in multimea M
            nails.push_back(interval.second);
            // updatam ultimul punt inserat
            last_point = interval.second;
        }
    }

    return nails;
}
```

Complexitate

Soluția va avea următoarele complexități:

- **complexitate temporala** : $T(n) = O(n * log(n))$
  - explicație
    - sortare: $O(n * logn)$
    - parcurgerea intervalelor: $O(n)$
- **complexitate spațială** : depinde de algoritmul de sortare folosit.

Concluzii şi observații

Aspectul cel mai important de reținut este că soluțiile găsite trebuie să reprezinte optimul global și nu doar local. Se pot confunda ușor problemele care se rezolvă cu Greedy cu cele care se rezolvă prin Programare Dinamică (vom vedea saptamana viitoare).

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab02.zip.

Rucsac

Fie un set cu n obiecte (care pot fi **taiate** - varianta continua a problemei). Fiecare obiect i are asociata o pereche (w_i, p_i) cu semnificatia:

- w_i = $weight_i$ = greutatea obiectului cu numarul i
- p_i = $price_i$ = pretul obiectului cu numarul i
  - $w_i >= 0$ si $p_i > 0$

Gigel are la dispozitie un rucsac de **volum infinit**, dar care suporta o **greutate maxima** (notata cu W - weight knapsack).

El vrea sa gaseasca **o submultime de obiecte** (nu neaparat intregi) pe care sa le bage in rucsac, astfel incat suma profiturilor sa fie **maxima**.

Daca Gigel baga in rucsac obiectul i, caracterizat de (w_i, p_i), atunci profitul adus de obiect este p_i (presupunem ca il vinde cu cat valoareaza).

In aceasta varianta a problemei, Gigel poate taia oricare dintre obiecte, obtinand o proportie din acesta. Daca Gigel alege alege doar x din greutatea w_i a obiectului i, atunci el castiga doar $\frac{x}{w_i} * p_i$.

Task-uri:

- Sa se determine profitul maxim pentru Gigel.

- Care este complexitatea solutiei (timp + spatiu)? De ce?

obiecte:

index	0	1	2
greutate	60	100	120
valoare	10	20	30

greutate = 50

Output: 12.5 Explicatie: avem 50 capacitate si toate obiectele au o greutatate mai mare, decidem sa luam cat putem din produsul cu raportul valoare / greutate cel mai mare. profitu = 30 / 120 * 50 = 12.5

obiecte:

index	0	1	2
greutate	20	50	30
valoare	60	100	120

greutate = 50

Output: 180 Explicatie: Sortam obiectele dupa raportul valoare profit si avem in ordine: {30, 120}, {20, 60}, {50, 100} Introducem obiecte pana cand umplem sacul ⇒ intra primele 2 obiecte. Calculam profitul 120 + 60 = 180

Distante

Consideram 2 localitati A si B aflate la distanta D. Intre cele 2 localitati avem un numar de n benzinarii, date prin distanta fata de localitatea A. Masina cu care se efectueaza deplasarea intre cele 2 localitati poate parcurge maxim m kilometri avand rezervorul plin la inceput. Se doreste parcurgerea drumului cu un numar minim de opriri la benzinarii pentru realimentare (dupa fiecare oprire la o benzinarie, masina pleaca cu rezervorul plin).

Distantele catre benzinarii se reprezinta printr-o lista de forma $0 < d1 < d2 < \ldots < dn$, unde di ($1 <= i <= n$) reprezinta distanta de la A la benzinaria i. Pentru simplitate, se considera ca localitatea A se afla la 0, iar $dn = D$ (localitatea B se afla in acelasi loc cu ultima benzinarie).

Se garanteaza ca exista o planificare valida a opririlor astfel incat sa se poata ajunge la localitatea B.

$n = 5$

$m = 10$

$d = (2, 8, 15, 25, 30)$

Raspunsul este 3, efectuand 3 opriri la a 2-a, a 3-a, respectiv a 4-a benzinarie.

Teme la ACS

Pe parcursul unui semestru, un student are de rezolvat n teme (nimic nou pana aici…). Se cunosc enunțurile tuturor celor n teme de la **începutul semestrului**.

Timpul de rezolvare pentru oricare dintre teme este de **o săptămână** și **nu** se poate lucra la mai multe teme în același timp. Pentru fiecare tema se cunoaște un termen limita $d[i]$ (exprimat în săptămâni - deadline pentru tema i) și un punctaj $p[i]$.

Nicio fracțiune din punctaj nu se mai poate obține după expirarea termenului limită.

Task-uri:

- Să se definească o planificare de realizare a temelor, în așa fel încât punctajul obținut să fie **maxim**.
- Care este complexitatea solutiei (timp + spatiu)? De ce?

index	0	1	2	3	4
deadline	6	6	2	7	7
punctaj	5	4	1	5	8

Output: $1 + 4 + 5 + 5 + 8 = 23$

Explicatie: Putem face toate temele deoarece pana ajungem la deadline-urile lor avem suficiente unitati de timp.

index	0	1	2	3	4	5	6	7
deadline	3	3	3	3	9	11	11	11
punctaj	4	9	6	5	10	4	2	6

Output: $5 + 6 + 9 + 10 + 2 + 4 + 6 = 42$

Explicatie: Pana in deadline 3 avem la dispozitie 3 unitati de timp si 4 teme. Deci sortam dupa punctaj si le includem pe cele mai valoroase: 5, 6 ,9. Pana la deadline 9 avem la dispoztie 6 unitati de timp si 4 teme. Le includem pe toate.

BONUS

Rezolvati problema Dishonest Sellers [http://codeforces.com/problemset/problem/779/C].

Hint: aici [http://codeforces.com/blog/entry/50724].

Extra

Incercati problema MaxSum [https://www.hackerrank.com/contests/test-practic-pa-2017-v1-plumbus/challenges/1-1-usoare] de la test PA 2017.

Problema 1 de la tema PA 2017. Puteti descarca enuntul si checkerul de aici [https://ocw.cs.pub.ro/courses/_media/pa/teme/pa2017_tema1.zip].

Problema 3 de la tema PA 2017. Puteti descarca enuntul si checkerul de aici [https://ocw.cs.pub.ro/courses/_media/pa/teme/pa2017_tema1.zip].

Referințe

[0] Capitolul **Greedy Algorithms** din **Introductions to Algorithms** de către T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein

[1] http://en.wikipedia.org/wiki/Greedy_algorithm [http://en.wikipedia.org/wiki/Greedy_algorithm]

[2] http://ww3.algorithmdesign.net/handouts/Greedy.pdf [http://ww3.algorithmdesign.net/handouts/Greedy.pdf]

Laborator 3: Programare Dinamică

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Autori:

- Darius Neațu (2018) [mailto:neatudarius@gmail.com]
- Radu Vișan (2018) [mailto:visanr95@gmail.com]
- Cristian Banu (2018) [mailto:cristb@gmail.com]
- Răzvan Chițu (2018) [mailto:razvan.ch95@gmail.com]

Obiective laborator

- Înțelegerea noțiunilor de bază despre programare dinamică
- Insușirea abilităților de implementare a algoritmilor bazați programare dinamică.

Precizari initiale

Toate exemplele de cod se gasesc in demo-lab03.zip.

Acestea apar incorporate si in textul laboratorului pentru a facilita parcurgerea cursiva a laboratorului.

- Toate bucatile de cod prezentate in partea introductiva a laboratorului (inainte de exercitii) au fost testate. Cu toate acestea, este posibil ca din cauza mai multor factori (formatare, caractere invizibile puse de browser etc) un simplu copy-paste sa nu fie de ajuns pentru a compila codul.
- Va rugam sa incercati si codul din arhiva **demo-lab03.zip**, inainte de a raporta ca ceva nu merge.
- Pentru orice problema legata de continutul acestei pagini, va rugam sa dati email unuia dintre responsabili.

Ce este DP?

Similar cu greedy, tehnica de programare dinamica este folosită in general pentru rezolvarea **problemelor de optimizare**. In continuare vom folosi acronimul **DP (dynamic programming)**.

De asemenea, DP se poate folosi si pentru probleme in care nu cautam un optim, cum ar fi **problemele de numarare** (vom exemplifica in lab04).

Aplicatii DP

Programarea dinamică are un **câmp larg de aplicare**, insa la PA ne vom rezuma la cateva aplicatii care vor fi mentionate pe parcursul laboratoarelor 3 si 4. De asemenea, aceasta tehnica va fi folosita si in laboratoarele de grafuri (ex. algoritmul Floyd-Warshall - pe care il veti implementa si la PA; algoritmi pe arbori etc).

Programare dinamică presupune rezolvarea unei probleme prin **descompunerea ei în subprobleme** şi rezolvarea acestora. Spre deosebire de divide et impera, subproblemele nu sunt disjuncte, ci **se suprapun**.

Pentru a evita recalcularea porțiunilor care se suprapun, rezolvarea se face pornind de la cele mai mici subprobleme şi folosindu-ne de rezultatul acestora calculăm subproblema imediat mai mare. Cele mai mici subprobleme sunt numite subprobleme unitare, acestea putând fi rezolvate într-o complexitate constantă, ex: cea mai mare subsecvență dintr-o mulțime de un singur element.

Pentru a nu recalcula soluțiile subproblemelor ce ar trebui rezolvate de mai multe ori, pe ramuri diferite, se reține soluția subproblemelor folosind o tabelă (matrice uni, bi sau multi-dimensională în funcție de problemă) cu rezultatul fiecărei subprobleme. Aceasta tehnica se numește **memoizare**.

Această tehnică determina ”valoarea” soluției pentru fiecare din subprobleme. Mergând de la subprobleme mici la subprobleme din ce în ce mai mari ajungem la soluția optimă, la nivelul întregii probleme. ”valoarea” is schimba înțelesul logic de la o problema la alta. În probleme de minimizarea costului, ”valoarea” este reprezentata de costul minim. In probleme care presupun identificarea unei componente maxime, ”valoarea” este caracterizată de dimensiunea componentei.

După calcularea valorii pentru toate subproblemele se poate determina efectiv mulțimea de elemente care compun soluția. „Reconstrucția” soluţiei se face mergând din subproblemă în subproblemă, începând de la problema cu valoarea optimă și ajungând în subprobleme unitare. Metoda și recurența variază de la problemă la problemă, dar în urma unor exerciții practice va deveni din ce în ce mai facil să le identificați.

Cei curiosi pot citi aici [https://en.wikipedia.org/wiki/Dynamic_programming#History] adevarul despre numele acestei tehnici. 😎

Ce determina DP?

Aplicând aceasta tehnică determinăm **una din soluțiile optime**, problema putând avea mai multe soluții optime. În cazul în care se dorește determinarea tuturor soluțiilor optime, algoritmul trebuie combinat cu unul de backtracking în vederea construcției soluțiilor.

In cazul problemelor de numarare, aceasta tehnica ne va furniza **numarul cautat**.

Tipar general DP

Aplicarea acestei tehnici de programare poate fi descompusă în următoarea secvență de pași:

1. Identificarea structurii și a metricilor utilizate în caracterizarea soluției optime;
2. Determinarea unei metode de calcul recursiv pentru a afla valoarea fiecărei subprobleme;
3. Calcularea “bottom-up” a acestei valori (de la subproblemele cele mai mici la cele mai mari);
4. Reconstrucția soluției optime pornind de la rezultatele obținute anterior.

Exemple clasice

Programarea Dinamică este cea mai flexibilă tehnica din programare. Cel mai ușor mod de a o înțelege presupune parcurgerea cât mai multor exemple.

Propunem cateva categorii de recurente, pe care le vom grupa astfel:

- recurente de tip **SSM** (Subsecventa de Suma Maxima)
- recurente de tip **RUCSAC**
- recurente de tip **PODM** (Parantezare Optima de Matrici)
- recurente de tip **numărat**
- recurente pe **grafuri**

Pentru o problema data, este **posibil** sa gasim **mai multe recurente corecte** (mai multe solutii posibile). Evident, criteriul de alegere intre acestea va fi cel bazat pe complexitate.

Categoria 1: SSM

Aceste recurente au o oarecare asemanare cu problema SSM (enunt + solutie).

SSM

Enunt

Fie un vector v cu n elemente intregi. O subsecventa de numere din sir este de forma: $v_i, v_{i+1}, \ldots, v_j$ ($i <= j$), avand suma asociata $s_{ij} = v_i + v_{i+1} + \ldots + v_j$. O subsecventa **nu** poate fi vida.

Cerinta

Sa se determine subsecventa de suma maxima (notata **SSM**).

Exemple

n = 6

i	1	2	3	4	5	6
v[i]	-10	2	3	-1	2	-3

Raspuns: SSM este intre 2 si 5 (pozitii). Are suma +6. ($SSM = 2, 3, -1, 2$)

Explicatie: avem numere pozitive, deci exista o solutie simpla in care putem sa alegem doar un numar pozitiv/mai multe numere pozitive de pe pozitii alaturate (adica incercam sa evitam numere negative). Cele mai lungi subsecvente cu numere pozitive sunt 2,3 si 2. Observam ca daca extindem $2, 3$ la $2, 3, -1, 2$, desi am inclus un numar negativ, suma secventei creste.

n = 4

i	1	2	3	4
v[i]	10	20	30	40

Raspuns: SSM este intre 1 si 4 (pozitii). Are suma 100. ($SSM = 10, 20, 30, 40$)

Explicatie: deoarece toate numerele sunt **pozitive**, SSM cuprinde toate numerele.

n = 4

i	1	2	3	4
v[i]	-10	-20	-30	-40

Raspuns: SSM este intre 1 si 1 (pozitii). Are suma -10. ($SSM = -10$)

Explicatie: deoarece toate numerele sunt **negative**, SSM cuprinde doar cel mai mare numar.

Rezolvare

Tipar

Tiparul acestei probleme ne sugereaza ca o solutie este obtinuta incremental, in sensul ca **putem** privi problema astfel: gasim cea mai buna solutie folosind **primele** $i-1$ elemente din sir, apoi incercam sa o **extindem** folosind elementul **i** (adica ne extindem la dreapta ~CU~ $v[i]$).

Numire recurenta

Intrucat la fiecare pas trebuie sa retinem **cea mai buna solutie** folosind un **prefix** din vectorul v, solutia va fi salvata intr-un tablou auxiliar definit astfel:

$dp[i]$ = suma subsecventei de suma maxima (**suma SSM**) folosind **doar primele i** elemente din vectorul v si care se termina pe pozitia i

Mentiuni

- Pentru a mentine o conventie, toate tablourile de acest tip din laborator vor fi notate cu **dp** (dynamic programming).
- Ca sa rezolvam problema data, trebuie sa rezolvam o multime de subprobleme
  - $dp[i]$ reprezinta **solutia** pentru problema $v[1], \ldots, v[i]$ si care se termina cu $v[i]$
- Solutia pentru problema initiala este maximul din vectorul $dp[i]$.

Gasire recurenta

Intrucat dorim ca aceasta problema sa fie rezolvabila printr-un algoritm/bucata de cod, trebuie sa descriem o metoda concreta prin care vom calcula $dp[i]$.

- **Cazul de baza**
  - In general in probleme putem avea mai multe cazuri de baza, care in principiu se leaga de valori extreme are dimensiunilor subproblemelor.
  - In cazul SSM, avem un singur caz de baza, cand avem un singur element in prefix: $dp[1] = v[1]$.
  - Explicatie: daca avem un singur element, atunci acesta formeaza singura subsecventa posibila, deci $SSM = v[1]$

- **Cazul general**
- presupune inductiv ca avem rezolvate toate subproblemele mai mici
- in cazul SSM, presupunem ca avem calculat $dp[i-1]$ si dorim sa calculam $dp[i]$ (cunoastem cea mai buna solutie folosind primele i-1 elememente si vedem daca elementul de pe pozitia i o poate imbunatati)
- la fiecare pas avem de ales daca $v[i]$ extinde cea mai buna solutie care se termina pe $v[i-1]$ sau se incepe o noua secventa cu $v[i]$
- decidem in functie de $dp[i-1]$ si $v[i]$
  - **daca** $dp[i-1] >= 0$ (cea mai buna solutie care se termina pe i - 1 are cost nenegativ)
    - extindem secventa care se termina cu v[i-1] folosind elementul v[i]: $dp[i] = dp[i-1] + v[i]$
    - Explicatie: $dp[i-1] + v[i] >= v[i]$ (inca are rost sa extind)
  - **daca** $dp[i-1] < 0$ (cea mai buna solutie care se termina pe i - 1 are cost negativ)
    - vom incepe o noua secventa cu $v[i]$, adica $dp[i] = v[i]$
    - Explicatie: $v[i] > dp[i-1] + v[i]$, deci prin extindere nu obtin solutie maxima!

Implementare recurenta

In majoritatea problemelor de DP, gasirea recurentei ocupa cea mai mare parte a timpului de rezolvare (lucru adevarat si in cazul problemelor de la PA). De aceea, faptul ca ati reusit sa scrieti pe foaie lucruri foarte complicate poate fi un indiciu ca ati pornit pe o cale gresita.

Mai jos se afla un exemplu simplu de implementare a recurentei gasite in C++.

```
// gaseste SSM pentru vectorul v cu n elemente
// pentru a mentine conventia din explicatii:
//      - elementele sunt indexate de la 0, dar le folosesc doar pe cele care incep de la 1
//                                          => v[1], ..., v[n]
int SSM(int n, vector<int> &v) {
        vector<int> dp(n + 1);    // vector cu n + 1 elemente (indexarea incepe de la 0)
                                  // am nevoie de dp[1], ..., dp[n]

        // caz de baza
        dp[1] = v[1];

        // caz general
        for (int i = 2; i <= n; ++i) {
                if (dp[i - 1] >= 0) {
                        // extinde la dreapta cu v[i]
                        dp[i] = dp[i - 1] + v[i];
                } else {
                        // incep o noua secventa
                        dp[i] = v[i];
                }
        }

        // solutia e maximul din vectorul dp
        int sol = dp[1];
        for (int i = 2; i <= n; ++i) {
                if (dp[i] > sol) {
                        sol = dp[i];
                }
        }

        return sol; // aceasta este suma asociata cu SSM
}
```

Daca dorim sa afisam si indicii intre care apare SSM, putem sa stocam si pozitia de start pentru fiecare solutie intermediara. Gasiti aceasta solutie in **demo-lab03.zip**. Hint: definiti **start[i]** = pozitia pe care a inceput subsecventa care da solutia cu cost dp[i].

Mentiuni

Intrucat aceasta solutie presupune calculul iterativ (coloana cu coloana) a matricei dp, complexitatea este liniara. De asemenea, se mai parcurge o data dp pentru a gasi maximul.

- **complexitate temporala** : $T = O(n)$
- **complexitate spatiala** : $S = O(n)$
  - desigur ca pentru problema SSM, nu era nevoie sa retinem, tablourile dp/start in memorie.
  - puteam sa construim element cu element si maximul din dp in aceleasi timp (intrucat ne trebuie ultima valoare la fiecare pas si maximul global).
  - in acest caz complexitatea spatiala devine $S = O(1)$

Pentru a ilustra toti pasii posibili intr-o astfel de problema, totul a fost prezentat cat mai simplu (NU in toate problemele putem facem simplificari de tipul "NU am nevoie sa stochez tabloul dp").

SCMAX

Enunt

Fie un vector v cu n elemente intregi. Un subsir de numere din sir este de forma: $v_{i_1}, v_{i_2}, \ldots, v_{i_k}$. Un subsir **nu** poate fi vid ($k >= 1$).

Cerinta

Sa se determine subsirul crescator maximal (notat **SCMAX**) - un subsir ordonat strict crescator si are lungime maxima (daca sunt mai multe solutii, sa se gaseasca una oarecare).

Exemple

n = 6

i	1	2	3	4	5	6
v[i]	100	12	13	-1	15	-30

Raspuns: $SCMAX = 12, 13, 15$ ($SCMAX = v[2], v[3], v[5]$).

Explicatie: Toate subsirurile ordonate strict crescator sunt:

- 100
- 12
- $12, 13$
- $12, 13, 15$
- $12, 15$
- 13
- $13, 15$
- -1
- $-1, 15$
- 15
- -30

Cel mentionat este singurul de lungime 3.

n = 6

i	1	2	3	4	5	6
v[i]	100	12	13	-1	15	14

Raspuns:

- $SCMAX = 12, 13, 15$ ($SCMAX = v[2], v[3], v[5]$).
- $SCMAX = 12, 13, 14$ ($SCMAX = v[2], v[3], v[6]$).

Explicatie: Toate subsirurile ordonate strict crescator sunt:

- 100
- 12
- $12, 13$
- $12, 13, 15$
- $12, 13, 14$
- 13
- $13, 15$
- $13, 14$
- -1
- $-1, 15$

- $-1, 14$
- 15
- 14

Cele 2 solutii indicate au ambele lungime maxima.

Rezolvare

Tipar

Verificam daca se aplica tiparul de la SSM: **gasim cea mai buna solutie folosind primele $i-1$ elemente din sir, apoi incercam sa o extindem folosind elementul i (adica ne extindem la dreapta ~CU~ $v[i]$)**.

- Daca avem cea mai buna solutie pentru intervalul $1, 2, . . , i-1$ si care se termina cu $v[i-1]$, atunci incercam sa extindem solutia cu $v[i]$ (putem daca $v[i-1] < v[i]$)
- Altfel.. Unde am putea sa il punem pe $v[i]$?
  - Pai am putea sa incercam sa il punem la finalul solutiei care se termina pe $v[i-2]$, $v[i-3]$, … sau $v[1]$

Numire recurenta

$dp[i]$ = lungimea celui mai lung subsir(**lungime SCMAX**) folosind (doar o parte) din primele i elemente din vectorul v si care se termina pe pozitia i

Mentiuni

- Ca sa rezolvam problema data, trebuie sa rezolvam o multime de subprobleme
  - $dp[i]$ reprezinta **solutia** pentru problema $v[1], \ldots, v[i]$ si care se termina cu $v[i]$
- Solutia pentru problema initiala este maximul din vectorul $dp[i]$.

Gasire recurenta

- **Cazul de baza**
  - Si in problema SCMAX, cazul pentru $i = 1$ este caz de baza.
    - daca avem un singur element, atunci avem o singura subsecventa de lungime 1, ea este solutia
    - $dp[1] = 1$
- **Cazul general**
  - presupune inductiv ca avem rezolvate toate subproblemele mai mici
  - in cazul SCMAX, presupunem ca avem calculate $dp[1], dp[2], \ldots, dp[i-1]$ si dorim sa calculam $dp[i]$ (cunoastem cea mai buna solutie folosind primele j elemente si vedem daca elementul de pe pozitia i o poate imbunatati - $j = 1 : i-1$)
  - deoarece nu stim unde e cel mai bine sa il pune pe $v[i]$ (dupa care v[j]?), incercam pentru toate valorile posibile ale lui j (unde $j = 1 : n-1$)
    - **daca** $v[j] < v[i]$, atunci subsirul crescator care se termina pe pozitia j, poate fi extins la dreapta cu elementul v[i], generand lungimea **dp[j] + 1**
      - deci dp[i] = max(dp[j] + 1), $j = 1 : i-1$ (daca nu exista un astfel de j, valoarea lui max(…) este 0)
    - Ce se intampla totusi daca nu exista un j care sa indeplineasca conditia de mai sus? Atunci $v[i]$ va forma singur un subsir crescator de lungime 1 (care poate fi la un pas

ulterior)

Reunind cele spuse mai sus:

- $dp[1] = 1$
- $dp[i] = 1 + max(dp[j])$, unde $j = 1 : i - 1$ **și** $v[j] < v[i]$

Implementare recurenta

Mai jos se afla un exemplu simplu de implementare a recurentei gasite in C++.

```
// n   = numarul de elemente din vector
// v   = vectorul dat (v[1], v[2], ..., v[n] - indexare de la 1 ca in explicatii)

void scmax(int n, vector<int> &v) {
        vector<int> dp(n + 1);   // in explicatii indexarea incepe de la 1

        // caz de baza
        dp[1] = 1;   // [ v[1] ] este singurul subsir (crescator) care se termina pe 1

        // caz general
        for (int i = 2; i <= n; ++i) {
                dp[i] = 1;   // [ v[i] ] - este un subsir (crescator) care se termina pe i

                // incerc sa il pun pe v[i] la finalul tuturor solutiilor disponibile
                // o solutie se termina cu un element v[j]
                for (int j = 1; j < i; ++j) {
                        // solutia triviala: v[i]
                        if (v[j] < v[i]) {
                                // din (..., v[j]) pot obtine (..., v[j], v[i])
                                // (caz in care prec[i] = j)

                                // voi alege j-ul curent, cand alegerea imi gaseste o solutie mai buna decat ce am deja
                                if (dp[j] + 1 > dp[i]) {
                                        dp[i] = dp[j] + 1;
                                }
                        }
                }
        }

        // solutia e maximul din vectorul dp
        int sol = dp[1], pos = 1;
        for (int i = 2; i <= n; ++i) {
                if (dp[i] > sol) {
                        sol = dp[i];
                        pos = i;
                }
        }

        return sol;
}
```

In **demo-lab03.zip** gasiti un exemplu de implementare care arata si cum puteti reconstitui SCMAX. Fata de implementarea anterioara, in aceasta versiune se foloseste un tablou auxiliar prec.

$prec[i]$ = indicele j al elementului v[j], pentru care $dp[j] + 1 == dp[i]$ (adica acel j pentru care subsirul crescator maximal care se termina cu $v[i]$ este extinderea cu un element a celui care se termina cu $v[j]$.

- daca nu exista un astfel de j, atunci $prec[i] = 0$ (prin conventie)

Mentiuni

Intrucat aceasta solutie presupune calculul iterativ (coloana cu coloana) a matricei dp, complexitatea este polinomiala (patratica - pentru fiecare element din tabloul, facem o trecere prin elementele deja calculate).

- **complexitate temporala** : $T = O(n^2)$
  - se poate obtine o solutie in complexitate $T = O(nlogn)$ daca se foloseste o cautare binara pentru a gasi elementul j dorit (ex. implemetare [https://www.infoarena.ro/job_detail/1248867?action=view-source]).
- **complexitate spatiala** : $S = O(n)$
  - NU putem obtine o complexitate spatiala mai buna, intrucat avem nevoie sa stocam cel putin vectorul dp (stocam si vectorul prec daca avem nevoie sa reconstituim SCMAX)

Categoria 2: RUCSAC

Aceste recurente au o oarecare asemanare cu problema RUCSAC - varianta discreta (enunt + solutie).

RUCSAC

Enunt

Fie un set (vector) cu n obiecte (care nu pot fi taiate - varianta discreta a problemei). Fiecare obiect i are asociata o pereche (w_i, p_i) cu semnificatia:

- w_i = $weight_i$ = greutatea obiectului cu numarul i
- p_i = $price_i$ = pretul obiectului cu numarul i
  - $w_i >= 0$ si $p_i > 0$

Gigel are la dispozitie un rucsac de **volum infinit**, dar care suporta o **greutate maxima** (notata cu W - weight knapsack).

El vrea sa gaseasca **o submultime de obiecte** pe care sa le bage in rucsac, astfel incat **suma profiturilor sa fie maxima**.

Daca Gigel baga in rucsac obiectul i, caracterizat de (w_i, p_i), atunci profitul adus de obiect este p_i (presupunem ca il vinde cu cat valoreaza obiectul).

Cerinta

Sa se determine **profitul maxim** pentru Gigel.

Exemple

$n = 5$ si $W = 10$

	1	2	3	4	5
w	3	3	1	1	2
p	6	3	2	8	5

Raspuns: **24** (profitul maxim)

Explicatie: va alege toate obiectele :D.

$n = 5$ si $W = 3$

	1	2	3	4	5
w	3	3	1	1	2

p	6	3	2	8	5

Raspuns: **13** (profitul maxim)

Explicatie: va alege obiectele cu indicii 4 si 5 (profit: 8 + 5)

Rezolvare

Tipar

Cum am transpune tiparul de la SSM/SCMAX in problema RUCSAC?

- stim care este profitul maxim pe care il obtine daca folosim
  - doar primul element
  - doar primele 2 elemente
  - …
  - doar primele $i-1$ elemente
- ajung sa ma gandesc la obiectul (elementul) i
  - este posibil ca acesta sa nu apara neaparat in solutia cea mai buna, caz in care nu il folosesc, deci solutia maxima se gaseste intre cele mentionate mai sus
  - daca folosesc elementul i caracterizat de (w_i, p_i), in primul rand acesta trebuie sa incapa in ghiozdan…
    - cum verific acest lucru?
    - o recurenta de tipul $dp[i] = \ldots$ nu va fi suficienta, pentru ca in aceasta problema am 2 dimensiuni: **obiectele** (submultimile de indici) si **greutatile** (asociate cu obiectele / submultimile de obiecte).

Numire recurenta

Intrucat la fiecare pas trebuie sa retinem **cea mai buna solutie** folosind un **prefix** din vectorul de obiecte, dar pentru ca trebuie sa punem si **o restrictie de greutate** necesara (ocupata in rucsac), solutia va fi salvata intr-un tablou auxiliar definit astfel:

$dp[i][cap]$ = profitul maxim (**profit RUCSAC**) obtinut folosind (doar o parte) din primele i obiecte si avand un rucsac de **capacitate maxima cap**

Observatii:

- NU exista restrictie daca in solutia mentionata de $dp[i][cap]$ este folosit OBLIGATORIU elementul i
- Solutia problemei se gaseste in $dp[n][W]$ (profitul maxim folosind (doar o parte) din primele n elemente - adica toate; capacitatea maxima folosita este W - adica capacitatea maxima a rucsacului).

Gasire recurenta

- **Cazul de baza**
  - Daca avem o submultime vida de obiecte selectate.
    - $dp[0][cap] = 0$
    - Explicatie: Daca nu alegem obiecte, atunci profitul este 0 indiferent de capacitate.
- **Cazul general**
  - $dp[i][cap] = ?$
  - presupune inductiv ca avem rezolvate toate subproblemele mai mici

- subprobleme mai mici inseamna sa foloseasca mai putine obiecte sau un rucsac cu capacitatea mai mica
- vedem daca prin folosirea obiectului i, obtinem cea mai buna solutie in $dp[i][cap]$
  - **NU folosesc obiectul i**
    - in acest caz, o sa alegem cea mai buna solutie formata cu celelalte $i-1$ elemente si aceeasi capacitate a rucsacului
    - solutia generata de acest caz: $dp[i][cap] = dp[i-1][cap]$
  - **folosesc obiectul i**
    - daca il folosesc, inseamna ca pentru el trebuie sa am rezervata in rucsac o capacitate egala cu w_i
      - adica cand am selectat dintre primele $i-1$ elemente, nu trebuia sa ocup mai mult de $cap - w_i$ din capacitatea rucsacului
      - fata de subproblema mentionata, castig in plus p_i (profitul pe care il aduce acest obiect
    - solutia generata de acest caz: $dp[i][cap] = dp[i-1][cap - w_i] + p_i$

Reunind cele spuse mai sus, obtinem:

- $dp[0][cap] = 0$, pentru $cap = 0 : G$
- $dp[i][cap] = max(dp[i-1], cap], dp[i-1][cap - w_i] + p_i)$
  - pentru $i = 1 : n$, $cap = 0 : W$

Implementare recurenta

Mai jos se afla un exemplu simplu de implementare a recurentei gasite in C++.

```
// n   = numarul de obiecte din colectie
// W   = capacitatea maxima a rucsacului
// (w[i], p[i]) = caracteristicile obiectului i ($i = 1 : n)

int rucsac(int n, int W, vector<int> &w, vector<int> &p) {
    // dp este o matrice de dimensiune (n + 1) x (W + 1)
    // pentru ca folosim dp[0][*] pentru multimea vida
    //                   dp[*][0] pentru situatia in care ghiozdanul are capacitate 0
    vector< vector<int> > dp(n + 1, vector<int>(W + 1, 0));

    // cazul de baza
    for (int cap = 0; cap <= W; ++cap) {
        dp[0][cap] = 0;
    }

    // cazul general
    for (int i = 1; i <= n; ++i) {
        for (int cap = 0; cap <= W; ++cap) {
            // nu folosesc obiectu i => e solutia de la pasul i - 1
            dp[i][cap] = dp[i-1][cap];

            // folosesc obiectul i, deci trebuie sa rezerv w[i] unitati in rucsac
            // inseamna ca inainte trebuie sa ocup maxim cap - w[i] unitati
            if (cap - w[i] >= 0) {
                int sol_aux = dp[i-1][cap - w[i]] + p[i];

                dp[i][cap] = max(dp[i][cap], sol_aux);
            }
        }
    }

    return dp[n][W];
}
```

Mentiuni

Intrucat aceasta solutie presupune calculul iterativ (linie cu linie) a matricei dp, complexitatea este polinomiala.

- **complexitate temporala** : $T = O(n * W)$
- **complexitate spatiala** : $S = O(n * W)$
  - daca nu ne intereseaza sa reconstituim solutia (sa afisam submultimea efectiv), atunci putem sa NU stocam toata matricea dp
  - ca sa calculam o linie, avem nevoie doar de ultima linie
  - putem sa stocam la orice moment de timp doar ultima linie si linia curenta
  - complexitatea spatiala se reduce astfel la $S = O(W)$

Exercitii

In laboratorul de astazi, implementarea exercitiilor nu va fi punctata. Cu toate acestea, daca doriti sa implementati problemele propuse spre rezolvare, puteti folosi scheletul de laborator din arhiva skel-lab03.zip.

1. Not again!

Gigel are o colectie impresionanta de monede. El ne spune ca are **n tipuri** de monede, avand un numar nelimitat de monede din fiecare tip. Cunoscand aceasta informatie (data sub forma unui vector **v** cu **n** elemente), el se intreaba care este numarul minim de monede cu care poate plati o **suma S**.

Task-uri:

- 1.1 **Determinati numarul minim de monede** (din cele pe care le are) cu care Gigel poate forma suma S.
- 1.2 Care este complexitatea solutiei (timp + spatiu)? De ce?

Este posibil ca pentru anumite valori ale lui S si v, aceasta problema sa nu aiba solutie. In acest caz raspunsul este **-1**.

$n = 4$ si $S = 12$

i	1	2	3	4
v	1	2	3	6

Raspuns: 2

Explicatie: Avem 4 tipuri de monede: 1 euro, 2 euro, 3 euro si 6 euro (lui Gigel nu ii mai place sa foloseasca RON). Avem la dispozitie oricate monede din fiecare tip. Suma 12 poate fi obtinuta in urmatoarele moduri:

- $12 = 6 + 6$
- $12 = 6 + 3 + 3$
- $12 = 6 + 3 + 2 + 1$
- $12 = 6 + 2 + 2 + 2$
- $12 = 6 + 3 + 3$
- $12 = 3 + 3 + 3 + 3$
- $12 = 3 + 3 + 3 + 2 + 1$
- $12 = 3 + 3 + 2 + 2 + 2$
- $12 = 3 + 2 + 2 + 2 + 2 + 1$
- $12 = 2 + 2 + 2 + 2 + 2 + 2$
- … (ati inteles ideea :D)

Solutia cu numar minim de monede se obtine pentru modul $6 + 6$.

$n = 3$ si $S = 11$

i	1	2	3
v	1	2	5

Raspuns: 3

Explicatie: Avem 3 tipuri de monede: 1 euro, 2 euro si 5 euro (lui Gigel nu ii mai place sa foloseasca RON). Avem la dispozitie oricate monede din fiecare tip. Suma 11 poate fi obtinuta in urmatoarele moduri:

- $11 = 5 + 5 + 1$
- $11 = 5 + 2 + 2 + 2$
- $11 = 5 + 2 + 2 + 1 + 1$
- $11 = 5 + 2 + 1 + 1 + 1 + 1$
- $11 = 5 + 1 + 1 + 1 + 1 + 1 + 1$
- $11 = 2 + 2 + 1 + 1 + 1 + 1 + 1 + 1 + 1$
- $11 = 2 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1$
- $11 = 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1$

Solutia cu numar minim de monede se obtine pentru modul $5 + 5 + 1$.

$n = 3$ si $S = 11$

i	1	2	3
v	2	4	6

Raspuns: -1

Explicatie: Nu putem forma suma 11 folosind tipurile (valorile) 2, 4, 6.

2. CMLSC

Fie doi vectori cu numere intregi: **v** cu **n** elemente si **w** cu **m** elemente. Sa se gaseasca **cel mai lung subsir comun** (notat **CMLSC**) care apare in cei doi vectori. Se cere o solutie de complexitate optima. Daca exista mai multe solutii, se poate gasi oricare.

Task-uri:

- 2.1 **Determinare lungime** CMLSC. (Hint: DP)
- 2.2 **Reconstituire** CMLSC (afisati si care sunt termenii CMLSC).
- 2.3 Care este **complexitatea** solutiei (timp + spatiu)? De ce?

Rezolvati in ordine task-urile.

subsir (**subsequence** in engleza) pentru un vector **v** inseamna un alt vector $u = [v[i_1], v[i_2], \ldots, v[i_k]]]$ unde $i_1 < i_2 < \ldots < i_k$.

$n = 3$ si $m = 5$

i	1	2	3
v	6	-1	9

j	1	2	3	4	5

w	0	6	2	9	8

Raspuns: $lungime = 2$, $CMLSC = [6, 9]$

Explicatie: Toate subsirurile comune posibile sunt:

- $[6]$
- $[6, 9]$
- $[9]$

Solutia mentionata are lungime maxima.

$n = 8$ si $m = 5$

i	1	2	3	4	5	6	7	8
v	2	1	5	3	4	5	2	7

j	1	2	3	4	5
w	1	5	5	7	4

Raspuns: $lungime = 4$, $CMLSC = [1, 5, 5, 7]$

Explicatie: Toate subsirurile comune posibile sunt (duplicatele vor fi mentionate o singura data):

- $[1]$
- $[1, 5]$
- $[1, 7]$
- $[1, 4]$
- $[1, 5, 5]$
- $[1, 5, 7]$
- $[1, 5, 4]$
- $[1, 5, 5, 7]$
- $[5]$
- $[5, 5]$
- $[5, 7]$
- $[5, 4]$
- $[4]$

Solutia mentionata are lungime maxima.

$n = 8$ si $m = 5$

i	1	2	3	4	5	6	7	8
v	2	1	5	3	4	5	2	7

j	1	2	3	4	5
w	1	5	7	-5	4

Raspuns: $lungime = 3$, $CMLSC = [1, 5, 7]$ (exemplu de solutie)

Explicatie: Toate subsirurile comune posibile sunt (duplicatele vor fi mentionate o singura data):

- $[1]$
- $[1, 5]$
- $[1, 7]$

- $[1, 4]$
- $[1, 5, 7]$
- $[1, 5, 4]$
- $[5]$
- $[5, 7]$
- $[5, 4]$
- $[4]$

Solutii pot fi: $[1, 5, 7]$ si $[1, 5, 4]$. Pentru $[1, 5, 7]$, se observa ca sunt 2 astfel de subsiruri in vectorul v. Oricare este bun.

BONUS

Rezolvati pe infoarena problema custi [https://infoarena.ro/problema/custi].

Extra

Modificati solutia de la Rucsac prezentata in laborator pentru a obtine o complexitate spatiala mai buna (se va retine un numar minim de linii din matrice). Puteti testa solutia voastra pe infoarena la problema rucsac [https://infoarena.ro/problema/rucsac].

Se da o matrice de dimensiuni $n * m$ si Q intrebari de forma: "Care este suma din submatricea care are coltul stanga-sus (x, y) si coltul dreapta-jos (p,q)?"

Se considera proprietatea: Q este mult mai mare decat dimensiunile matricei.

Sa se raspunda in mod eficient la cele Q intrebari.

Rezolvatie pe infoarena problema joctv [https://infoarena.ro/problema/joctv].

Solutie: Se fixeaza 2 linii pentru zona dreptunghiulara. Se reduce problema la SSM in O(n). Complexitate: $O(n^3)$.

Rezolvati pe leetcode problema Interleaving String [https://leetcode.com/problems/interleaving-string/description/].

https://www.geeksforgeeks.org/cutting-a-rod-dp-13/ [https://www.geeksforgeeks.org/cutting-a-rod-dp-13/] Cutting rod [https://www.geeksforgeeks.org/cutting-a-rod-dp-13/] [0]

Partition Problem [https://www.geeksforgeeks.org/partition-problem-dp-18/]

Referințe

[0] Capitolul **Dynamic Programming** din **Introductions to Algorithms** de către T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein

[1] http://infoarena.ro/problema/ssm [http://infoarena.ro/problema/ssm]

[2] http://infoarena.ro/problema/scmax [http://infoarena.ro/problema/scmax]

[3] http://infoarena.ro/problema/rucsac [http://infoarena.ro/problema/rucsac]

[4] Colecție de probleme de programare dinamică (geeksforgeeks.com) [https://www.geeksforgeeks.org/dynamic-programming/]

[5] Probleme de interviu care se rezolvă cu PD [https://practice.geeksforgeeks.org/explore/?category%5B%5D=Dynamic%20Programming&page=1&sortBy=accuracy]

[6] Sniedovich, Moshe. Dynamic programming: foundations and principles. CRC press, 2010.

pa/laboratoare/laborator-03.txt · Last modified: 2020/03/10 00:12 by darius.neatu

Laborator 4: Programare Dinamică (continuare)

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Autori:

- Darius Neațu (2018) [mailto:neatudarius@gmail.com]
- Radu Vișan (2018) [mailto:visanr95@gmail.com]
- Cristian Banu (2018) [mailto:cristb@gmail.com]
- Răzvan Chițu (2018) [mailto:razvan.ch95@gmail.com]

Obiective laborator

- Înțelegerea noțiunilor de bază despre programare dinamică
- Insușirea abilităților de implementare a algoritmilor bazați programare dinamică.

Precizari initiale

Toate exemplele de cod se gasesc in demo-lab04.zip.

Acestea apar incorporate si in textul laboratorului pentru a facilita parcurgerea cursiva a laboratorului.

- Toate bucatile de cod prezentate in partea introductiva a laboratorului (inainte de exercitii) au fost testate. Cu toate acestea, este posibil ca din cauza mai multor factori (formatare, caractere invizibile puse de browser etc) un simplu copy-paste sa nu fie de ajuns pentru a compila codul.
- Va rugam sa incercati si codul din arhiva **demo-lab04.zip**, inainte de a raporta ca ceva nu merge. :D
- Pentru orice problema legata de continutul acestei pagini, va rugam sa dati email unuia dintre responsabili.

Ce este DP?

Similar cu greedy, tehnica de programare dinamica este folosită pentru rezolvarea **problemelor de optimizare**. In continuare vom folosi acronimul **DP (dynamic programming)**.

De asemenea, DP se poate folosi si pentru probleme in care nu cautam un optim, cum ar fi **problemele de numarare**.

Pentru restul notiunilor prezentate pana acum despre DP, va rugam sa consulati pagina laboratorului 3.

Exemple clasice

Programarea Dinamică este cea mai flexibilă tehnica din programare. Cel mai ușor mod de a o înțelege presupune parcurgerea cât mai multor exemple.

Propunem cateva categorii de recurente, pe care le vom grupa astfel:

- recurente de tip **SSM** (Subsecventa de Suma Maxima)
- recurente de tip **RUCSAC**
- recurente de tip **PODM** (Parantezare Optima de Matrici)
- recurente de tip **numărat**
- recurente pe **grafuri**

Pentru o problema data, este **posibil** sa gasim **mai multe recurente corecte** (mai multe solutii posibile). Evident, criteriul de alegere intre acestea va fi cel bazat pe complexitate.

Categoria 3: PODM

Aceste recurente au o oarecare asemanare cu problema PODM (enunt + solutie).

ATENTIE! Acest tip de recurente poate fi mai greu (decat celelalte). Puteti consulta **acasa** materialele puse la dispozitie pentru intelege mai bine aceasta categorie.

Caracteristici:

- Acest tip de problema presupune ca o putem formula ca pe o problema de tip **subinterval** $[i,j]$.
- Daca dorim sa gasim optimul pentru acest interval, va trebuie sa luam in calcul toate combinatiile de 2 subproblemele care ar fi putut genera solutie pentru probleme $[i,j]$.
- Se considera fiecare divizare in 2 subprobleme, data de intermediarul k
  - $[i,k]$ si $[k+1,j]$ sunt cele 2 subprobleme pentru care cunoastem solutiile
  - atunci o solutie pentru $[i,j]$ se poate obtine imbinandu-le pe cele doua
  - ca sa gasim solutia cea mai buna
    - vom itera prin toate valorile k posibile
    - vom alege pe cea care maximizeaza solutia problemei $[i,j]$
- Calculul se face de la intervale mici (probleme usoare - $[i,i]$ sau $[i,i+1]$) spre probleme generale (dimensiune generala - $[i,j]$). In final se ajunge si la dimensiunile initiale ($[1,n]$).
  - Privind imaginea de ansamblu, adica cum se completeaza matricea, obervam ca matricea dp se completeaza **diagonala cu diagonala**.

Exemple clasice

PODM

Enunt

Fie un produs matricial $M = M_1 M_2 \dots M_n$. Putem pune paranteze in mai multe moduri si vom obtine acelasi rezultat (inmultire asociativa), dar este posibil sa obtinem numar diferit de **inmultiri scalare** .

Matricea M_i are (prin conventie), dimensiunile $d_{i-1} d_i$.

Cerinta

Se cere sa se gaseasca o **parantezare optima de matrice** (PODM), adica sa se gaseasca o parantezare care sa minimizeze numarul de inmultiri scalare.

Exemple

$n = 3$

i	0	1	2	3
d	2	3	4	5

Raspuns: **64** (inmultiri scalare)

Explicatie: Avem 3 matrici:

- A de dimensiuni (2, 3)
- B de (3, 4)
- C de (4, 5)

In functie de ordinea efectuarii inmultirilor matriciale , numarul total de inmultiri scalare poate sa fie foarte diferit:

- $(AB)C \Rightarrow 24 + 40 = 64$ de inmultiri
  - explicatie: $X = (AB)$ genereaza $2 * 3 * 4 = 24$ inmultiri, (XC) genereaza $2 * 4 * 5 = 40$ de inmultiri
- $A(BC) \Rightarrow 60 + 30 = 90$ de inmultiri
  - explicatie: $X = (BC)$ genereaza $3 * 4 * 5 = 60$ inmultiri, (AX) genereaza $2 * 3 * 5 = 30$ de inmultiri

Rezultatul optim se obtine pentru cea de a treia parantezare: $(AB)C$.

$n = 4$

i	0	1	2	3	4
d	2	3	4	2	3

Raspuns: **48** (inmultiri scalare)

Explicatie: Avem 4 matrici:

- A de dimensiuni (2, 3)
- B de (3, 4)
- C de (4, 2)
- D de (2, 3)

In functie de ordinea efectuarii inmultirilor matriciale, numarul total de inmultiri scalare poate sa fie foarte diferit:

- $(AB)C)D \Rightarrow 24 + 16 + 12 = 52$ inmultiri
  - explicatie: $X = (AB)$ genereaza $2 * 3 * 4 = 24$ inmultiri scalare, $Y = (XC)$ genereaza $2 * 4 * 2 = 16$ inmultiri scalare, $Z = YD$ genereaza $2 * 2 * 3 = 12$ inmultiri scalare
- $(A(BC))D \Rightarrow 24 + 12 + 12 = 48$ inmultiri
  - explicatie: $X = (BC)$ genereaza $3 * 4 * 2 = 24$ inmultiri scalare, $Y = (AX)$ genereaza $2 * 3 * 2 = 12$ inmultiri scalare, $Z = YD$ genereaza 2 * 2 * 3 $= 12$ inmultiri scalare
- $(AB)(CD) \Rightarrow$ = inmultiri
  - explicatie: $X = (AB)$ genereaza $2 * 3 * 4 = 24$ inmultiri scalare, $Y = (CD)$ genereaza $4 * 2 * 3 =$ 24 inmultiri scalare, $Z = XY$ genereaza $2 * 4 * 3 = 24$ inmultiri scalare
- $A((BC)D) \Rightarrow 24 + 18 + 27 = 69$ inmultiri
  - explicatie: $X = (BC)$ genereaza $3 * 4 * 2 = 24$ inmultiri scalare, $Y = (XD)$ genereaza $3 * 2 * 3 = 18$ inmultiri scalare, $Z = AY$ genereaza $3 * 3 * 3 = 27$ inmultiri scalare
- $A(B(CD)) \Rightarrow 24 + 36 + 18 = 78$ inmultiri
  - explicatie: $X = (CD)$ genereaza $4 * 2 * 3 = 24$ inmultiri scalare, $Y = (BX)$ genereaza $3 * 4 * 3 = 36$ inmultiri scalare, $Z = AY$ genereaza $2 * 3 * 3 = 18$ inmultiri scalare

Rezultatul optim se obtine pentru cea de a treia parantezare: $((A(BC))D)$.

$n = 4$

i	0	1	2	3	4
d	13	5	89	3	34

Raspuns: **2856** (inmultiri scalare)

Explicatie: Avem 4 matrici:

- A de dimensiuni (13, 5)
- B de (5, 89)
- C de (89, 3)
- D de (3, 34)

In functie de ordinea efectuarii inmultirilor matriciale , numarul total de inmultiri scalare poate sa fie foarte diferit:

- $((AB)C)D \Rightarrow$ 10582 inmultiri
- $(AB)(CD) \Rightarrow$ 54201 inmultiri
- $(A(BC))D \Rightarrow$ 2856 inmultiri
- $A((BC)D) \Rightarrow$ 4055 inmultiri
- ...

Rezultatul optim se obtine pentru cea de a treia parantezare: $(A(BC))D$.

TIPAR

A fost descris in detaliu mai sus (cand s-a vorbit de categorie).

Numire recurenta

$dp[i][j]$ = **numarul minim de inmultiri scalare** cu care se poate obtine produsul $M_i * M_{i+1} * \ldots * M_j$

Gasire recurenta

- **Cazul de baza** :
  - $dp[i][i] = 0$
    - NU avem nici un efort daca nu avem ce inmulti.
  - $dp[i][i+1] = d_{i-1}d_id_{i+1}$
    - Daca avem doua matrice, putem doar sa le inmultim. Nu are sens sa folosim paranteze.
    - Daca inmultim 2 matrice de dimensiuni $d_{i-1} * d_i$ si $d_i * d_{i+1}$, avem costul $d_{i-1}d_id_{i+1}$
- **Cazul general**: $dp[i][j] = min(dp[i][k] + dp[k+1][j] + d_{i-1}d_kd_j)$, unde $k = i : j-1$
  - daca avem de efectuat sirul de inmultiri $M_i \ldots M_j$, atunci putem pune paranteze oriunde si sa facem inmultirile astfel $(M_i \ldots M_k)(M_{k+1} \ldots M_j)$
    - costul minim pentru $(M_i \ldots M_k)$ este $dp[i][k]$
    - costul minim pentru $(M_{k+1} \ldots M_j)$ este $dp[k+1][j]$
    - vom avea in final de inmultit 2 matrice de dimensiune $d_{i-1} * d_k$ si $d_k * d_j$, operatie care are costul $d_{i-1}d_kd_j$
  - insumam cele 3 costuri intermediare

Implementare

Puteti rezolva si testa problema PODM pe infoarena aici [https://infoarena.ro/problema/podm].

Un exemplu de implementare in C++ se gaseste mai jos.

```
// kInf este valoarea maxima - "infinitul" nostru
const unsigned long long kInf = std::numeric_limits<unsigned long long>::max();

// T = O(n ^ 3)
// S = O(n ^ 2) - stocam n x n intregi in tabloul dp
 unsigned long long solve_podm(int n, const vector<int> &d) {
    // dp[i][j] = numarul MINIM inmultiri scalare cu codare poate fi calculat produsul
    //            matricial M_i * M_i+1 * ... * M_j
    vector<vector<unsigned long long>>  dp(n + 1, vector<unsigned long long> (n + 1, kInf));

    // Cazul de baza 1: nu am ce inmulti
    for (int i = 1; i <= n; ++i) {
        dp[i][i] = 0ULL;  // 0 pe unsigned long long (voi folosi mai incolo si 1ULL)
    }

    // Cazul de baza 2: matrice d[i - 1] x d[i] inmultia cu matrice d[i] x d[i + 1]
    // (matrice pe pozitii consecutive)
    for (int i = 1; i < n; ++i) {
        dp[i][i + 1] =  1ULL * d[i - 1] * d[i] * d[i + 1];
    }

    // Cazul general:
    // dp[i][j] = min(dp[i][k] + dp[k + 1][j] + d[i - 1] * d[k] * d[j]), k = i : j - 1
    for (int len = 2; len <= n; ++len) {            // fixam lungimea intervalului (2, 3, 4, ...)
        for (int i = 1; i + len - 1 <= n; ++i) {    // fixam capatul din stanga: i
            int j = i + len - 1;                    // capatul din dreapta se deduce: j

            // Iteram prin indicii dintre capete, spargand sirul de inmultiri in doua (paranteze).
            for (int k = i; k < j; ++k) {
                // M_i * ... M_j  = (M_i * .. * M_k) * (M_k+1 *... * M_j)
                unsigned long long new_sol = dp[i][k] + dp[k + 1][j] + 1ULL * d[i - 1] * d[k] * d[j];

                // actualizam solutia daca este mai buna
                dp[i][j] = min(dp[i][j], new_sol);
            }
        }
    }

    // Rezultatul se afla in dp[1][n]: Numarul MINIM de inmultiri scalare
    // pe care trebuie sa le facem pentru a obtine produsul M_1 * ... * M_n
    return dp[1][n];
```

```
}
```

Sursa a fost scrisa pentru a fi testata pe infoarena. In cazul problemei PODM [https://infoarena.ro/problema/podm], deoarece avem o suma de foarte multe produse, rezultatul este foarte mare. Pe infoarena se cerea ca rezultatul sa fie afisat asa cum e, garantandu-se ca incape pe 64 biti.

Reamintim ca prin inmultirea/adunarea a doua variabile de tipul **int**, rezultatul poate sa nu incapa pe 32 biti. De aceea, in solutia prezentata, s-a facut cast pe 64biti.

ATENTIE! La PA, in general, vom folosi conventia $expresie \% kMod$, care va fi detaliata in capitolul urmatorul din acest laborator.

Complexitate

Intrucat solutia presupune fixarea capetelor unui subinterval (i, j), apoi alegerea unui intermediar (k), complexitatea este data de aceste 3 cicluri.

- **complexitate temporala**: $T(n) = O(n^3)$
- **complexitate spatiala**: $S(n) = O(n^2)$

Categoria 4: NUMARAT

Aceste recurente au o oarecare asemanare:

- toate numara lucruri! :p
- interesante sunt cazurile cand numarul cautat este foarte mare (altfel am putea apela la alte metode - ex. generarea tuturor candidatilor posibili cu backtracking)
  - in acest caz, deoarece numarul poate sa nu incapa pe un tip reprezentabil standard (ex. int pe 32/64 de biti), se cere (de obicei) restul impartirii numarului cautat la un numar **MOD** (vom folosi in continuare aceasta notatie).

Sfaturi / Reguli

- cand cautati o recurenta pentru o problema de numarare trebuie sa aveti grija la doua aspecte:
  - 1) sa **NU** numarati acelasi obiect de doua ori.
  - 2) sa numarati toate obiectele in cauza.
- de multe ori o problema de numarare implica o partitionare a **tuturor** posibilelor solutii dupa un anumit criteriu (relevant). Gasirea criteriului este partea esentiala pentru gasirea recurentei.

Regulile de lucru cu clase de resturi

Reamintim cateva proprietati matematice pe care ar trebui sa le aveti in vedere atunci in implementati pentru obtine corect resturile pentru anumite expresii. (corect poate sa insemne, de exemplu, sa evitati overflow :D - lucru neintuitiv cateodata).

- proprietati de de baza
  - $(a + b) \% MOD = ((a \% MOD) + (b \% MOD)) \% MOD$
  - $(a * b) \% MOD = ((a \% MOD) * (b \% MOD)) \% MOD$
  - $(a - b) \% MOD = ((a \% MOD) - (b \% MOD) + MOD) \% MOD$ (restul nu poate fi ceva negativ; in C++ **%** nu functioneaza pe numere negative)
- invers modular
  - $\frac{a}{b} \% MOD = ((a \% MOD) * (b^{MOD-2} \% MOD)) \% MOD)$
    - **DACA** MOD este prim; **DACA** a si b nu sunt multipli ai lui MOD

- **definitie** : **b** este inversul modular a lui **a** in raport cu **MOD** daca $a * b = 1 (modulo\ MOD)$
- **utilizare** : $\frac{a}{b} \% MOD = ((a \% MOD) * (invers(b) \% MOD)) \% MOD$
- **calculare** : deoarece la PA aceasta discutie are sens doar in contextul posibilitatii implementarii unei recurente DP in care folosim resturile doar pentru a evita overflow/imposibilitatea de a retine rezultatul pe tipurile standard de tip int (adica nu ne intereseaza sa dam o metoda generala pentru invers modular), vom simplifica problema - **MOD este prim!!!**
- **Mica teorema a lui Fermat**: Daca p este un numar prim si a este un număr intreg care nu este multiplu al lui p, atunci $a^{p-1} = 1 (modulo\ p)$.
  - din definitia inversului modular, reiese ca **a** si **b** nu sunt multipli ai lui **MOD**
  - introducand notatiile noastre in teorema si prelucrand obtinem
    - $a^{MOD-1} = 1 (modulo\ MOD) <=> a * (a^{MOD-2}) = 1 (modulo\ MOD)$
    - deci inversul modular al lui a (in aceste conditii specifice) este $b = a^{MOD-2}$

Reamintim ca prin inmultirea/adunarea a doua variabile de tipul int, rezultatul poate sa nu incapa pe 32 biti. E posibil sa trebuiasca sa combinam regulile cu resturi cu urmatoarele:

- C++
  - **1LL / 1ULL** - constanta 1 pe 64 biti cu semn / fara semn
    - **1LL * a * b** - am grija ca rezultatul sa nu dea overflow si sa se stocheze direct pe 64biti (cu semn)
- Java
  - **1L** - constanta 1 pe 64biti cu semn (in Java nu exista unsigned types)
    - **1L * a * b** - am grija ca rezultatul sa nu dea overflow si sa se stocheze direct pe 64biti (cu semn)

Gardurile lui Gigel

Enunt

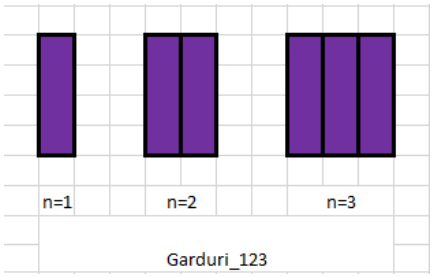
Gigel trece de la furat obiecte cu un rucsac la numarat garduri (fiecare are micile lui placeri :D). El doreste sa construiasca un gard folosind in mod repetat **un singur tip de piesa**.

O piesa are dimensiunile **4 x 1** (o unitate = 1m). Din motive irelevante pentru aceasta problema, orice gard construit trebuie sa aiba **inaltime 4m** in orice punct.

O piesa poate fi pusa in pozitie **orizontala** sau in pozitie **verticala**.

Cerinta

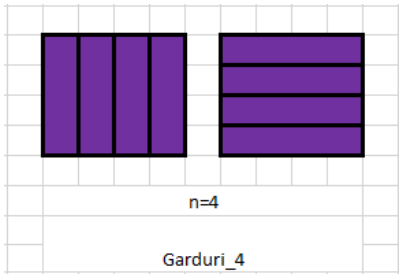
Gigel se intreaba **cate garduri de lungime n** si inaltime 4 exista? Deoarece celalalt prenume al lui este Bulănel, el intuieste ca acest numar este foarte mare, de aceea va cere **restul impartirii** acestui numar la **1009**.



$n = 1$ sau $n = 2$ sau $n = 3$

Raspuns: **1** (un singur gard)

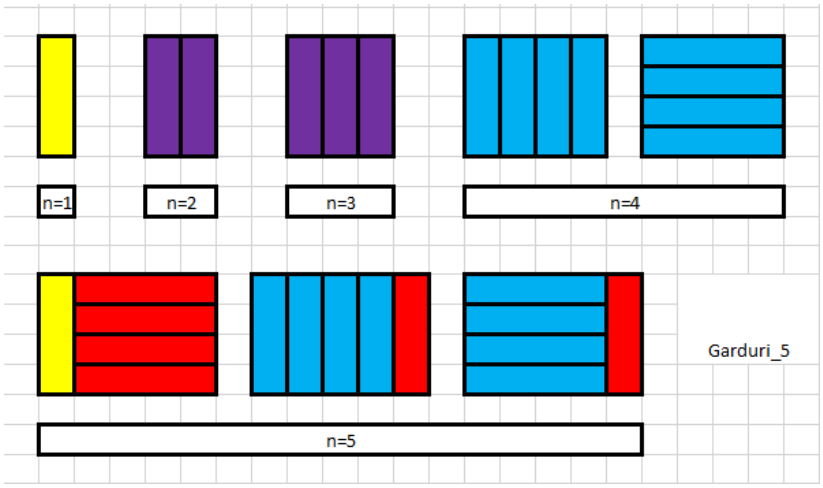
Explicatie: Se poate forma un singur gard in fiecare caz, dupa cum este ilustrat si in figura **Garduri_123**.



$n = 4$

Raspuns: **2**

Explicatie: Se pot forma 2 garduri, in functie de cum asezam piesele, dupa cum este ilustrat si in figura **Garduri_4**. Observam ca de fiecare daca cand punem o piesa in pozitie orizontala, de fapt suntem obligati sa punem 4 piese, una peste alta!

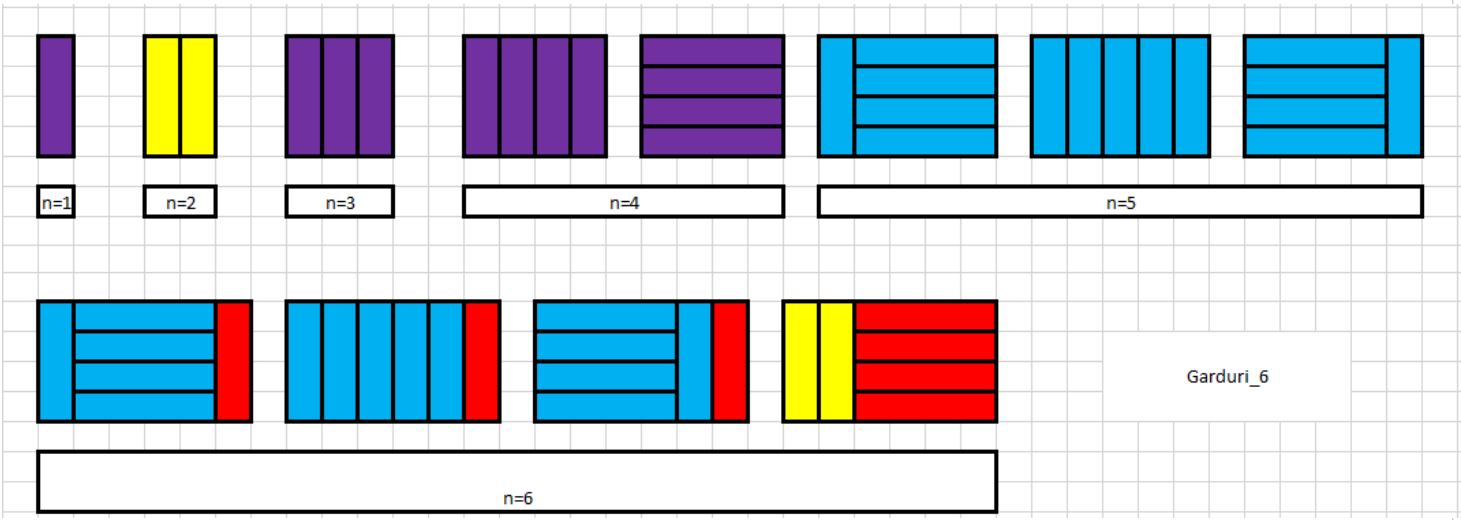


$n = 5$

Raspuns: **3**

Explicatie: Se pot forma 3 garduri, in functie de cum asezam piesele, dupa cum este ilustrat si in figura **Garduri_5**.

- daca dorim ca acest gard sa se termine cu 4 piese in pozitie **orizontala** (una peste alta - marcat cu rosu), atunci la stanga mai ramane de completat **un subgard de lungime 1**, in toate modurile posibile
- daca dorim ca acest gard sa se termine cu o piesa in pozitie **verticala** (marcat cu rosu), atunci la stanga mai ramane de completat **un subgard de lungime 4**, in toate modurile posibile



$n = 6$

Raspuns: **4**

Explicatie: Se pot forma 4 garduri, in functie de cum asezam piesele, dupa cum este ilustrat si in figura **Garduri_6**.

- daca dorim ca acest gard sa se termine cu o piesa in pozitie **verticala** (marcat cu rosu), atunci la stanga mai ramane de completat **un subgard de lungime 5**, in toate modurile posibile
- daca dorim ca acest gard sa se termine cu 4 piese in pozitie **orizontala** (una peste alta - marcat cu rosu), atunci la stanga mai ramane de completat **un subgard de lungime 2**, in toate modurile posibile

Recurenta

Numire recurenta

$dp[i]$ = numarul de garduri de lungime i si inaltime 4 (nimic special - exact ceeea ce se cere in enunt)

Raspunsul la problema este $dp[n]$.

Gasire recurenta

- **Caz de baza**
  - $dp[1] = dp[2] = dp[3] = 1$; $dp[4] = 2$
- **Caz general**
  - atunci dorim sa formam un gard de lungime i ($i >= 5$) am vazut ca putem alege cum sa punem ultima/ultimele piese
    - **DACA** alegem ca ultima piesa sa fie pusa in pozitie verticala, atunci la stanga mai ramane de completat **un subgard de lungime** $i - 1$
      - numarul de moduri in care putem face acest subgard este $dp[i - 1]$
    - **DACA** alegem ca ultima piesa sa fie in pozitie orizontala (de fapt punem 4 piese in pozitie orizontala), atunci la stanga mai ramane de completat **un subgard de lungime** $i - 4$
      - numarul de moduri in care putem face acest subgard este $dp[i - 4]$
  - $dp[i] = (dp[i - 1] + dp[i - 4]) \% MOD$
  -

Asa cum am zis in sectiunea de sfaturi si reguli [http://ocw.cs.pub.ro/courses/pa/laboratoare/laborator-04?&#sfaturireguli] vrem sa facem o **partionare** dupa un anumit **criteriu**, in cazul problemei de fata criteriul de partionare este daca gardul se termina cu o scandura verticala sau orizontala.

De asemenea tot in sectiunea sfaturi si reguli [http://ocw.cs.pub.ro/courses/pa/laboratoare/laborator-04?&#sfaturireguli] am precizat ca nu vrem **sa numaram un obiect** (un mod de a construi gardul) **de doua ori**. Recurenta noastra (dp[i] = dp[i-1] + dp[i-4]) nu ia un obiect de doua ori pentru ca orice solutie care vine din dp[i-4] e diferita de alta care vine din dp[i-1] pentru ca difera in cel putin ultima scandura asezata)

Implementare recurenta

Aici puteti vedea un exemplu simplu de implementare in C++.

```
#define MOD 1009
int gardurile_lui_Gigel(int n) {
    // cazurile de baza
    if (n <= 3) return 1;
    if (n == 4) return 2;

    vector<int> dp(n + 1); // pastrez indexarea de la 1 ca in explicatii

    // cazurile de baza
    dp[1] = dp[2] = dp[3] = 1;
    dp[4] = 2;

    // cazul general
    for (int i = 5; i <= n; ++i) {
        dp[i] = (dp[i - 1] + dp[i - 4]) % MOD;
    }
```

```
    return dp[n];
}
```

Mentionez ca am folosit expresia $dp[i] = (dp[i-1] + dp[i-4]) \% MOD$ in loc de $dp[i] = ((dp[i-1] \% MOD) + (dp[i-4] \% MOD)) \% MOD$, deoarece pe valorile anterior calculate in dp a fost deja aplicata operatia .

Am plecat cu numerele $1, 1, 1, 2$ si la fiecare pas rezultatul stocat este $\% MOD$, deci tot ce este stocat **deja** in dp este un rest in raport cu MOD. NU mai era nevoie deci sa aplica **%** si pe termenii din paranteza.

Complexitate

- **complexitate temporala**: $T = O(n)$
  - explicatie: avem o singura parcurgere in care construim tabloul dp
  - se poate obtine $T = O(logn)$ folosind exponentiere pe matrice!
- **complexitate spatiala**: $S = O(n)$
  - explicatie: stocam tabloul dp
  - se poate ontine $S = O(1)$ folosind exponentiere pe matrice!

Tehnici folosite in DP

De multe ori este nevoie sa folosim cateva tehnici pentru a obtine performanta maxima cu recurenta gasita.

In laboratorul 3 se mentiona tehnica de memoizare (in prima parte a laboratorului). In acesta ne vom rezuma la cum putem folosi cunostintele de lucru matricial pentru a favoriza implementarea unor anumite tipuri de recurente.

Exponentiere pe matrice pentru recurente liniare

Recurente liniare

O recurenta liniara in contextul laboratorului de DP este de forma:

- $dp[i] = \sum_{k=1}^{KMAX} c_k * dp[i-k]$
  - pentru **KMAX o constanta**
  - de obicei, KMAX este foarte mica comparativ cu dimensinea n a problemei
  - c_k constante reale (unele pot fi nule)

O astfel de recurenta ar insemna ca pentru a calcula **costul problemei i** , imbinam costurile problemelor $i-1, i-2, \ldots, i-k$, fiecare contribuind cu un anumit coeficient $c_1, c_2, \ldots, c_k$.

Presupunand ca nu mai exista alte specificatii ale problemei si ca avand cele KMAX cazuri de baza (primele KMAX valori ar trebui stiute/deduse prin alte reguli), atunci un algoritm poate implementa recurenta de mai sus folosind 2 cicluri de tip for (for i = 1 : n, for k = 1 : KMAX …).

- **complexitatea temporala** : $T = O(n * KMAX) = O(n)$
  - reamintim ca acea valoarea KMAX este o constanta foarte mica in compartie cu n (ex. KMAX < 100)
- **complexitatea spatiala** : $S = O(n)$
- am presupus ca avem nevoie sa retinem doar tabloul dp

Exponentiere pe matrice

Facem urmatoarele notatii:

- S_i = starea la pasul i
  - $S_i = (dp[i-k+1], dp[i-k+2], \ldots, dp[i-1], dp[i])$
- S_k = starea initiala (in care cunoaste cele k cazuri de baza)
  - $S_k = (dp[1], dp[2], \ldots, dp[k-1], dp[k])$
- C = matrice ce coeficienti constanti
  - are dimensiune $KMAX * KMAX$
  - putem pune constante in clar
  - putem pune constantele c_k care tin de problema curenta

Algoritm naiv

Putem formula problema astfel:

- S_k = este starea initiala
- pentru a obtine starea urmatoare, aplicam algoritmul urmator
  - $S_i = S_{i-1} C$

Determinare C

Pentru a determina elementele matricei C, trebuie sa ne uitam la inmultirea matriceala de mai sus si sa alegem elementele lui C astfel incat prin inmultirealui S_{i-1} cu C sa obtinem elementele din S_i.

$$\begin{bmatrix} dp[i-k+1] & \dots & dp[i-1] & dp[i] \end{bmatrix} = \begin{bmatrix} dp[i-k] & \dots & dp[i-2] & dp[i-1] \end{bmatrix} \begin{bmatrix} 0 & 0 & \dots & 0 & 0 & c_k \\ 1 & 0 & \dots & 0 & 0 & c_{k-1} \\ 0 & 1 & \dots & 0 & 0 & c_{k-2} \\ \dots & \dots & \dots & \dots & \dots & \\ 0 & 0 & \dots & 1 & 0 & c_2 \\ 0 & 0 & \dots & 0 & 1 & c_1 \end{bmatrix}$$

- ultima coloana contine toti coeficientii c_k intrucat $dp[i] = \sum_{k=1}^{KMAX} c_k * dp[i-k]$
- celelalte coloane contin doar cate o valoare nenula
  - pe coloana j vom avea valoarea 1 pe linia $j+1$ ($j = 1 : KMAX-1$)
    - cum obtinem, de exemplu, $dp[i-1]$?
    - pai avem $dp[i-1]$ chiar si in starea S_{i-1}, deci trebuie sa il copiam in starea S_i
      - copierea se realizeaza prin inmultirea cu 1
      - daca $dp[i-1]$ era pe ultima pozitiei (pozitia k) in starea S_{i-1}, in noua stare S_i este pe penultima pozitie (pozitia $k-1$)
        - deci s-a deplasat la stanga cu o pozitie!
  - in noua stare, noua pozitie este deplasata cu o unitate la stanga fata de starea precedenta
    - de aceea pe coloana j, vrem sa avem elementul 1 pe linia $j+1$ ($j = 1 : KMAX-1$)
    - cand inmultim S_{i-1} cu coloana C_j **dorim sa**
      - ce copiam?
        - valoarea $dp[i-KMAX+j]$ din S_{i-1} in S_i
        - adica sa copiam a j-a valoare de pe linie
      - unde copiam?
        - de pe pozitia $j+1$ pe pozitia j

Exponentiere logaritmica pe matrice

Algoritmul naiv de mai sus are dezavantajul ca are tot o complexitate temporala $O(n)$.

Sa executam cativa pasi de inductie pentru a vedea cum este determinat S_i.

$$S_i = S_{i-1}C$$

$$S_i = S_{i-2}C^2$$

$$S_i = S_{i-3}C^3$$

$$\dots$$

$$S_i = S_kC^{i-k}$$

In laboratorul 2 (Divide et Impera) am invatat ca putem calcula x^n in timp logaritmic. Deoarece si inmultirea matricilor este asociativa, putem calcula C^n in timp logaritmic.

Obtinem astfel o solutie cu urmatoarele complexitati:

- **complexitate temporala** : $T = O(KMAX^3 * log(n))$
  - explicatie
    - facem doar $O(logn)$ pasi, dar un pas implica inmultire de matrice
    - o inmultire de matrice patratica de dimensiune KMAX are $KMAX^3$ operatii
  - aceasta metoda este eficienta cand $KMAX << n$ (KMAX este mult mai mic decat n)
- **complexitatea spatiala** : $S = O(KMAX^3)$
  - explicatie
    - este nevoie sa stocam cateva matrici

Observatie! In ultimele calcule nu am sters contanta KMAX, intrucat apare la puterea a 3-a! $KMAX = 100$ implica $KMAX^3 = 10^6$, valoare care nu mai poate fi ignorata in practica ($KMAX^3$ poate fi comparabil cu n).

Gardurile lui Gigel (optimizare)

Dupa cum am vazut mai sus, in problema cu garduri data de Gigel solutia este o recurenta liniara:

- $dp[1] = dp[2] = dp[3] = 1$; $d[4] = 2$;
- $dp[i] = dp[i-1] + dp[i-4]$, pentru $i > 4$

Exponentiere rapida

- $k = 4$
- $S_4 = (dp[1], dp[2], dp[3], dp[4]) = (1, 1, 1, 4)$
- $S_i = (dp[i-3], dp[i-2], dp[i-1], dp[i])$
- Raspunsul se afla efectuand operatia $S_n = S_4 * C^{n-4}$, unde C are urmatorul continut:

$$C = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Mai jos se afla o implementare simplista in C++ care cuprinde toate etapele pe care trebuie sa le realizati in cod, dupa ce stiti cum arata recurenta sub forma matriceala.

```
#define MOD  1009
#define KMAX 4

// C = A * B
void multiply_matrix(int A[KMAX][KMAX], int B[KMAX][KMAX], int C[KMAX][KMAX]) {
    int tmp[KMAX][KMAX];

    // tmp = A * B
    for (int i = 0; i < KMAX; ++i) {
        for (int j = 0; j < KMAX; ++j) {
            unsigned long long sum = 0; // presupun ca suma  incape pe 64 de biti

            for (int k = 0; k < KMAX; ++k) {
                sum += 1LL * A[i][k] * B[k][j];
            }

            tmp[i][j] = sum % MOD;
        }
    }

    //  C = tmp
    memcpy(C, tmp, sizeof(tmp));
}

// R = C^p
void power_matrix(int C[KMAX][KMAX], int p, int R[KMAX][KMAX]) {
    // tmp = I (matricea identitate)
    int tmp[KMAX][KMAX];
    for (int i = 0; i < KMAX; ++i) {
        for (int j = 0; j < KMAX; ++j) {
            tmp[i][j] = (i == j) ? 1 : 0;
        }
    }

    while (p != 1) {
        if  (p % 2 == 0) {
            multiply_matrix(C, C, C);     // C = C*C
            p /= 2;                       // ramane de calculat C^(p/2)
        } else {
            // reduc la cazul anterior:
            multiply_matrix(tmp, C, tmp); // tmp = tmp*C
            --p;                          // ramane de calculat C^(p-1)
        }
    }

    // avem o parte din rezultat in C si o parte in tmp
    multiply_matrix(C, tmp, R);           // rezultat = tmp * C
}

int garduri_rapide(int n) {
    // cazurile de baza
    if (n <= 3) return 1;
    if (n == 4) return 2;

    // construiesc matricea C
    int C[KMAX][KMAX] = { {0, 0, 0, 1},
                          {1, 0, 0, 0},
                          {0, 1, 0, 0},
                          {0, 0, 1, 1}};
   // vreau sa aplic formula S_n = S_4 * C^(n-4)

   // C = C^(n-4)
   power_matrix(C, n - 4, C);

   // sol = S_4 * C = dp[n] (se afla pe ultima pozitie din S_n,
   // deci voi folosi ultima coloana din C)
   int sol = 1 * C[0][3] + 1 * C[1][3] + 1 * C[2][3] + 2 * C[3][3];
   return sol % MOD;
}
```

Remarcati faptul ca in functia de inmultire se foloseste o matrice temporara tmp. Motivul este ca vrem sa apelam functia $multiply(C, C, C)$, unde C joaca atat rol de intrare cat si de iesire. Daca am pune rezultatele direct in C, atunci am strica inputul inainte sa obtinem rezultatul.

Putem spune ca acea functie este **matrix_multiply_safe**, in sensul ca pentru orice A,B,C care respecta dimensiunile impuse, functia va calcula corect produsul.

In arhiva **demo-lab04.zip** gasiti o sursa completa in care se realizeaza:

- o verificare a faptului ca cele 2 implementari (**gardurile_lui_Gigel** si **garduri_rapide**) produc aceleasi rezultate
- un benchmark in care cele 2 implementari sunt comparate
    - pe sistem uzual (laptop) s-au obtinut urmatoarele rezulate:

```
test case: varianta simpla
n = 100000000 sol = 119; time = 0.984545 s
test case: varianta rapida
n = 100000000 sol = 119; time = 0.000021 s


test case: varianta simpla
n = 1000000000 sol = 812; time = 9.662377 s
```

```
test case: varianta rapida
n = 1000000000 sol = 812; time = 0.000022 s
```

- se observa clar diferenta intre cele 2 solutii (am confirmat ceea ce spunea si teoria: $O(n)$ vs $O(log(n))$); aceasta tehnica imbunatateste drastic o solutie gasita relativ usor.

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab04.zip.

DP or math?

Fie un sir de **numere naturale strict pozitive**. Cate **subsiruri** (submultimi nevide) au suma numerelor **para**?

subsir (**subsequence** in engleza) pentru un vector **v** inseamna un alt vector $u = [v[i_1], v[i_2], \ldots, v[i_k]]$ unde $i_1 < i_2 < \ldots < i_k$.

Task-uri:

- Se cere o **solutie folosind DP**.
- Inspectand recurenta gasita la punctul precedent, incercati sa o inlocuiti cu o **formula matematica**.
- Care este **complexitatea** pentru fiecare solutie (timp + spatiu)? Care este mai buna? De ce? :D

Deoarece rezultatul poate fi prea mare, se cere **restul impartirii** lui la 1000000007 ($10^9 + 7$).

Pentru punctaj maxim pentru aceasta problema, este necesar sa rezolvati toate subpunctele (ex. nu folositi direct formula, gasiti mai intai recurenta DP). Trebuie sa implementati **cel putin** solutia cu DP.

$n = 3$

i	1	2	3
v	2	6	4

Raspuns: 7

Explicatie: Toate subsirurile posibile sunt

- $[2]$
- $[2, 6]$
- $[2, 6, 4]$
- $[2, 4]$
- $[6]$
- $[6, 4]$
- $[4]$

Toate subsirurile de mai sus au suma para.

$n = 3$

i	1	2	3
v	2	1	3

Raspuns: 3

Explicatie: Toate subsirurile posibile sunt

- $[2]$
- $[2, 1]$
- $[2, 1, 3]$
- $[2, 3]$
- $[1]$
- $[1, 3]$
- $[3]$

Subsirurile cu suma para sunt: $[2]$, $[2, 1, 3]$, $[1, 3]$.

$n = 3$

i	1	2	3
v	3	2	1

Raspuns: 3

Explicatie: Toate subsirurile posibile sunt

- $[3]$
- $[3, 2]$
- $[3, 2, 1]$
- $[3, 1]$
- $[2]$
- $[2, 1]$

- [1]

Subsirurile cu suma para sunt: [3, 2, 1], [3, 1], [2].

Morala: există probleme pentru care găsim o soluție cu DP, dar pentru care pot exista și alte soluții mai bune (am ignorat citirea/afișarea).

In problemele de numarat, exista o **sansa** buna sa putem gasi (si) o formula matematica, care poate fi implementata intr-un mod mai eficient decat o recurenta DP.

Cate subsiruri au suma **impara**?

Expresie booleana

Se da o expresie booleana corecta cu n termeni. Fiecare din termeni poate fi unul din stringurile **true**, **false**, **and**, **or**, **xor**.

Numarati modurile in care se pot aseza paranteze astfel incat rezultatul sa fie **true**. Se respecta regulile de la logica (tabelele de adevar pentru operatiile **and**, **or**, **xor**).

Deoarece rezultatul poate fi prea mare, se cere **restul impartirii** lui la 1000000007 ($10^9 + 7$).

In schelet vom codifica cu valori de tip char cele 5 stringuri:

- **false**: 'F'
- **true**: 'T'
- **and**: '&'
- **or**: '|'
- **xor**: '^'

Functia pe care va trebui sa o implementati voi va folosi variabilele **n** (numarul de termeni) si **expr** (vectorul cu termenii expresiei).

$n = 5$ si `expr = ['T', '&', 'F', '^', 'T']` (expr = [**true and false xor true**])

Raspuns: 2

Explicatie: Exista 2 moduri corecte de a paranteza expresia astfel incat sa obtinem rezultatul **true** (1).

- `T&(F^T)`
- `(T&F)^T`

```
Complexitate temporală dorita este $O(n ^ 3)$.

Optional, se pot defini functii ajutatoare precum **is_operand**, **is_operator**, **evaluate**.
```

Pentru rezolvarea celor doua probleme ganditi-va la ce scrie in sectiunea Sfaturi / Reguli [http://ocw.cs.pub.ro/courses/pa/laboratoare/laborator-04?&#sfaturireguli].
Pentru fiecare dintre cele doua probleme facem o **partitionare dupa un anumit criteriu**.

Pentru problema **DP or math?** partitionam toate subsirurile dupa critieriul **paritatii sumei subsirului** (cate sunt pare/impare).
Pentru problema **expresie booleana** partitionam **toate parantezarile posibile dupa rezultatul lor** (cate dau true/false).

Bonus

Asistentul va alege una dintre problemele din sectiunea Extra.

Recomandam sa **NU** fie una din cele 3 probleme de la Test PA 2017. Recomandam sa le incercati dupa ce recapitulati acasa DP1 si DP2, pentru a verifica daca cunostintele acumulate sunt la nivelul asteptat.

Extra

Rezolvati problema extraterestrii [https://www.hackerrank.com/contests/test-practic-pa-2017-v1-plumbus/challenges/test-1-extraterestrii] de la Test PA 2017.

Rezolvati problema Secvente [https://www.hackerrank.com/contests/test-practic-pa-2017-v1-plumbus/challenges/test-1-secvente] de la Test PA 2017.

Rezolvati problema PA Country [https://www.hackerrank.com/contests/test-practic-pa-2017-v2-meeseeks/challenges/test-2-pa-country-medie] de la Test PA 2017.

Rezolvati pe infoarena problema iepuri [http://infoarena.ro/problema/iepuri].

Hint: Exponentiere logaritmica pe matrice

Solutie:

- $dp[0] = X; dp[1] = Y; dp[0] = Z;$
- $dp[i] = (A * dp[i-1] + B * dp[i-2] + C * dp[i-3]) \% 666013$

Pentru punctaj maxim, pentru fiecare test se foloseste ecuatia matriceala atasata. Complexitate: $O(T * log(n))$.

Rezolvati pe leetcode problema Minimum Path Sum [https://leetcode.com/problems/minimum-path-sum/description/#].

Rezolvati pe infoarena problema Lacusta [http://infoarena.ro/problema/Lacusta].

Rezolvati pe infoarena problema Suma4 [http://infoarena.ro/problema/Suma4].

Rezolvati pe infoarena problema subsir [https://www.infoarena.ro/problema/subsir].

Rezolvati pe infoarena problema 2sah [https://infoarena.ro/problema/2sah].

Hint: Exponentiere logaritmica pe matrice

O descrie detaliata se afla in arhiva OJI 2015 [http://olimpiada.info/oji2015/index.php?cid=arhiva].

Referințe

[0] Capitolul **Dynamic Programming** din **Introductions to Algorithms** de către T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein

[1] http://infoarena.ro/problema/podm [http://infoarena.ro/problema/podm]

[2] http://infoarena.ro/problema/kfib [http://infoarena.ro/problema/kfib]

pa/laboratoare/laborator-04.txt · Last modified: 2020/03/17 16:20 by radu.iacob

Laborator 5: Backtracking

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Autori:

- Radu Stochitoiu (2018) [mailto:radu.stochitoiu@gmail.com]
- Razvan Chitu (2018) [mailto:razvan.m.chitu@gmail.com]

Obiective laborator

- Intelegerea notiunilor de baza despre backtracking;
- Insusirea abilitatilor de implementare a algoritmilor bazati backtracking;
- Rezolvarea unor probleme NP-complete în timp exponential.

Precizari initiale

Toate exemplele de cod se gasesc in demo-lab05.zip. Acestea apar incorporate si in textul laboratorului pentru a facilita parcurgerea cursiva a laboratorului.

Ce este Backtracking?

Backtracking este un algoritm care cauta **una sau mai multe solutii** pentru o problema, printr-o cautare exhaustiva, mai eficienta insa in general decat o abordare „genereaza si testeaza", de tip „forta bruta", deoarece un candidat partial care nu duce la o solutie este abandonat. Poate fi folosit pentru orice problema care presupune o cautare in **spatiul starilor**. In general, in timp ce cautam o solutie e posibil sa dam de o infundatura in urma unei alegeri gresite sau sa gasim o solutie, dar sa dorim sa cautam in continuare alte solutii. In acel moment trebuie sa ne intoarcem pe pasii facuti (**backtrack**) si la un moment dat sa luam alta decizie. Este relativ simplu din punct de vedere conceptual, dar complexitatea algoritmului este exponentiala.

Importanţa – aplicaţii practice

Exista foarte multe probleme (de exemplu, probleme NP-complete sau NP-dificile) care pot fi rezolvate prin algoritmi de tip backtracking mai eficient decat prin „forta bruta" (adica generarea tuturor alternativelor si selectarea solutiilor). Atentie insa, complexitatea computationala este de cele mai multe ori exponentiala. O eficientizare se poate face prin combinarea cu tehnici de propagare a restrictiilor. Orice problema care are nevoie de parcurgerea spatiului de stari se poate rezolva cu backtracking.

Descrierea problemei si a rezolvarilor

Pornind de la strategiile clasice de parcurgere a **spatiului de stari**, algoritmii de tip backtracking practic enumera un set de candidati partiali, care, dupa completarea definitiva, pot deveni solutii potentiale ale problemei initiale. Exact ca strategiile de **parcurgere în latime/adancime** si backtracking-ul are la baza expandarea unui nod curent, iar determinarea solutiei se face într-o maniera incrementala. Prin natura sa, backtracking-ul este recursiv, iar în arborele expandat top-down se aplica operatii de tipul pruning (taiere) daca solutia partiala nu este valida.

Algoritm de baza

```
/* Domain - domeniul curent (cu un cardinal mai mic decat la pasul trecut)
Solution - solutia curenta pe care o extindem catre cea finala */
back(Domain, Solution):
    if check(Solution):
        print(Solution)
        return

    for value in Domain:
        NextSolution = Solution.push(value)
        NextDomain = Domain.erase(value)
        back(NextDomain, NextSolution)
```

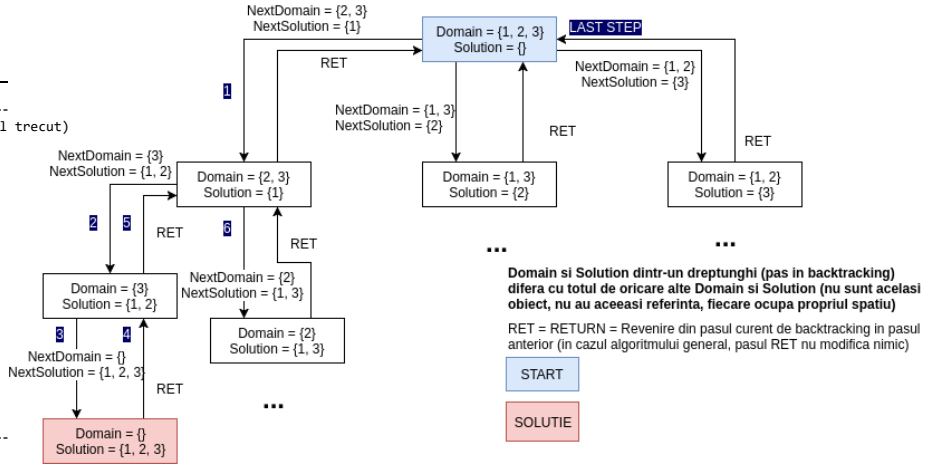
Algoritm de baza (modificat pentru transmitere prin referinta)

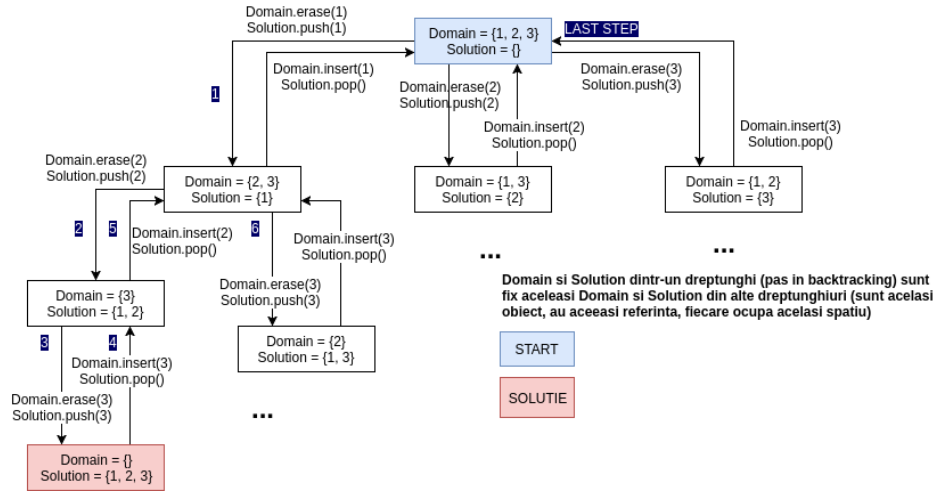
```
/* Domain - domeniul curent (cu un cardinal mai mic decat la pasul trecut)
Solution - solutia curenta pe care o extindem catre cea finala */
back(Domain, Solution):
    if check(Solution):
        print(Solution)
        return

    for value in Domain:
        /* DO */
        Solution = Solution.push(value)
        Domain = Domain.erase(value)

        /* RECURSION */
        back(Domain, Solution)

        /* UNDO */
        Solution = Solution.pop()
        Domain = Domain.insert(value)
```





Exemple clasice

Ne vom ocupa in continuare de urmatoarele probleme:

- Permutari
- Combinari
- Aranjamente
- Submultimi
- Genererare de siruri
- Problema damelor
- Problema soricelului
- Tic-Tac-Toe
- Sudoku
- Ultimate Tic-Tac-Toe

Sudoku si Ultimate Tic-Tac-Toe sunt probleme foarte grele. In general nu putem explora tot spatiul starilor pentru un input arbitrar dat.

Permutari

Enunt

Se da un numar N. Sa se genereze toate permutarile multimii formate din toate numerele de la 1 la N.

Exemple

N = 3 ⇒ M = {1, 2, 3}

Solutie:

- {1, 2, 3}
- {1, 3, 2}
- {2, 1, 3}
- {2, 3, 1}
- {3, 1, 2}
- {3, 2, 1}

Solutii

Backtracking (algoritmul in cazul general)

```
/* deoarece numerele sunt sterse din domeniu odata ce sunt folosite, solutia generata este garantata
sa nu contina duplicate. Astfel, atunci cand domeniul ajunge vid, solutia este intotdeauna corecta */
bool check(std::vector<int> solution) {
    return true;
}

void printSolution(std::vector<int> solution) {
    for (int s : solution) {
        std::cout << s << " ";
    }
    std::cout << "\n";
}

void back(std::vector<int> domain, std::vector<int> solution) {
    /* dupa ce am folosit toate elementele din domeniu putem verifica daca
    am gasit o solutie */
    if (domain.size() == 0) {
        if(check(solution)) {
            printSolution(solution);
        }
        return;
```

```
    }

    /* incercam sa adaugam in solutie toate valorile din domeniu, pe rand */
    for (unsigned int i = 0; i < domain.size(); ++i) {
        /* cream o solutie noua si un domeniu nou care sunt identice cu cele
        de la pasul curent */
        std::vector<int> newSolution(solution), newDomain(domain);

        /* adaugam in noua solutie elementul ales din domeniu */
        newSolution.push_back(domain[i]);
        /* stergem elementul ales din noul domeniu */
        newDomain.erase(newDomain.begin() + i);

        /* apelam recursiv backtracking pe noul domeniu si noua solutie */
        back(newDomain, newSolution);
    }
}

int main() {
    /* dupa ce am citit n initializam domeniul cu n elemente, numerele de la 1 la n,
    iar solutia este vida initial */
    std::vector<int> domain(n), solution;
    for (int i = 0; i < n; ++i) {
        domain[i] = i + 1;
    }

    /* apelam backtracking pe domeniul nostru, cautand solutia in vectorul solution */
    back(domain, solution);
}
```

Apelarea initiala (din “int main”) se face astfel: “back(domain, solution);”, unde domain reprezinta un vector cu elementele de la 1 la N, iar solution este un vector gol.

Nu este indicata implementarea backtracking-ului astfel deoarece este foarte costisitor din punct de vedere al memoriei, deoarece cream noi domenii si solutii la fiecare pas.

Complexitate

Solutia va avea urmatoarele complexitati:

- complexitate temporala : $T(n) = O(n * n!) = O(n!)$
  - explicatie : Complexitate generarii permutarilor, $O(n!)$, se inmulteste cu complexitatea copierii vectorilor solutie si domeniu si a stergerii elementelor din domeniu, $O(n)$
- complexitate spatiala : $S(n) = O(n^2)$
  - explicatie : Fiecare nivel de recursivitate are propria lui copie a solutiei si a domeniului. Sunt n nivele de recursivitate, deci complexitatea spatiala este $O(n * n) = O(n^2)$

Backtracking (date transmise prin referinta)

```
/* deoarece numerele sunt sterse din domeniu odata ce sunt folosite, solutia generata este garantata sa nu contina duplicate. Astfel, atunci cand domeniul ajunge vid, solutia este intotde
bool check(std::vector<int> solution) {
    return true;
}

void printSolution(std::vector<int> &solution) {
    for (int s : solution) {
        std::cout << s << " ";
    }
    std::cout << "\n";
}

void back(std::vector<int> &domain, std::vector<int> &solution) {
    /* dupa ce am folosit toate elementele din domeniu putem verifica daca
    am gasit o solutie */
    if (domain.size() == 0) {
        if(check(solution)) {
            printSolution(solution);
        }
        return;
    }

    /* incercam sa adaugam in solutie toate valorile din domeniu, pe rand */
    for (unsigned int i = 0; i < domain.size(); ++i) {
        /* retinem valoarea pe care o scoatem din domeniu ca sa o readaugam dupa
        apelarea recursiva a backtracking-ului */
        int tmp = domain[i];

        /* adaug elementul curent la potentiala solutie */
        solution.push_back(domain[i]);
        /* sterg elementul curent din domeniu ca sa il pot pasa prin referinta
        si sa nu fie nevoie sa creez alt domeniu */
        domain.erase(domain.begin() + i);

        /* apelez recursiv backtracking pe domeniul si solutia modificate */
        back(domain, solution);

        /* refac domeniul si solutia la modul in care aratau inainte de apelarea
        recursiva a backtracking-ului, adica readaug elementul eliminat in
        domeniu si il sterg din solutie */
        domain.insert(domain.begin() + i, tmp);
        solution.pop_back();
    }
}

int main() {
    /* dupa ce am citit n initializam domeniul cu n elemente, numerele de la 1 la n,
    iar solutia este vida initial */
    std::vector<int> domain(n), solution;
    for (int i = 0; i < n; ++i) {
        domain[i] = i + 1;
    }

    /* apelam backtracking pe domeniul nostru, cautand solutia in vectorul solution */
```

```
    back(domain, solution);
}
```

Apelarea initiala (din “int main”) se face astfel: “back(domain, solution);”, unde domain reprezinta un vector cu elementele de la 1 la N, iar solution este un vector gol.

Complexitate

Solutia va avea urmatoarele complexitati:

- complexitate temporala : $T(n) = O(n * n!)$
  - explicatie : Complexitate generarii permutarilor, $O(n!)$, se inmulteste cu complexitatea stergerii elementelor din domeniu, $O(n)$
- complexitate spatiala : $S(n) = O(n)$
  - explicatie : Spre deosebire de solutia anterioara, toate nivelele de recursivitate folosesc aceeasi solutie si acelasi domeniu. Complexitatea spatiala este astfel redusa la $O(n)$

Abordarea aceasta este mai eficienta decat cea generala prin evitarea folosirii memoriei auxiliare.

Backtracking (taierea ramurilor nefolositoare)

```
bool check(std::vector<int> &solution) {
    return true;
}

void printSolution(std::vector<int> &solution) {
    for (auto s : solution) {
        std::cout << s << " ";
    }
    std::cout << "\n";
}

void back(int step, int stop, std::vector<int> &domain,
        std::vector<int> &solution, std::unordered_set<int> &visited) {

    /* vom verifica o solutie atunci cand am adaugat deja N elemente in solutie,
    adica step == stop */
    if (step == stop) {
        /* deoarece am avut grija sa nu se adauge duplicate, "check()" va returna
        intotdeauna "true" */
        if(check(solution)) {
            printSolution(solution);
        }
        return;
    }

    /* Adaugam in solutie fiecare element din domeniu care *NU* a fost vizitat
    deja renuntand astfel la nevoia de a verifica duplicatele la final prin
    functia "check()" */
    for (unsigned int i = 0; i < domain.size(); ++i) {
        /* folosim elementul doar daca nu e vizitat inca */
        if (visited.find(domain[i]) == visited.end()) {
            /* il marcam ca vizitat si taiem eventuale expansiuni nefolositoare
            viitoare (ex: daca il adaug in solutie pe 3 nu voi mai avea
            niciodata nevoie sa il mai adaug pe 3 in continuare) */
            visited.insert(domain[i]);

            /* adaugam elementul curent in solutie pe pozitia pasului curent
            (step) */
            solution[step] = domain[i];

            /* apelam recursiv backtracking pentru pasul urmator */
            back(step + 1, stop, domain, solution, visited);

            /* stergem vizitarea elementului curent (ex: pentru N = 3, dupa ce
            la pasul "step = 0" l-am pus pe 1 pe prima pozitie in solutie si
            am continuat recursiv pana am ajuns la solutiile {1, 2, 3} si
            {1, 3, 2}, ne dorim sa il punem pe 2 pe prima pozitie in solutie si
            sa continuam recursiv pentru a ajunge la solutiile {2, 1, 3} etc.) */
            visited.erase(domain[i]);
        }
    }
}

int main() {
    /* dupa ce am citit n initializam domeniul cu n elemente, numerele de la 1 la n,
    iar solutia este initializata cu un vector de n elemente (deoarece o permutare
    contine n elemente) */
    std::vector<int> domain(n), solution(n);
    std::unordered_set<int> visited;
    for (int i = 0; i < n; ++i) {
        domain[i] = i + 1;
    }

    /* apelam back cu step = 0 (atatea elemente avem adaugate in solutie),
    stop = n (stim ca vrem sa adaugam n elemente in solutie pentru ca o
    permutare e alcatuita din n elemente), domain este vectorul de valori
    posibile, solution este vectorul care simuleaza stiva pe care o vom
    umple, visited este un unordered_set (initial gol) in care retinem daca
    un element din domeniu se afla deja in solutia curenta la un anumit pas */
    back(0, n, domain, solution, visited);
}
```

Apelarea initiala (din “int main”) se face astfel: “back(0, n, domain, solution, visited);”, unde domain reprezinta un vector cu elementele de la 1 la N, iar solution este un vector de n elemente, 0 este pasul curent, n este pasul la care dorim sa ne oprim, iar visited este map-ul care ne permite sa tinem cont de ce elemente au fost vizitate sau nu.

Complexitate

Solutia va avea urmatoarele complexitati:

- complexitate temporala : $T(n) = O(n * n!) = O(n!)$

- explicatie : Complexitate generarii permutarilor, O(n!), se inmulteste cu complexitatea

iterarii prin domeniu, $O(n)$

- complexitate spatiala : $S(n) = O(n)$
  - explicatie : Toate nivelele de recursivitate folosesc aceeasi solutie si acelasi domeniu.

Aceasta solutie este optima si are complexitate temporala $T(n) = O(n!)$. Nu putem sa obtinem o solutie mai buna, intrucat trebuie sa generam n! permutari.

De asemenea, este optima si din punct de vedere spatial, intrucat trebuie sa avem $S(n) = O(n)$, din cauza stocarii permutarii generate.

Combinari

Enunt

Se dau numerele N si K. Sa se genereze toate combinarile multimii formate din toate numerele de la 1 la N, luate cate K.

Exemple

N = 4, K = 2 ⇒ M = {1, 2, 3, 4}

Solutie:

- {1, 2}
- {1, 3}
- {1, 4}
- {2, 3}
- {2, 4}
- {3, 4}

Solutii

Backtracking (taierea ramurilor nefolositoare)

```
bool check(std::vector<int> &solution) {
    return true;
}

void printSolution(std::vector<int> &solution, std::vector<int> &domain, int stop) {
    for (unsigned i = 0; i < stop; ++i) {
        std::cout << domain[solution[i]] << " ";
    }
    std::cout << "\n";
}

void back(int step, int stop, std::vector<int> &domain,
        std::vector<int> &solution) {
    /* vom verifica o solutie atunci cand am adaugat deja K elemente in solutie,
    adica step == stop */
    if (step == stop) {
        /* deoarece am avut grija sa se adauge elementele doar in ordine
        crescatoare, "check()" va returna intotdeauna "true" */
        if(check(solution)) {
            printSolution(solution, domain, stop);
        }
        return;
    }

    /* daca este primul pas, alegem fiecare element din domeniu ca potential
    candidat pentru prima pozitie in solutie; altfel, pentru a elimina ramurile
    in care de exemplu {2, 1} se va genera dupa ce s-a generat {1, 2} (adica
    ar fi duplicat), vom folosi doar elementele din domeniu care sunt mai mari
    decat ultimul element adaugat in solutie (solution[step - 1]) */
    unsigned i = step > 0 ? solution[step - 1] + 1 : 0;
    for (; i < domain.size(); ++i) {
        solution[step] = i;
        back(step + 1, stop, domain, solution);
    }
}

int main() {
    /* dupa ce citim n si k initializam domeniul cu valorile de la 1 la n,
    iar solutia este initializata cu un vector de k elemente (fiindca o
    combinare de "n luate cate k" are k elemente) */
    std::vector<int> domain(n), solution(k);
    for (int i = 0; i < n; ++i) {
        domain[i] = i + 1;
    }

    back(0, k, domain, solution);
}
```

In aceasta solutie ne bazam pe faptul ca toate combinarile pot fi generate in ordine crescatoare, adica solutia {1, 3, 4} e echivalenta cu {4, 1, 3}.

Aceasta solutie este optima intrucat toate solutiile generate sunt corecte (de aceea functia check intoarce true). Deoarece problema cere obtinerea tuturor combinarilor, aceasta complexitate nu poate fi mai mica de Combinari(n, k).

Complexitate

Solutia va avea urmatoarele complexitati:

- complexitate temporala : $T(n) = O(Combinari(n, k))$

- complexitate spatiala : $S(n) = O(n + k) = O(n)$
  - explicatie : $k <= n$, deci $O(n + k) = O(n)$

Problema soricelului

Enunt

Se da un numar N si o matrice patratica de dimensiuni N x N in care elementele egale cu 1 reprezinta ziduri (locuri prin care nu se poate trece), iar cele egale cu 0 reprezinta spatii goale. Aceasta matrice are un soricel in celula (0, 0) si o bucata de branza in celula (N - 1, N - 1). Scopul soricelului e sa ajunga la bucata de branza. Afisati toate modurile in care poate face asta stiind ca acesta poate merge doar in dreapta sau in jos cu cate o celula la fiecare pas.

Exemple

- 2
- 0 1
- 0 0

Exista 1 drum posibil:

- (0, 0)→(1, 0)→(1, 1)

- 3
- 0 0 0
- 0 1 0
- 0 0 0

Exista 2 drumuri posibile:

- (0,0)→(0,1)→(0,2)→(1,2)→(2,2)
- (0,0)→(1,0)→(2,0)→(2,1)→(2,2)

- 4
- 0 0 0 1
- 0 1 1 0
- 0 0 0 0
- 0 0 0 0

Exista 4 drumuri posibile:

- (0,0)→(1,0)→(2,0)→(2,1)→(2,2)→(2,3)→(3,3)
- (0,0)→(1,0)→(2,0)→(2,1)→(2,2)→(3,2)→(3,3)
- (0,0)→(1,0)→(2,0)→(2,1)→(3,1)→(3,2)→(3,3)
- (0,0)→(1,0)→(2,0)→(3,0)→(3,1)→(3,2)→(3,3)

Solutii

Backtracking (transmitere prin referinta)

```
bool check(std::vector<std::pair<int, int> > &solution, int walls[100][100]) {
    for (unsigned i = 0; i < solution.size() - 1; ++i) {
        /* line_prev si col_prev reprezinta celula in care se afla soricelul la
        pasul i; line_next si col_next reprezinta celula in care se afla
        la pasul i + 1; trebuie sa fim siguri ca soricelul nu a ajuns pe zid
        si ca urmatoarea celula este sub sau in dreapta celulei curente */
        int line_prev = solution[i].first;
        int line_next = solution[i + 1].first;
        int col_prev = solution[i].second;
        int col_next = solution[i + 1].second;

        /* walls[x][y] == 1 inseamna ca este zid pe linia x, coloana y */
        if (walls[line_prev][col_prev] == 1 ||
                !((line_next == line_prev + 1 && col_next == col_prev) ||
                (line_next == line_prev && col_next == col_prev + 1))) {
            return false;
        }
    }

    return true;
}

void printSolution(std::vector<std::pair<int, int> > &solution) {
    for (std::pair<int, int> s : solution) {
        std::cout << "(" << s.first << "," << s.second << ")->";
    }
    std::cout << "\n";
}

void back(std::vector<std::pair<int, int> > &domain, int walls[100][100],
        std::vector<std::pair<int, int> > &solution, int max_iter) {
    /* daca am facut "max_iter" pasi ma opresc si verific daca este corecta
    solutia */
    if (solution.size() == max_iter) {
        if(check(solution, walls)) {
            printSolution(solution);
        }
        return;
    }

    /* avand domeniul initializat cu toate celulele din matrice, incercam sa
    adaugam oricare dintre aceste celule la solutie, verificand la final daca
```

```
    solutia este buna */
    for (unsigned int i = 0; i < domain.size(); ++i) {
        /* pastram elementul curent pentru a-l readauga in domeniu dupa
        apelarea recursiva */
        std::pair<int, int> tmp = domain[i];

        /* adaugam elementul curent la solutia candidat */
        solution.push_back(domain[i]);
        /* stergem elementul curent din domeniu */
        domain.erase(domain.begin() + i);

        /* apelam recursiv backtracking */
        back(domain, walls, solution, max_iter);

        /* adaugam elementul sters din domeniu inapoi */
        domain.insert(domain.begin() + i, tmp);
        /* stergem elementul curent din solutia candidat pentru a o forma pe
        urmatoarea */
        solution.pop_back();
    }
}

int main() {
    /* initializam domeniul si solutia ca vectori de perechi de int-uri;
    domeniul va contine initial toate perechile de indici posibile din
    matrice ((0, 0), (0, 1) ... (n - 1, n - 1)), iar solutia va fi initial
    vida */
    std::vector<std::pair<int, int> > domain, solution;

    fin >> n;
    for (i = 0; i < n; ++i) {
        for (j = 0; j < n; ++j) {
            /* walls[i][j] == 1 daca pe pozitia (i, j) este zid; 0 altfel */
            fin >> walls[i][j];
            domain.push_back({i, j});
        }
    }

    /* apelam back cu domeniul format initial, cu matricea de ziduri, cu
    solutia vida si cu numarul maxim de iteratii = 2 * n - 1 pentru ca
    mergand doar in dreapta si in jos, in 2 * n - 1 pasi va ajunge din
    (0, 0) in (n - 1, n - 1) */
    back(domain, walls, solution, 2 * n - 1);
}
```

Apelarea initiala (din “int main”) se face astfel: “back(domain, walls, solution, 2 * n - 1);”, unde domain reprezinta un vector cu perechi in care sunt toate celulele matricii, solution este un vector gol, walls este matricea care ne arata daca este zid sau nu pe o anumita pozitie, iar 2 * n - 1 e numarul e pasi in care ar trebui ca soricelul sa ajunga la branza pe oriunde ar merge.

Complexitate

Solutia va avea urmatoarele complexitati:

- complexitate temporala : $T(n) = O(Aranjamente(n^2, 2n - 1))$
  - explicatie: Initial in domeniu avem n^2 valori. Noi dorim sa generam toate submultimile ordonate de cate $2n - 1$ elemente. Acestea sunt tocmai aranjamentele de n^2 luate cate $2n - 1$.
- complexitate spatiala : $S(n) = O(n^2)$
  - explicatie: Trebuie sa stocam informatie despre drum, care are $2n - 1$ celule; stocam domeniul care are n^2 elemente

Backtracking (taierea ramurilor nefolositoare)

```
bool check(std::vector<std::pair<int, int> > &solution) {
    return true;
}

void printSolution(std::vector<std::pair<int, int> > &solution) {
    for (std::pair<int, int> s : solution) {
        std::cout << "(" << s.first << "," << s.second << ")->";
    }
    std::cout << "\n";
}

void back(int step, int stop, int walls[100][100],
        std::vector<std::pair<int, int> > &solution, int line_moves[2],
        int col_moves[2]) {
    /* ne oprim dupa ce am ajuns la pasul "stop" si verificam daca solutia este
    corecta */
    if (step == stop) {
        /* deoarece am eliminat ramurile nefolositoare am ajuns la o solutie care
        sigur este corecta */
        if(check(solution)) {
            printSolution(solution);
        }
        return;
    }

    /* daca este primul pas stiu ca soricelul este in pozitia (0, 0) */
    if (step == 0) {
        /* adaugam (0, 0) la solutia candidat */
        solution.push_back({0, 0});

        /* apelam backtracking recursiv la pasul urmator */
        back(step + 1, stop, walls, solution, line_moves, col_moves);

        /* scoatem (0, 0) din solutie */
        solution.pop_back();
        return;
    }

    /* sunt doar doua mutari pe care le pot face intr-un pas: dreapta si jos;
    acestea sunt encodate prin vectorii de directii line_moves[2] = {0, 1} si
    col_moves[2] = {1, 0} care reprezinta la indicele 0 miscarea in dreapta, iar
    la indicele 1 miscarea in jos */
    for (unsigned int i = 0; i < 2; ++i) {
```

```
        /* cream noua linie si noua coloana cu ajutorul vectorilor de directii */
        int new_line = solution.back().first + line_moves[i];
        int new_col = solution.back().second + col_moves[i];
        int n = (stop + 1) / 2;

        /* daca linia si coloana sunt valide (nu ies din matrice) si nu este
        zid pe pozitia lor, putem continua pe acea celula */
        if (new_line < n && new_col < n && walls[new_line][new_col] == 0) {
            /* adaugam noua celula in solutia candidat;
            NOTE: {new_line, new_col} este echivalent cu
            std::pair<int, int>(new_line, new_col) si se numeste "initializer
            list", feature in C++11 */
            solution.push_back({new_line, new_col});

            /* apelam backtracking recursiv la pasul urmator */
            back(step + 1, stop, walls, solution, line_moves, col_moves);

            /* scoatem celula adaugata din solutie */
            solution.pop_back();
        }
    }
}

int main() {
    /* initializam solutia ca vector de perechi de int-uri */
    std::vector<std::pair<int, int> > solution;

    fin >> n;
    for (i = 0; i < n; ++i) {
        for (j = 0; j < n; ++j) {
            /* citim matricea zidurilor; 1 pentru zid, 0 altfel */
            fin >> walls[i][j];
        }
    }

    /* apelam back cu step = 0, stop = 2 * n - 1 deoarece in 2 * n - 1
    pasi soricelul va ajunge la branza, vectorul de ziduri, vectorul in
    care vom stoca solutia, vectorii de directii line_moves[2] = {0, 1} si
    col_moves[2] = {1, 0}; nu avem nevoie de domeniu deoarece folosind
    vectorii de directii vom sti din ultima pozitie pusa in solutie cele
    doua solutii in care putem merge, astfel domeniul nostru va fi alcatuit
    din doua solutii la fiecare pas (daca ultima pozitie din solutie a fost
    (5, 7) => domeniul pasului curent = {(5 + 0, 7 + 1) si (5 + 1, 7 + 0)}
    care este egal cu {(5, 8), (6, 7)}. */
    back(0, 2 * n - 1, walls, solution, line_moves, col_moves);
}
```

Pentru aceasta abordare la fiecare pas de backtracking vom merge doar in doua directii, in loc sa mergem in oricare celula din matrice, lucru care imbunatateste semnificativ complexitatea temporala.

Complexitate

Solutia va avea urmatoarele complexitati:

- complexitate temporala : $T(n) = O(2^{2n})$
  - explicatie: avem de urmat un sir de $2n - 1$ mutari, iar la fiecare pas avem 2 variante posibile
- complexitate spatiala : $S(n) = O(n)$
  - explicatie: stocam maximum $2n - 1$ casute

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab05.zip.

Aranjamente

Fie N si K doua **numere naturale strict pozitive**. Se cere afisarea tuturor aranjamentelor de N elemente luate cate K din multimea {1, 2, …, N}.

Fie N = 3, K = 2 ⇒ M = {1, 2, 3}

Solutie:

- {1, 2}
- {1, 3}
- {2, 1}
- {2, 3}
- {3, 1}
- {3, 2}

Se doreste o complexitate $T(n, k) = A(n, k)$.

Folositi-va de problema **Permutari**.

Solutiile se vor genera in ordine lexico-grafica!

Checkerul asteapta sa le stocati in aceasta ordine.

Submultimi

Fie N un **numar natural strict pozitiv**. Se cere afisarea tuturor submultimilor multimii {1, 2, …, N}.

Fie N = 4 ⇒ M = {1, 2, 3, 4}

Solutie: {} - multimea vida {1} {1, 2} {1, 2, 3} {1, 2, 3, 4} {1, 2, 4} {1, 3} {1, 3, 4} {1, 4} {2} {2, 3} {2, 3, 4} {2, 4} {3} {3, 4} {4}

Se doreste o complexitate $T(n) = O(2^n)$.

Folositi-va de problema **Combinari**.

Solutiile se vor genera in ordine lexico-grafica!

Checkerul asteapta sa le stocati in aceasta ordine.

Problema damelor

Problema damelor (sau problema reginelor) trateaza plasarea a 8 regine de sah pe o tablă de șah de dimensiuni 8 x 8 astfel incat sa nu existe doua regine care se ameninta reciproc. Astfel, se cauta **o solutie** astfel incat nicio pereche de doua regine sa nu fie pe acelasi rand, pe aceeasi coloana, sau pe aceeasi diagonala. Problema cu opt regine este doar un caz particular pentru problema generala, care presupune plasarea a N regine pe o tablă de sah N x N în aceleasi conditii. Pentru aceasta problema, există solutii pentru toate numerele naturale N cu excepția lui N = 2 si N = 3.

Fie N = 5

Solutie:

X	-	-	-	-
-	-	X	-	-
-	-	-	-	X
-	X	-	-	-
-	-	-	X	-

X reprezinta o dama, - reprezinta spatiu gol.

E nevoie sa facem backtracking pe matrice sau e suficient pe vector?

Se va cauta o singura solutie (**oricare** solutie corecta), care va fi returnata sub format unui vector cu $n + 1$ elemente.

Solutia este $sol[0], sol[1], \ldots, sol[n]$, unde $sol[i]$ = coloana unde vom plasa regina de pe linia i.

Elementul 0 este nefolosit, dorim sa pastram conventia cu indexare de la 1.

Generare de siruri

Vi se da o lista de caractere si o lista de frecvente (pentru caracterul de pe pozitia i, frecventa de pe pozitia i). Vi se cere sa generati toate sirurile care se pot forma cu aceste caractere si aceste frecvente stiind ca nu pot fi mai mult de K aparitii consecutive ale aceluiasi caracter.

Fie caractere[] = {'a', 'b', 'c'}, freq[] = {1, 1, 2}, K = 5

Solutie:

- abcc
- acbc
- accb
- bacc
- bcac
- bcca
- cabc
- cacb
- cbac
- cbca
- ccab
- ccba

Fie caractere[] = {'b', 'c'}, freq[] = {3, 2}, K = 2

Solutie:

- bbcbc
- bbccb
- bcbbc
- bcbcb
- bccbb
- cbbcb
- cbcbb

Solutiile se vor genera in ordine lexico-grafica!

Checkerul asteapta sa le stocati in aceasta ordine.

Bonus

Problema damelor (AC3)

Aplicati AC3 pe problema damelor.

Algoritmul AC-3 (Arc Consistency Algorithm) este de obicei folosit in probleme de satisfacere a constrangerilor (CSP). Acesta sterge arcele din arborele de stari care sigur nu se vor folosi niciodata.

AC-3 lucreaza cu:

- constrangeri
- variabile
- domenii de variabile

O variabila poate lua orice valoare din domeniul sau la orice pas. O constrangere este o relatie sau o limitare a unor variabile.

Exemplu AC-3

Consideram A, o variabila ce are domeniul D(A) = {0, 1, 2, 3, 4, 5, 6} si B o variabila ce are domeniul D(B) = {0, 1, 2, 3, 4}. Cunoastem constrangerile: C1 = "A trebuie sa fie impar" si C2 = "A + B trebuie sa fie egal cu 5".

Algoritmul AC-3 va elimina in primul rand toate valorile pare ale lui A pentru a respecta C1 ⇒ D(A) = {1, 3, 5}. Apoi, va incerca sa satisfaca C2, asa ca va pastra in domeniul lui B toate valorile care adunate cu valori din D(A) pot da 5 ⇒ D(B) = {0, 2, 4}.

AC-3 a redus astfel domeniile lui A si B, reducand semnificativ timpul folosit de algoritmul backtracking.

Extra

Immortal

Enunt [https://www.infoarena.ro/problema/immortal]

OJI 2010 - clasele 11-12 [http://olimpiada.info/oji2010/index.php?cid=arhiva]

Ultimate Tic Tac Toe

The AI Games - Ultimate Tic Tac Toe [http://theaigames.com/competitions/ultimate-tic-tac-toe]

pa/laboratoare/laborator-05.txt · Last modified: 2020/03/21 19:44 by darius.neatu

Laborator 06: Minimax

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

- Insusirea unor cunostinte de baza despre **teoria jocurilor** precum si despre **jocurile de tip joc de suma zero (suma nula, zero-sum games)**
- Insusirea abilitatii de rezolvare a problemelor ce presupun cunoasterea si exploatarea conceptului de joc de suma zero (zero-sum game);
- Insusirea unor cunostinte elementare despre algoritmii necesari rezolvarii unor probleme de joc de suma zero (zero-sum game).

Precizari initiale

Un curs foarte bine explicat este pe canalul de YouTube de la MIT. Va sfatuim sa vizionati integral Search: Games, Minimax, and Alpha-Beta [https://www.youtube.com/watch?v=STjW3eH0Cik&t=2480s&list=PLqUJoA5RTkI7UbJXJaHX80FotR1QJWHJt&index=1] inainte sa parcurgeti materialul de pe ocw.

Importanţă – aplicaţii practice

Algoritmul **Minimax** si variantele sale imbunatatite (**Negamax**, **Alpha-Beta** etc.) sunt folosite in diverse domenii precum teoria jocurilor (Game Theory), teoria jocurilor combinatorice (Combinatorial Game Theory – CGT), teoria deciziei (Decision Theory) si statistica.

Astfel, diferite variante ale algoritmului sunt necesare in proiectarea si implementarea de aplicatii legate de inteligenta artificiala, economie, dar si in domenii precum stiinte politice sau biologie.

Descrierea problemei și a rezolvărilor

Algoritmii Minimax permit abordarea unor probleme ce tin de teoria jocurilor combinatorice. CGT este o ramura a matematicii ce se ocupa cu studierea jocurilor in doi (two-player games), in care participantii isi modifica rand pe rand pozitiile in diferite moduri, prestabilite de regulile jocului, pentru a indeplini una sau mai multe conditii de castig.

Exemple de astfel de jocuri sunt: sah, go, dame (checkers), X si O (tic-tac-toe) etc.

CGT nu studiaza jocuri ce presupun implicarea unui element aleator (sansa) in derularea jocului precum poker, blackjack, zaruri etc. Astfel decizia abordarii unor probleme rezolvabile prin metode de tip Minimax se datoreaza in principal simplitatii atat conceptuale, cat si raportat la implementarea propriu-zisa.

Minimax

Strategia pe care se bazeaza ideea algoritmului este ca jucatorii implicati adopta urmatoarele strategii:

- Jucatorul 1 (**maxi**) va incerca mereu sa-si **maximizeze** propriul castig prin mutarea pe care o are de facut;

- Jucatorul 2 (**mini**) va incerca mereu sa **minimizeze** castigul jucatorului 1 la fiecare mutare.

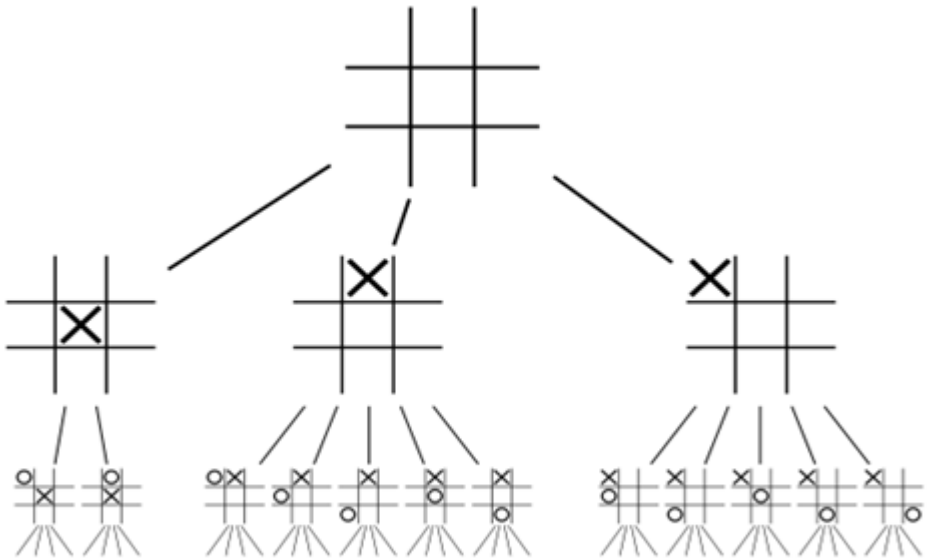
De ce merge o astfel de abordare? Dupa cum se preciza la inceput, discutia se axeaza pe jocuri de suma zero (zero-sum game). Acest lucru garanteaza, printre altele, ca orice castig al Jucatorului 1 este egal cu modulul sumei pierdute de Jucatorul 2. Cu alte cuvinte cat pierde Jucator 2, atat castiga Jucator 1. Invers, cat pierde Jucator 1, atat castiga Jucator 2. Sau

$$Win_{Player_1} = |Loss_{Player_2}|$$

$$|Loss_{Player_1}| = Win_{Player_2}$$

Reprezentarea spatiului solutiilor

In general spatiul solutiilor pentru un joc in doi de tip zero-sum se reprezinta ca un **arbore**, fiecarui nod fiindu-i asociata o stare a jocului in desfasurare (game state). Pentru exemplul nostru de X si O putem considera urmatorul arbore (partial) de solutii, ce corespunde primelor mutari ale lui X, respectiv O:



Metodele de reprezentare a arborelui variaza in functie de paradigma de programare aleasa, de limbaj, precum si de gradul de optimizare avut in vedere.

Avand notiunile de baza asupra strategiei celor doi jucatori, precum si a reprezentarii spatiului solutiilor problemei, putem formula o prima varianta a algoritmului Minimax:

```
int evaluate(stare);   // functia returneaza un scor asociat cu starea
                       // functia returneaza mereu scorul din perspectiva lui maxi


void apply_move(move); // functia modifica starea curenta: executa miscarea move
void undo_move(move);  // functia restaureaza starea anterioara (de dinainte de executa lui move)

// alege cea mai buna mutare pentru jucatorul maxi
// functia returneaza best_score pentru maxi
int maxi(int depth) {
    // daca jocul s-a terminat sau am atins nivelul maxim de recursivitate ales
    if (gameOver() || depth == 0 ) {
        return evaluate();            // ne oprim si evaluam starea curenta
    }

    int max = -oo;                    // jucatorul maxi doreste sa-si maximizeze

    // incercam pe rand fiecare care miscare posibila move
    for (move : all_moves) {
        apply_move(move);         // executa move
```

```
        // incercam sa simulam jocul mai departe:
        // daca maxi face move, ce ar face mini?
        int score = mini(depth - 1);

        // dintre toate variantele pe care mini le permite (!!!),
        // maxi o va alege pe cea cu scor maxim
        if (score > max) {
            max = score;
        }

        undo_move(move);         // restaureaza starea de dinainte de move
    }

    // cel mai bun scor pe care il poate obtine maxi este max
    // (s-a tinut cont si de ce i-ar permite mini, avand in vedere ca si el joaca optim)
    return max;
}

// alege cea mai buna mutare pentru jucatorul mini
// functia returneaza best_score pentru mini
int mini(int depth) {
    // daca jocul s-a terminat sau am atins nivelul maxim de recursivitate ales
    if (gameOver() || depth == 0 ) {
        return -evaluate();            // ne oprim si evaluam starea curenta
    }

    int min = +oo;                    // jucatorul mini doreste sa minimizeze scorul lui maxi

    // incercam pe rand fiecare care miscare posibila move
    for (move : all_moves) {
        apply_move(move);         // executa move

        // incercam sa simulam jocul mai departe:
        // daca mini face move, ce ar face maxi?
        int score = maxi(depth - 1);

        // dintre toate variantele pe care maxi le permite (!!!),
        // mini o va alege pe cea cu scor minim
        if (score < min) {
            min = score;
        }

        undo_move(move);         // este o functie care incearca sa refaca
                                 // starea de dinainte de aplicarea lui move
    }

    // cel mai bun scor pe care il poate obtine mini este min
    // (s-a tinut cont si de ce i-ar permite maxi, avand in vedere ca si el joaca optim)
    return min;
}
```

Argumentarea utilizarii unei adancimi maxime

Datorita **spatiului de solutii mare**, de multe ori coplesitor ca volum, o inspectare completa a acestuia nu este fezabila si devine impracticabila din punctul de vedere al timpului consumat sau chiar a memoriei alocate (se vor discuta aceste aspecte in paragraful legat de complexitate).

Astfel, de cele mai multe ori este preferata o abordare care parcurge arborele numai pana la o anumita **adancime maxima („depth”)**. Aceasta abordare permite examinarea arborelui destul de mult pentru a putea lua decizii minimalist coerente in desfasurarea jocului.

Totusi, **dezavantajul major** este ca pe termen lung se poate dovedi ca decizia luata la adancimea depth nu este global favorabila jucatorului in cauza.

De asemenea, se observa recursivitatea indirecta. Prin conventie acceptam ca **inceputul algoritmului** sa fie cu functia maxi. Astfel, se analizeaza succesiv diferite stari ale jocului din punctul de vedere al celor doi jucatori pana la adancimea depth. Rezultatul intors este scorul final al miscarii celei mai bune.

Negamax

Negamax este o varianta a minimax, ce se bazeaza pe urmatoarea observatie: intr-un joc cu suma zero castigul unui jucator este egal cu modulul sumei pierdute de celalalt jucator si invers.

Intr-adevar putem spune ca jucatorul mini incearca de fapt sa maximizeze in modul suma pierduta de maxi. Astfel putem formula urmatoarea implementare ce profita de observatia de mai sus.

Nota: putem exprima aceasta observatie si pe baza formulei **max(a, b) = -min(-a, -b)**.

```
int evaluate(stare);   // functia returneaza un scor asociat cu starea
                       // functia returneaza mereu scorul din perspectiva jucatorului CURENT!


void apply_move(move); // functia modifica starea curenta: executa miscarea move
void undo_move(move);  // functia restaureaza starea anterioara (de dinainte de executa lui move)

// alege cea mai buna mutare pentru jucatorul CURENT (EU)
// functia returneaza best_score
int negamax(int depth) {
    // daca jocul s-a terminat sau am atins nivelul maxim de recursivitate ales
    if (gameOver() || depth == 0 ) {
       return evaluate();            // ne oprim si evaluam starea curenta
    }

    int max = -oo;                    // jucatorul CURENT doreste sa-si  maximizeze scorul

    // incercam pe rand fiecare care miscare posibila move
    for (move : all_moves) {
        apply_move(move);         // executa move

        // incercam sa simulam jocul mai departe:
        // daca jucatorul CURENT face face move, ce ar face ADVERSARUL?
        int score = -negamax(depth - 1);  // cel mai bine pentru el,
                                          // este cel mai rau pentru mine si invers

        // dintre toate variantele pe care ADVERSARUL le permite (!!!),
        // EU (jucatorul CURENT) o voi alege pe cea cu scor maxim
        if (score > max) {
            max = score;
        }

        undo_move(move);         // restaureaza starea de dinainte de move
    }

    // cel mai bun scor pe care il pot obtine EU este max
    // (s-a tinut cont si de ce mi-ar permite ADVERSARUL, avand in vedere ca si el joaca optim)
    return max;
}
```

Se observa direct avantajele acestei formulari fata de Minimax-ul standard prezentat anterior:

- **claritatea** si **eleganta** sporita a codului
- usurinta in **intretinerea** si **extinderea** functionalitatii

Din punctul de vedere al complexitatii temporale, Negamax nu difera absolut deloc de Minimax (ambele examineaza acelasi numar de stari in arborele de solutii).

Putem concluziona ca este de **preferat** o implementare ce foloseste **negamax** fata de una bazata pe minimax in rezolvarea unor probleme ce tin de aceasta tehnica.

Alpha-beta pruning

Pana acum s-a discutat despre algoritmii Minimax / Negamax. Acestia sunt algoritmi exhaustivi (**exhausting search algorithms**). Cu alte cuvinte, ei gasesc solutia optima examinand intreg spatiul de solutii al problemei. Acest mod de abordare este extrem de ineficient in ceea ce priveste efortul de calcul necesar, mai ales considerand ca extrem de multe stari de joc inutile sunt explorate (este vorba de acele stari care nu pot fi atinse datorita incalcarii principiului de maximizare a castigului la fiecare runda).

O imbunatatire substantiala a minimax/negamax este **Alpha-beta pruning** (**taiere alfa-beta**). Acest algoritm incearca sa optimizeze mini/negamax profitand de o observatie importanta: **pe parcursul examinarii arborelui de solutii se pot elimina intregi subarbori, corespunzatori unei miscari m, daca pe parcursul analizei gasim ca miscarea m este mai slaba calitativ decat cea mai buna miscare curenta.**

Astfel, consideram ca pornim cu o prima miscare M1. Dupa ce analizam aceasta miscare in totalitate si ii atribuim un scor, continuam sa analizam miscarea M2. Daca in analiza ulterioara gasim ca adversarul are cel putin o miscare care transforma M2 intr-o miscare mai slaba decat M1 atunci orice alte variante ce corespund miscarii M2 (subarbori) nu mai trebuie analizate.

De ce? Pentru ca stim ca exista **cel putin** o varianta in care adversarul obtine un castig mai bun decat daca am fi jucat miscarea M1.

Nu conteaza exact cat de slaba poate fi miscarea M2 fata de M1. O analiza amanuntita ar putea releva ca poate fi si mai slaba decat am constatat initial, insa acest lucru este irelevant.

De ce insa ignoram intregi subarbori si miscari potential bune numai pentru o miscare slaba gasita? Pentru ca, in conformitate cu **principiul de maximizare al castigului** folosit de fiecare jucator, adversarul va alege exact acea miscare ce ii va da un castig maximal. Daca exista o varianta si mai buna pentru el este irelevant, deoarece noi suntem interesati daca cea mai slaba miscare buna a lui este mai buna decat miscarea noastra curent analizata.

O observatie foarte importanta se poate face analizand **modul de functionare** al acestui algoritm: este extrem de importanta **ordonarea miscarilor dupa valoarea castigului**.

In **cazul ideal** in care cea mai buna miscare a jucatorului curent este analizata prima, toate celelalte miscari, fiind mai slabe, vor fi eliminate din cautare timpuriu.

In **cazul cel mai defavorabil** insa, in care miscarile sunt ordonate crescator dupa castigul furnizat, Alpha-beta are aceeasi compelxitate cu Mini/Nega-max, neobtinandu-se nicio imbunatatire.

In **medie** se constata o imbunatatire vizibila a algoritmului Alpha-beta fata de Mini/Nega-max.

Rolul miscarilor analizate la inceput presupune stabilirea unor **plafoane de minim si maxim** legate de cat de bune/slabe pot fi miscarile.

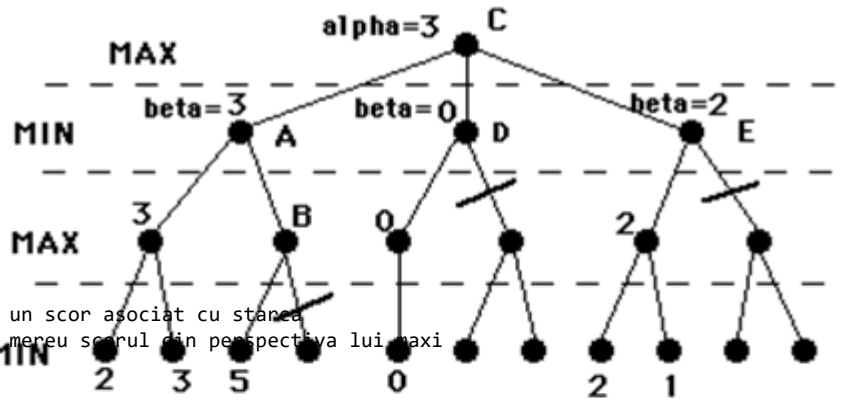
Astfel, plafonul de minim (Lower Bound), numit **alpha** stabileste ca o miscare nu poate fi mai slaba decat valoarea acestui plafon. Plafonul de maxim (Upper Bound), numit **beta**, este important doarece el foloseste la a stabili daca o miscare este prea buna pentru a fi luata in considerare. Depasirea plafonului de maxim inseamna ca o miscare este atat de buna incat adversarul nu ar fi permis-o, adica mai sus in arbore exista o miscare pe care ar fi putut s-o joace pentru a nu ajunge in situatia curent analizata.

Astfel alpha si beta furnizeaza o fereastra folosita pentru a filtra miscarile posibile pentru cei doi jucatori. Evident aceasta fereastra se poate actualiza pe masura ce se analizeaza mai multe miscari. De exemplul plafonul minim alpha se mareste pe masura ce gasim anumite tipuri de miscari mai bune (**better worst best moves**). Asadar, in implementare tinem seama si de aceste doua plafoane. In conformitate cu principiul Minimax, plafonul de minim al unui jocator (alpha-ul) este plafonul de maxim al celuilalt (beta-ul) si invers. Prezentam in continuare o descriere grafica a algoritmului Alpha-beta:

Un video cu un exemplu detaliat si foarte bine explicat se gaseste in tutorialul recomandat de pe YouTube (de la minutul 21:30 la 30:30).

Implementare

In continuare prezentam o implementare conceptuala a Alpha-beta, atat pentru Minimax, cat si pentru Negamax:



1. Start at C. Descend to full-ply depth and assign the heuristic to a state and all siblings (MIN 2, 3). Back up these values to their parent node (MAX 3).
2. Offer this value to the grandparent (A), as its beta value. So **A has beta=3**. A will be no larger than 3
3. Descend to A's other grandchildren. Terminate the search of their parent if any grandchildren is >= A's beta. **Node B is beta-pruned**, as shown, because its value must be at least 5.
4. Once A's value is known, offer it to its parent (C) as its alpha value. So **C has alpha=3**. C will be no smaller than 3.
5. Repeat this process, descending to C's great grandchildren (0) in a depth-first fashion. **D is alpha-pruned**, because no matter what happens on its right branch, it cannot be greater than 0.
6. Repeating on E, **E is alpha-pruned** because its beta value (2) is less than its parent's alpha value (3). So no matter what happens on its right branch, E cannot have a value greater than 2.
7. Therefore **C is 3.**

```
-----------------------------------------------
int evaluate(stare);   // functia returneaza un scor asociat cu starea
                       // functia returneaza mereu scorul din perspectiva lui maxi


void apply_move(move); // functia modifica starea curenta: executa miscarea move
void undo_move(move);  // functia restaureaza starea anterioara (de dinainte de executa lui move)

// alege cea mai buna mutare pentru jucatorul maxi
// functia returneaza best_score pentru maxi, dar tine cont si de ce ii permite mini
int aplhabeta_maxi(int alpha, int beta, int depth) {
    // daca jocul s-a terminat sau am atins nivelul maxim de recursivitate ales
    if (gameOver() || depth == 0 ) {
        return evaluate();            // ne oprim si evaluam starea curenta
    }

    // incercam pe rand fiecare miscare posibila move
    for (move : all_moves) {
        apply_move(move);         // executa move

        // incercam sa simulam jocul mai departe:
        // daca maxi face move, ce ar face mini?
        int score = alphabeta_mini(alpha, beta, depth - 1);

        if (score >= beta) {
            return beta;    // beta cut-off
        }

        // dintre toate variantele pe care mini le permite (!!!),
        // maxi o va alege pe cea cu scor maxim
        if (score > alpha) {
            alpha = score;
        }

        undo_move(move);         // restaureaza starea de dinainte de move
   }

    // cel mai bun scor pe care il poate obtine maxi este alpha
    // (s-a tinut cont si de ce i-ar permite mini, avand in vedere ca si el joaca optim)
    return alpha;
}


// alege cea mai buna mutare pentru jucatorul mini
// functia returneaza best_score pentru mini, dar tine cont si de ce ii permite maxi
int aplhabeta_mini(int alpha, int beta, int depth) {
    // daca jocul s-a terminat sau am atins nivelul maxim de recursivitate ales
    if (gameOver() || depth == 0 ) {
        return -evaluate();            // ne oprim si evaluam starea curenta
    }

    // incercam pe rand fiecare miscare posibila move
    for (move : all_moves) {
        apply_move(move);         // executa move

        // incercam sa simulam jocul mai departe:
        // daca mini face move, ce ar face maxi?
        int score = alphabeta_maxi(alpha, beta, depth - 1);

        if (score <= alpha) {
            return alpha;    // alpha cut-off
        }

        // dintre toate variantele pe care maxi le permite (!!!),
```

```
        // mini o va alege pe cea cu scor minim
        if (score < beta) {
            beta = score;
        }

        undo_move(move);         // restaureaza starea de dinainte de move
    }

    // cel mai bun scor pe care il poate obtine mini este beta
    // (s-a tinut cont si de ce i-ar permite maxi, avand in vedere ca si el joaca optim)
    return beta;
}
```

```
int evaluate(stare);   // functia returneaza un scor asociat cu starea
                       // functia returneaza mereu scorul din perspectiva jucatorului CURENT!


void apply_move(move); // functia modifica starea curenta: executa miscarea move
void undo_move(move);  // functia restaureaza starea anterioara (de dinainte de executa lui move)

// alege cea mai buna mutare pentru jucatorul CURENT (EU)
// functia returneaza best_score
int alphabeta_negamax(int aplha, int beta, int depth) {
    // daca jocul s-a terminat sau am atins nivelul maxim de recursivitate ales
    if (depth == 0 || gameOver()) {
       return evaluate();            // ne oprim si evaluam starea curenta
    }

    int max = -oo;

    // incercam pe rand fiecare care miscare posibila move
    for (move : all_moves) {
        apply_move(move);         // executa move

        // incercam sa simulam jocul mai departe:
        // daca jucatorul CURENT face face move, ce ar face ADVERSARUL?
        int score = -alphabeta_negamax(-beta, -alpha, depth - 1);  // cel mai bine pentru el,
                                                                   // este cel mai rau pentru mine
                                                                   // si invers

        // dintre toate variantele pe care ADVERSARUL le permite (!!!),
        // EU (jucatorul CURENT) o voi alege pe cea cu scor maxim
        if (score >= max) {
            max = score;
        }

        if (max > alpha) {
            alpha = max;
        }

        if (alpha >= beta) {
            break;               // cut-off
        }

        undo_move(move);         // restaureaza starea de dinainte de move
    }

    // cel mai bun scor pe care il pot obtine EU este alpha
    // (s-a tinut cont si de ce mi-ar permite ADVERSARUL, avand in vedere ca si el joaca optim)
    return alpha;
}
```

Din nou remarcam claritatea si coerenta sporita a variantei negamax!

Complexitate

In continuare prezentam complexitatile asociate algoritmilor prezentati anterior. Pentru aceasta vom introduce cateva notiuni:

- **branch factor** : **b** = **numarul mediu de ramificari** pe care le are un nod neterminal (care nu e frunza) din **arborele de solutii**
- **depth** : **d** = **adancimea maxima** pana la care se face cautarea in arborele de solutii
    - orice nod de adancime d va fi considerat terminal

Un arbore cu un branching factor **b**, care va fi examinat pana la un nivel **d** va furniza b^d noduri frunze ce vor trebui procesate (ex. calculam scorul pentru acele noduri).

Nivelurile sunt notate cu $0, 1, 2, \ldots, d$

- nivel 0: 1 nod (radacina)
- nivel 1: b noduri
- nivel 2: b^2 noduri
- nivel 3: b^3 noduri
- …
- nivel d: b^d noduri

- **minimax/negamax**
    - Un algoritm **mini/negamax** clasic, care analizeaza toate starile posibile, va avea complexitatea $O(b^d)$ - deci exponentiala.

- **alpha-beta**
    - Cat de bun este insa alpha-beta fata de un mini/nega-max naiv? Dupa cum s-a mentionat anterior, in functie de ordonarea miscarilor ce vor fi evaluate putem avea un caz cel mai favorabil si un caz cel mai defavorabil.
    - **best case** : miscarile sunt ordonate descrescator dupa castig (deci ordonate optim), rezulta o complexitate
        - $O(b * 1 * b * 1 * b * 1 ... de\ d\ ori \ldots b * 1)$ pentru d par
        - $O(b * 1 * b * 1 * b * 1 ... de\ d\ ori \ldots b)$ pentru d impar
        - restrangand ambele expresii rezulta o complexitate $O(b^{\frac{d}{2}}) = O(\sqrt{b^d})$
        - prin urmare, intr-un caz ideal, algoritmul alpha-beta poate explora de 2 ori mai putine nivele in arborele de solutii fata de un algoritm mini/nega-max naiv.
    - **worst case**: miscarile sunt ordonate crescator dupa castigul furnizat unui jucator, astfel fiind necesara o examinare a tuturor nodurilor pentru gasirea celei mai bune miscari.
        - in consecinta complexitatea devine egala cu cea a unui algoritm mini/negamax naiv.

Concluzii si observatii

Alpha-beta NU ofera o alta solutie fata de Minimax! Este doar o optimizare pusa deasupra algoritmului Minimax care ne permite sa exploram mai multe stari in acelasi timp sau pentru acelasi numar de stari sa optinem un timp de doua ori mai mic.

Exemple

Dintre cele mai importante jocuri in care putem aplica direct strategia minimax, mentionam:

- X și 0 [https://en.wikipedia.org/wiki/Tic-tac-toe]
    - joc foarte simplu/usor (spatiul starilor este mic).
    - Prin urmare tot arborele de solutii poate fi generat si explorat intr-un timp foarte scurt.

- sah [https://en.wikipedia.org/wiki/Chess]
  - joc foarte greu (spatiul starilor este foarte mare)
  - minimax/negamax simplu poate merge pana la $d = 7$ (nu reusea da bata campionul mondial la sah - campion uman)
  - alpha-beta poate merge pana la $d = 14$
  - **Deep Blue** a fost implementarea unui bot cu minimax si alpha-beta care a batut in 1997 campionul mondial la sah (Gary Kasparov).
- Ultimate tic-tac-toe [https://en.wikipedia.org/wiki/Ultimate_tic-tac-toe]
  - varianta mult mai grea de X si 0 (spatiul starilor foarte mare)
  - s-a dat la proiect PA 2016 :D
  - implemantarile se pot testa aici [http://theaigames.com/competitions/ultimate-tic-tac-toe/rules]
- Nim [https://en.wikipedia.org/wiki/Nim]
- Reversi [https://en.wikipedia.org/wiki/Reversi]

Alte exempe de jocuri interesante:

- Go [https://en.wikipedia.org/wiki/Go_(game)]
  - solutiile se bazeaza pe Monte Carlo Tree Search (nu pe minimax)
  - AlphaGo [https://en.wikipedia.org/wiki/AlphaGo] este botul cel mai bun pe tabla de 19×19

Nim

Fiind date 3 multimi de bile, fiecare jucator trebuie sa extraga la fiecare mutare 1, 2 sau 3 bile din oricare multime.

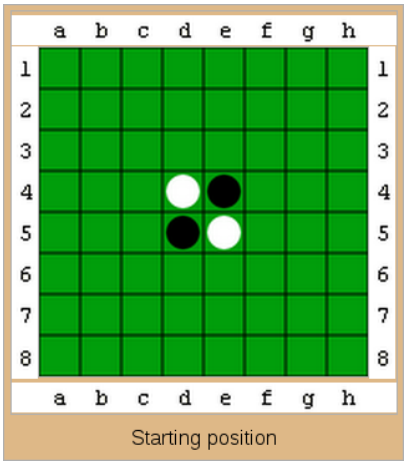
Cel care este fortat sa aleaga ultima bila, pierde.

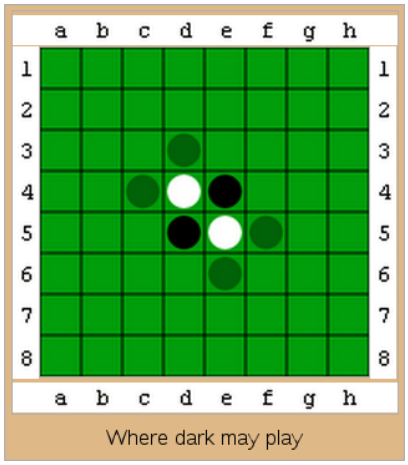
Reversi game

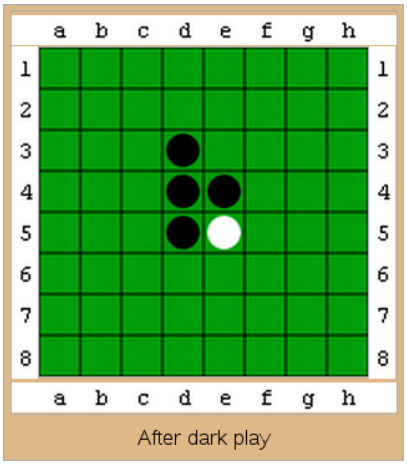
Tabla de joc consista dintr-un grid 6×6. Piesele pot fi reprezentate de monede, fiecarui jucator fiindu-i asociata o fata diferita a monezii. Jucatorii muta alternativ, dupa regula urmatoare:

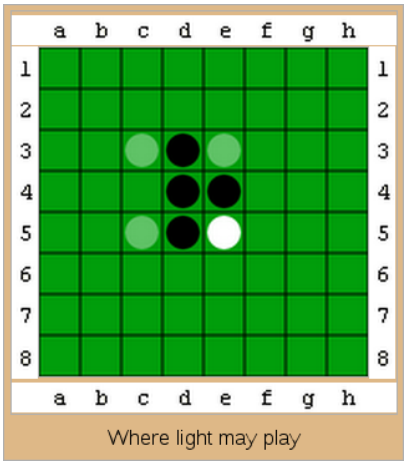
- Pozitia (x, y) in care este plasata piesa trebuie sa fie libera
- Trebuie sa existe o alta pozitie (x', y'), pe aceeasi linie, coloana sau diagonala a jucatorului aflat la mutare si toate pozitiile dintre (x, y) si (x', y') trebuie sa fie ocupate de piese ale adversarului
- Piesele adversarului dintre (x, y) si (x', y') vor fi capturate, intorcandu-se monedele pe cealalta fata.

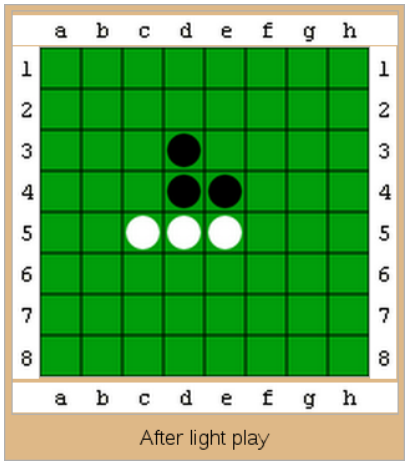
Mai jos, de la stanga spre dreapta: pozitia initiala, posibilitatile de mutare ale primului jucator, tabla dupa prima mutare, posibilitatile de mutare ale celuilalt jucator, tabla dupa a doua mutare.











Observatii:

- Jucatorul poate acapara piese ale adversarului in mai multe directii simultan
- Daca un jucator nu are unde muta, acesta cedeaza randul, adversarul efectuand a doua mutare la rand
- Jocul se incheie cand nimeni nu mai poate muta, invingatorul fiind acela care detine cele mai multe piese proprii

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab06.zip.

Fiecare problemă are surse și Makefile (ex. “cd cpp/p1/; make; make run”).

Vom implementa algoritmi pentru jocurile **Nim** si **Reversi**.

Minimax Nim

Se doreste implementarea algoritmulului minimax sau negamax pentru Nim.

Recomandam implementarea variantei Negamax.

Minimax Reversi

Se doreste implementarea algoritmulului minimax sau negamax pentru Reversi.

Recomandam implementarea variantei Negamax.

Bonus

Extindeti algoritmul implementat anterior pentru jocul **Reversi** intr-un algoritm de tip alpha-beta pruning. Cum puteti sa comparati cei doi algoritmi implementati pentru Reversi?

Referinte

[1] http://en.wikipedia.org/wiki/Minimax [http://en.wikipedia.org/wiki/Minimax]

[2] http://en.wikipedia.org/wiki/Negamax [http://en.wikipedia.org/wiki/Negamax]

[3] http://en.wikipedia.org/wiki/Alpha-beta_pruning [http://en.wikipedia.org/wiki/Alpha-beta_pruning]

[4] http://inst.eecs.berkeley.edu/~cs61b/fa14/ta-materials/apps/ab_tree_practice/ [http://inst.eecs.berkeley.edu/~cs61b/fa14/ta-materials/apps/ab_tree_practice/]

pa/laboratoare/laborator-06.txt · Last modified: 2020/04/22 18:52 by darius.neatu

Laborator 07: Parcurgerea Grafurilor. Sortare Topologica

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

- Intelegerea conceptului de graf si a modurilor de parcurgere aferente
- Utilitatea si aplicabilitatea sortarii topologice

Importanţă – aplicaţii practice

Grafurile sunt utile pentru a modela diverse probleme si se regasesc implementati in multiple aplicatii practice:

- Retele de calculatoare (ex: stabilirea unei topologii fara bucle)
- Pagini Web (ex: Google PageRank [1])
- Retele sociale (ex: calcul centralitate [2])
- Harti cu drumuri (ex: drum minim)
- Modelare grafica (ex: prefuse [3], graph-cut [4])

Descrierea problemei și a rezolvărilor

Graful poate fi modelat drept o pereche de multimi G = (V, E). Multimea V contine nodurile grafului (vertices), iar multimea E contine muchiile (edges), fiecare muchie stabilind o relatie de vecinatate intre doua noduri. O mare varietate de probleme se modeleaza folosind grafuri, iar rezolvarea acestora presupune explorarea spatiului. O parcurgere isi propune sa ia in discutie fiecare nod al grafului, exact o singura data, pornind de la un nod ales, numit in continuare nod sursa.

Reprezentarea in memorie a grafurilor se face, de obicei, cu liste de adiacenta sau cu matrice de adiacenta. Se pot folosi insa si alte structuri de date, de exemplu un map de perechi < <sursa,destinatie>,cost> .

Pe parcursul rularii algoritmilor de parcurgere, un nod poate avea 3 culori:

- Alb = nedescoperit
- Gri = a fost descoperit si este in curs de prcesare
- Negru = a fost procesat

Se poate face o analogie cu o pata neagra care se extinde pe un spatiu alb. Nodurile gri se afla pe frontiera petei negre. Algoritmii de parcurgere pot fi caracterizati prin completitudine si optimalitate. Un algoritm de explorare complet va descoperi intotdeauna o solutie, daca problema accepta solutie. Un algoritm de explorare optimal va descoperi solutia optima a problemei din perspectiva numarului de pasi care trebuie efectuati.

Parcurgerea in lățime - BFS

Parcurgerea in latime **(Breadth-first Search - BFS)** este un algoritm de cautare in graf, in care, atunci cand se ajunge intr-un nod oarecare v, nevizitat, se viziteaza toate nodurile nevizitate adiacente lui v, apoi toate varfurile nevizitate adiacente varfurilor adiacente lui v, etc. Atentie! BFS depinde de nodul de start.

Plecand dintr-un nod se va parcurge doar componenta conexa din care acesta face parte. Pentru grafuri cu mai multe componente conexe se vor obtine mai multi arbori de acoperire.

In urma aplicarii algoritmului BFS asupra fiecarei componente conexe a grafului, se obtine un arbore de acoperire (prin eliminarea muchiilor pe care nu le folosim la parcurgere). Pentru a putea reconstitui acest arbore, se pastreaza pentru fiecare nod dat identitatea parintelui sau. In cazul in care nu exista o functie de cost asociata muchiilor, BFS va determina si drumurile minime de la radacina la oricare nod.

Pentru implementarea BFS se foloseste o coada. In momentul adaugarii in coada, un nod trebuie colorat gri (a fost descoperit si urmeaza sa fie prelucrat).

Algoritmul de explorare BFS este complet si optimal.

Algoritm:

```
BFS(s, G) {
    foreach (u ∈ V) {
        p(u) = null; // initializari
        dist(s,u) = inf;
        c(u) = alb;
    }
    dist(s) = 0; // distanta pana la sursa este 0
    c(s) = gri; //incepem prelucrarea nodului, deci culoarea devine gri
    Q = (); //se foloseste o coada cu nodurile de prelucrat
    Q = Q + s; // adaugam sursa in coada
    while (!empty(Q)) { // cat timp mai am noduri de prelucrat
        u = top(Q); // se determina nodul din varful cozii
        foreach v ∈ succs(u) { // pentru toti vecinii
            if (c(v) = alb) {// nodul nu a fost gasit, nu e in coada
                // actualizam structura date
                dist(v) = dist(u) + 1;
                p(v) = u;
                c(v) = gri;
                Q = Q + v;
            } // close if
        } // close foreach
        c(u) = negru; //am terminat de prelucrat nodul curent
        Q = Q - u; //nodul este eliminat din coada
    } //close while
}
```

Complexitate:

- cu lista: O(|E|+|V|)
- cu matrice: O(|V|^2)

Parcurgerea in adancime – DFS

Parcurgerea in adancime **(Depth-First Search - DFS)** porneste de la un nod dat (nod de start), care este marcat ca fiind in curs de procesare. Se alege primul vecin nevizitat al acestui nod, se marcheaza si acesta ca fiind in curs de procesare, apoi si pentru acest vecin se cauta primul vecin nevizitat, si asa mai departe. In momentul in care nodul curent nu mai are vecini nevizitati, se marcheaza ca fiind deja procesat si se revine la nodul anterior. Pentru acest nod se cauta primul vecin nevizitat. Algoritmul se repeta pana cand toate nodurile grafului au fost procesate.

In urma aplicarii algoritmului DFS asupra fiecarei componente conexe a grafului, se obtine pentru fiecare dintre acestea cate un arbore de acoperire (prin eliminarea muchiilor pe care nu le folosim la parcurgere). Pentru a putea reconstitui acest arbore, pastram pentru fiecare nod dat identitatea parintelui sau.

Pentru fiecare nod se vor retine:

- timpul descoperirii
- timpul finalizarii
- parintele

- culoarea

Algoritmul de explorare DFS nu este nici complet (in cazul unei cautari pe un subarbore infinit), nici optimal (nu gaseste nodul cu adancimea minima).

Spre deosebire de BFS, pentru implementarea DFS se foloseste o stiva (abordare **LIFO** in loc de **FIFO**). Desi se poate face aceasta inlocuire in algoritmul de mai sus, de cele mai multe ori este mai intuitiva folosirea recusivitatii.

Algoritm:

```
DFS(G) {
    V = noduri(G)
    foreach (u ∈ V) {
        // initializare structura date
        c(u) = alb;
        p(u)=null;
    }
    timp = 0; // retine distanta de la radacina pana la nodul curent
    foreach (u ∈ V)
        if (c(u) = alb) explorare(u); // explorez nodul
}

explorare(u) {
    d(u) = timp++; // timpul de descoperire al nodului u
    c(u) = gri; // nod in curs de explorare
    foreach (v ∈ succes(u)) // incerc sa prelucrez vecinii
        if (c(v) = alb) { // daca nu au fost prelucrati deja
            p(v) = u;
            explorare(v);
        }
    c(u) = negru; // am terminat de prelucrat nodul curent
    f(u) = timp++; // timpul de finalizare al nodului u
}
```

Complexitate:

- cu lista: O(|E|+|V|)
- cu matrice: O(|V|^2)

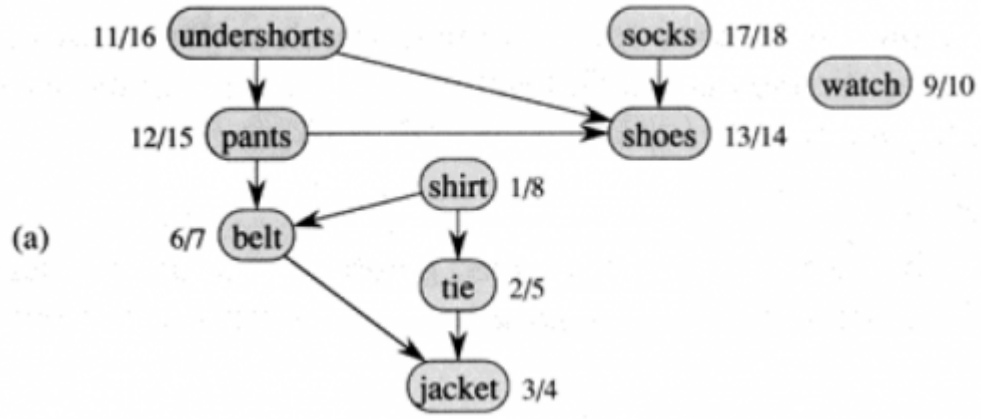
Sortarea Topologica

Dandu-se un graf orientat aciclic, sortarea topologica realizeaza o aranjare liniara a nodurilor in functie de muchiile dintre ele. Orientarea muchiilor corespunde unei relatii de ordine de la nodul sursa catre cel destinatie. Astfel, daca (u,v) este una dintre muchiile grafului, u trebuie sa apara inaintea lui v in insiruire. Daca graful ar fi ciclic, nu ar putea exista o astfel de insiruire (nu se poate stabili o ordine intre nodurile care alcatuiesc un ciclu).

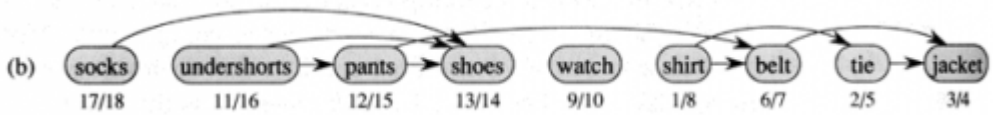
Sortarea topologica poate fi vazuta si ca plasarea nodurilor de-a lungul unei linii orizontale astfel incat toate muchiile sa fie directionate de la stanga la dreapta.

Exemplu:

Profesorul Bumstead isi sorteaza topologic hainele inainte de a se imbraca.



(a) Fiecare muchie (u,v) inseamna ca obiectul de imbracaminte u trebuie imbracat inaintea obiectului de imbracaminte v. Timpii de descoperire d(u) si de finalizare f(u) obtinuti in urma parcurgerii DFS sunt notati langa noduri.



(b) Acelasi graf, sortat topologic. Nodurile lui sunt aranjate de la stanga la dreapta in ordinea **descrescatoare** a **timpului de finalizare** f(u). Observati ca toate muchiile sunt orientate de la stanga la dreapta. Acum profesorul Bumstead se poate imbraca linistit.

Algoritm:

Sunt doi algoritmi cunoscuti pentru sortarea topologina.

Algoritmul bazat pe DFS:

- parcurgere DFS pentru determinarea timpilor
- sortare descrescatoare in functie de timpul de finalizare

Pentru a evita sortarea nodurilor in functie de timpul de finalizare, se poate folosi o stiva ce retine aceste noduri in ordinea terminarii parcurgerii.

Un alt algoritm este cel descris de Kahn:

```
TopSort(G) {
    V = noduri(G)
    L = vida;// lista care va contine elementele sortate
    // initializare S cu nodurile care nu au in-muchii
    foreach (u ∈ V) {
        if (u nu are in-muchii)
            S = S + u;
    }
    while (!empty(S)) { // cat timp mai am noduri de prelucrat
        u = random(S); // se scoate un nod din multimea S
        L = L + u;  // adaug U la lista finala
        foreach v ∈ succs(u) { // pentru toti vecinii
            sterge u-v; // sterge muchia u-v
            if (v nu are in-muchii)
                S = S + v;  // adauga v la multimea S
        } // close foreach
    } //close while
    if (G are muchii)
        print(eroare);  // graf ciclic
    else
        print(L); // ordinea topologica
}
```

Complexitate optima: O(|E|+|V|)

Concluzii si observatii

Grafurile sunt foarte importante pentru reprezentarea si rezolvarea unei multitudini de probleme. Cele mai uzuale moduri de reprezentare a unui graf sunt:

- liste de adiacenta
- matrice de adiacenta

Cele doua moduri uzuale de parcurgere neinformata a unui graf sunt:

- BFS – parcurgere in latime
- DFS – parcurgere in adancime

Sortarea topologica este o modalitate de aranjare a nodurilor in functie de muchiile dintre ele. In functie de nodul de start al DFS, se pot obtine sortari diferite, pastrand insa proprietatile generale ale sortarii topologice.

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab07.zip.

Inainte de a rezolva exercitiile, asigurati-va ca ati citit si inteles toate precizarile din sectiunea Precizari laboratoare 07-12 [https://ocw.cs.pub.ro/courses/pa/skel_graph].

Prin citirea acestor precizari va asigurati ca:

- cunoasteti **conventiile** folosite
- evitati **buguri**
- evitati **depunctari** la lab/teme/test

BFS

Se da un graf **neorientat** cu **n** noduri si **m** muchii. Se mai da un nod special **source**, pe care il vom numi sursa.

Se cere sa se gaseasca **numarul minim de muchii** ce trebuie parcurse de la **source** la **toate** celelalte noduri.

Restrictii si precizari:

- $n, m <= 10^5$
- timp de executie
  - C++: 1s
  - Java: 1s

Rezultatul se va returna sub forma unui vector **d** cu **n** elemente.

Convention:

- **d[node]** = numarul minim de muchii ce trebuie parcurse de la **source** la nodul **node**
- **d[source] = 0**
- **d[node] = -1**, daca nu se poate ajunge de la **source** la **node**

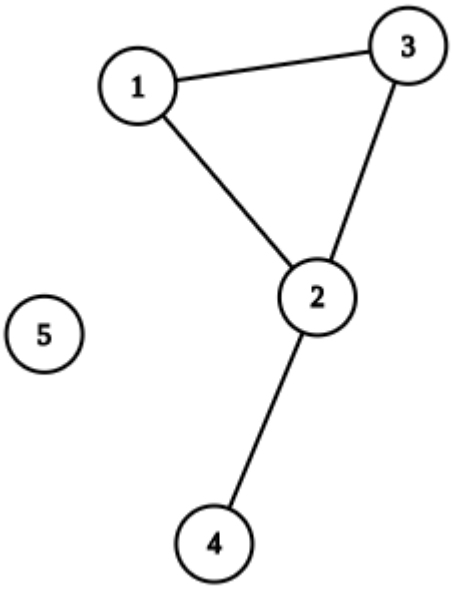
$n = 5\ m = 4\ source = 3$

$muchii : (1, 2); (1, 3); (2, 3); (2, 4);$

Raspuns:

node	1	2	3	4	5
d	1	1	0	2	-1

Explicatie: Graful dat este cel din figura urmatoare.



- **d[3] = 0** pentru ca 1 este sursa
- **d[1] = d[2] = 1** pentru ca exista muchie directa de la 2 la fiecare nod
- **d[4] = 2** pentru ca trebuie sa parcurgem 2 muchii $(3 - 2 - 4)$
- **d[5] = -1** pentru ca nu se poate ajunge de la 3 la 5

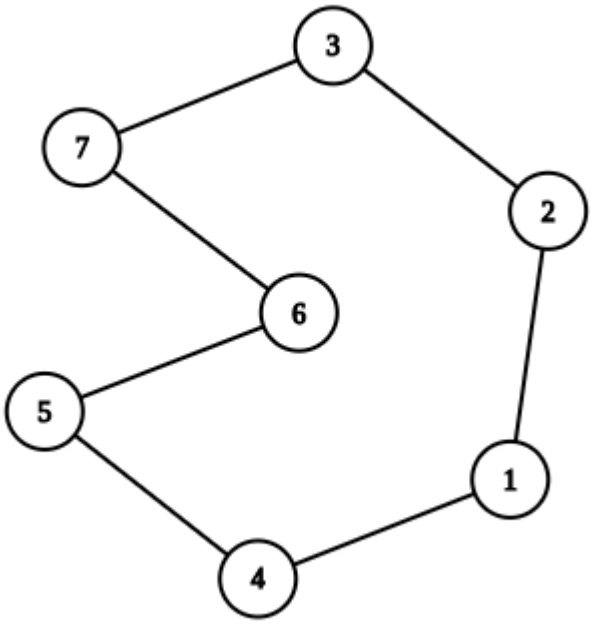
$n = 7\ m = 7\ source = 1$

$muchii : (1, 2); (1, 4); (2, 3); (4, 5); (5, 6); (3, 7); (7, 6)$

Raspuns:

node	1	2	3	4	5	6	7
d	0	1	2	1	2	3	3

Explicatie: Graful dat este cel din figura urmatoare.



- **d[1] = 0** pentru ca 1 este sursa
- **d[2] = d[4] = 1** pentru ca exista muchie directa de la 2 la fiecare nod
- **d[3] = d[5] = 2** pentru ca trebuie sa parcurgem 2 muchii ($1-2-3$, $1-4-5$)
- **d[6] = d[7] = 3** pentru ca trebuie sa parcurgem 3 muchii ($1-2-3-7$ sau $1-4-5-6$)

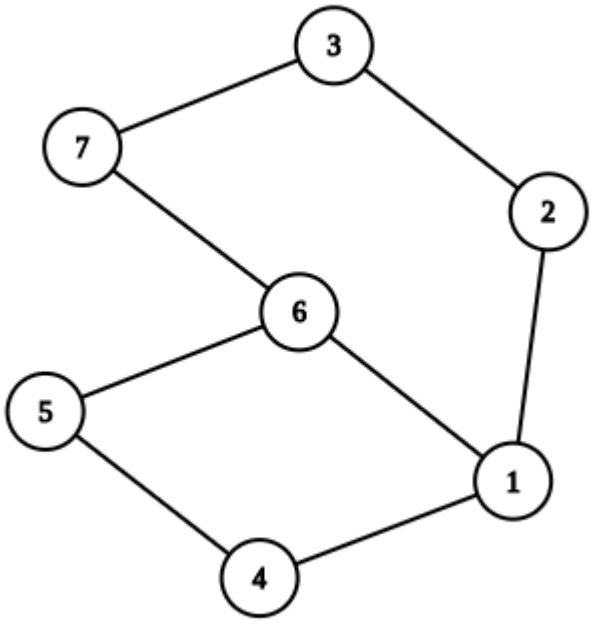
$n = 7$ $m = 8$ $source = 1$

$muchii$: $(1,2); (1,4); (2,3); (4,5); (5,6); (3,7); (7,6); (1,6)$

Raspuns:

node	1	2	3	4	5	6	7
d	0	1	2	1	2	1	2

Explicatie: Graful dat este cel din figura urmatoare.



- **d[1] = 0** pentru ca 1 este sursa
- **d[2] = d[4] = d[6] = 1** pentru ca exista muchie directa de la 2 la fiecare nod
- **d[3] = d[5] = d[7] = 2** pentru ca trebuie sa parcurgem 2 muchii ($1-2-3$, $1-4-5$, $1-6-7$)

TOPOLOGICAL SORT

Se da un graf **orientat** aciclic cu **n** noduri si **m** arce. Se cere sa se gaseaca **o sortare topologica** valida.

Restrictii si precizari:

- $n, m <= 10^5$
- timp de executie
  - C++: 1s
  - Java: 1s

Rezultatul se va returna sub forma unui vector **topsort** cu **n** elemente.

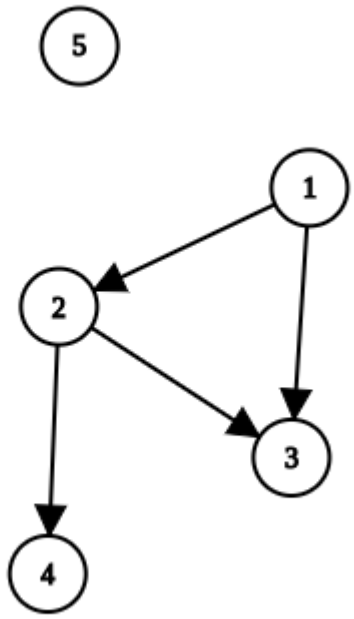
Vectorul **topsort** va reprezenta o permutare a multimii $1, 2, 3, \ldots, n$ reprezentand sortarea topologica gasita.

$n = 5$ $m = 4$

$arce: (1,2); (1,3); (2,3); (2,4);$

Raspuns: $topsort = [1, 2, 3, 4, 5]$

Explicatie: Graful dat este cel din figura urmatoare.



- deoarece avem arcele $1 \rightarrow 3$ si $1 \rightarrow 2$, 1 trebuie sa apara inainte lui 2 si 3
- deoarece avem arcul $2 \rightarrow 3$ si $2 \rightarrow 4$, 2 trebuie sa apara inainte lui 3 si 4
- 5 nu depinde de nimeni, poate sa apara oriunde

Toate sortarile topologice valide sunt:

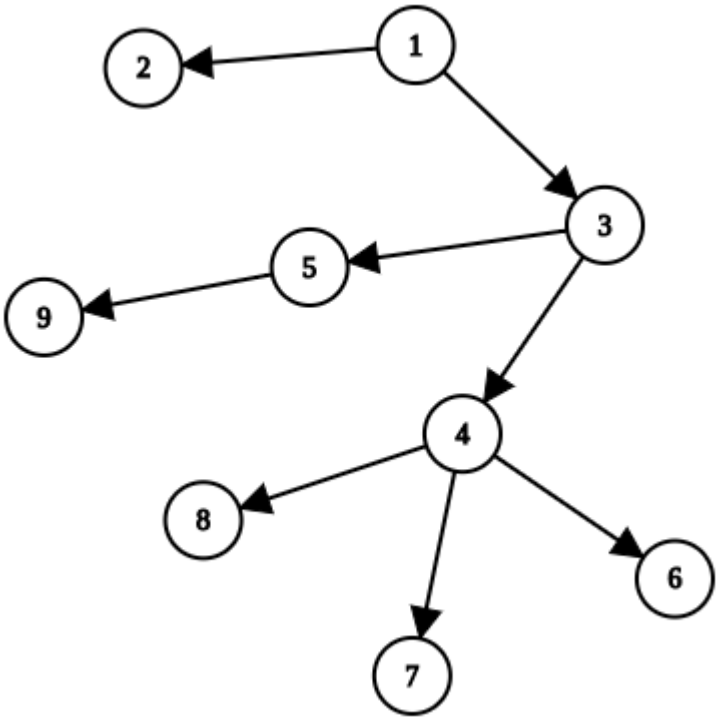
- cele date de ordinea relativa a primelor 4 noduri: 1,2,3,4)
  - $topsort = [1, 2, 3, 4, 5]$
  - $topsort = [1, 2, 3, 5, 4]$
  - $topsort = [1, 2, 5, 3, 4]$
  - $topsort = [1, 5, 2, 3, 4]$
  - $topsort = [5, 1, 2, 3, 4]$
- cele date de ordinea relativa a primelor 4 noduri: (1,2,4,3)
  - $topsort = [1, 2, 4, 3, 5]$
  - $topsort = [1, 2, 4, 5, 3]$
  - $topsort = [1, 2, 5, 4, 3]$
  - $topsort = [1, 5, 2, 4, 3]$
  - $topsort = [5, 1, 2, 4, 3]$

$n = 9$ $m = 8$

$arce : (1, 2); (1, 3); (3, 4); (3, 5); (5, 9); (4, 6); (4, 7); (4, 8);$

Raspuns: $topsort = [1, 2, 3, 4, 6, 7, 8, 5, 9]$

Explicatie: Graful dat este cel din figura urmatoare.



Se observa din desen ca solutia mentionata este valida.

BONUS

Se da un tablou bidimensional de dimensiune **n x n**. Fiecare celula este o camera.

O valoare din matrice poate sa fie **0**, **1** sau **2**.

- 0 = celula libera
- 1 = celula blocata (contine un obstacol - nu se poate intra intr-o astfel de camera)
- 2 = celula contine o poarta de teleportare (ex. iesire catre exterior - cine stie ce o fi afara)

Un student de la Poli se afla intr-un punct interior dat prin coodonatele **start_row** si **start_col**. Deoarece studentul este dornit sa termine facultatea cat mai repede, se intreaba care este **lungimea celui mai scurt drum** de la pozitia sa catre o poarta de teleportare.

Studentul se poate deplasa doar pe verticala sau pe orizontala.

Task-uri:

- gasiti **lungimea** ceruta
- reconstituiti **drumul** de lungime minima (daca sunt mai multe, se poate afisa oricare)

Restrictii si precizari:

- $n <= 10^3$
- timp de executie
  - C++: 1s
  - Java: 1s

$n = 5$

1	0	2	1	0

0	0	0	1	1
2	1	0	0	1
1	1	0	1	0
0	1	2	1	0

Raspuns: $lungime = 3$

$drum = (3,4) \rightarrow (3,3) \rightarrow (2,3) \rightarrow (1,3)$

Explicatie: Exista doua posibilitati de a porni din punctul de coordonate $(3,4)$ si a ajunge printr-un drum de lungime minima la o poarta. Unul dintre acestea urmeaza traseul mentionat.

Extra

Rezolvati problema muzeu [https://infoarena.ro/problema/muzeu] pe infoarena.

Rezolvati problema arbore3 [https://infoarena.ro/problema/arbore3] pe infoarena.

Rezolvati problema Pokemon GO AWAY [https://www.hackerrank.com/contests/test-practic-pa-2017-v2-meeseeks/challenges/test-2-pokemon-go-away-grea] de la test PA 2017.

Cu ce problema seamana?

Rezolvati problema insule [https://infoarena.ro/problema/insule] pe infoarena.

Rezolvati problema tsunami [https://infoarena.ro/problema/tsunami] pe infoarena.

Rezolvati problema berarii2 [https://infoarena.ro/problema/berarii2] pe infoarena.

Referinte

[1] http://en.wikipedia.org/wiki/PageRank [http://en.wikipedia.org/wiki/PageRank]

[2] http://en.wikipedia.org/wiki/Social_network#Social_network_analysis [http://en.wikipedia.org/wiki/Social_network#Social_network_analysis]

[3] http://prefuse.org/ [http://prefuse.org/]

[4] http://classes.engr.oregonstate.edu/eecs/spring2008/cs419/Lectures/jun_graphcut.pdf [http://classes.engr.oregonstate.edu/eecs/spring2008/cs419/Lectures/jun_graphcut.pdf]

[5] http://en.wikipedia.org/wiki/Breadth-first_search [http://en.wikipedia.org/wiki/Breadth-first_search]

[6] http://en.wikipedia.org/wiki/Depth-first_search [http://en.wikipedia.org/wiki/Depth-first_search]

[7] http://en.wikipedia.org/wiki/Topological_sorting [http://en.wikipedia.org/wiki/Topological_sorting]

[8] Introducere in Algoritmi, T. Cormen s.a., pag 403-419

[9] http://ww3.algorithmdesign.net/handouts/DFS.pdf [http://ww3.algorithmdesign.net/handouts/DFS.pdf]

[10] http://ww3.algorithmdesign.net/handouts/BFS.pdf [http://ww3.algorithmdesign.net/handouts/BFS.pdf]

Laborator 08: Aplicatii DFS

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

Întelegerea noţiunilor teoretice:

- tare conexitate, componente tare conexe (pentru grafuri orientate)
- punct de articulaţie (pentru grafuri neorientate)
- punţi (pentru grafuri neorientate)
- componente biconexe (în general, pentru grafuri neorientate)

Înţelegerea algoritmilor ce rezolvă aceste probleme şi implementarea acestor algoritmi.

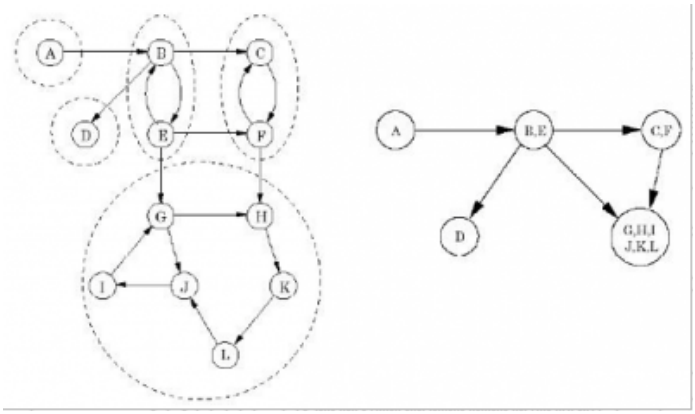
Importanţă – aplicaţii practice

- Componentele biconexe au aplicaţii importante în reţelistică, deoarece o componentă biconexă asigură redundanţa.
- Descompunerea în componente tare conexe: data mining, compilatoare, calcul ştiinţific, 2SAT

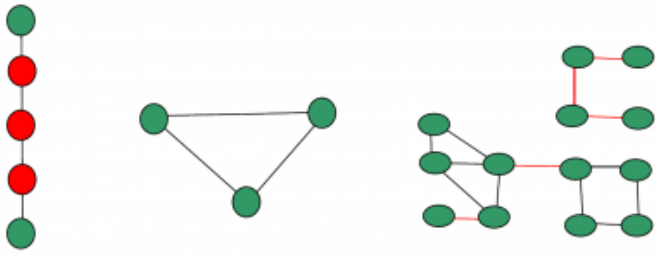
Notiuni teoretice

- **Tare conexitate.** Un graf orientat este tare conex, dacă oricare ar fi două vârfuri u şi v, ele sunt tare conectate (strongly connected) - există drum atât de la u la v, cât şi de la v la u.
- O componentă tare conexă este un subgraf maximal tare conex al unui graf orientat, adică o submulţime de vârfuri U din V, astfel încât pentru orice u şi v din U ele sunt tare conectate. Dacă fiecare componentă tare conexă este redusă într-un singur nod, se va obţine **un graf orientat aciclic**.

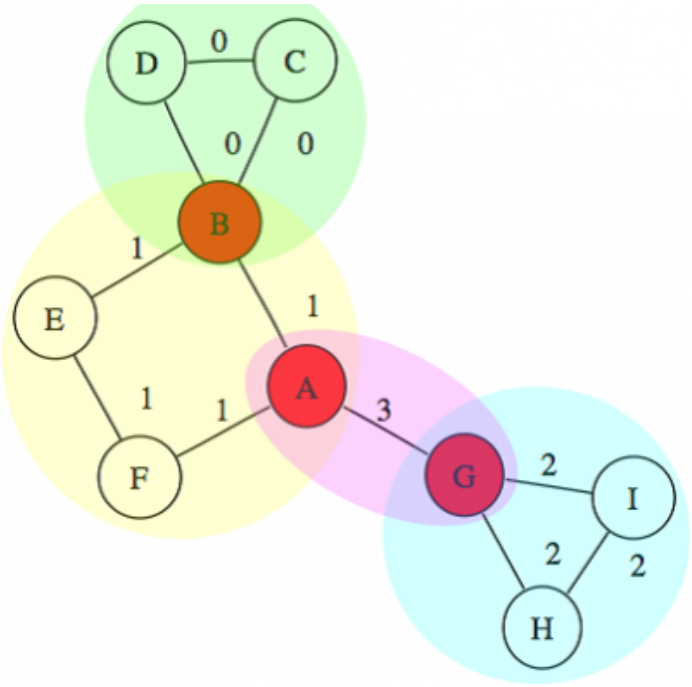
De exemplu:



- **Un punct de articulaţie (cut vertex)** este un nod al unui graf a cărui eliminare duce la creşterea numărului de componente conexe ale acelui graf.
- **O punte (bridge)** este o muchie a unui graf (se mai numeşte şi **muchie critică**) a cărei eliminare duce la creşterea numărului de componente conexe ale acelui graf.



- **Biconexitate**. Un graf biconex este un graf conex cu proprietatea că eliminând oricare nod al acestuia, graful rămâne conex.
- O componentă biconexă a unui graf este o mulţime **maximală** de noduri care respectă proprietatea de biconexitate.



Componente tare conexe

Vom porni de la definiţie pentru a afla componenta tare conexă din care face parte un nod v. Vom parcurge graful (DFS sau BFS) pentru a găsi o mulţime de noduri S ce sunt accesibile din v. Vom parcurge apoi graful transpus (obţinut prin inversarea arcelor din graful iniţial), determinând o nouă mulţime de noduri T ce sunt accesibile din v în graful transpus. Intersecţia dintre S şi T va reprezenta componenta tare conexa. Graful iniţial şi cel transpus au aceleaşi componente conexe.

Algoritmul lui Kosaraju

Algoritmul foloseşte două DFS (una pe graful iniţial şi una pe graful transpus) şi o stivă pentru a reţine ordinea terminării parcurgerii nodurilor grafului original (evitând astfel o sortare a nodurilor după acest timp la terminarea parcurgerii).

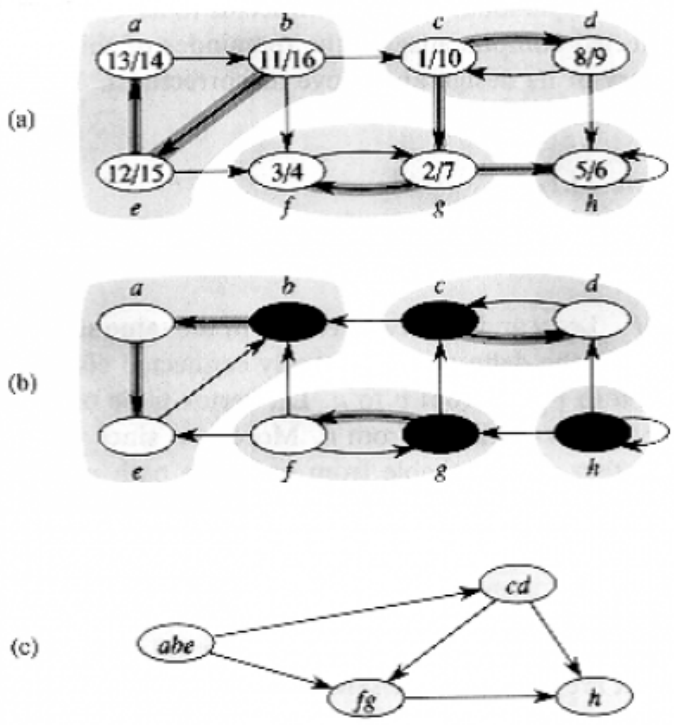
```
ctc(G = (V, E))
        S <- stiva vida
        culoare[1..n] = alb
        cat timp exista un nod v din V care nu e pe stiva
                dfs(G, v)
        culoare[1..n] = alb
        cat timp S != stiva vida
                v = pop(S)
                dfsT(GT, v) /* toate nodurile ce pot fi vizitate din v fac  parte din ctc; dupa vizitare, aceastea sunt scoase din S si din G */

dfs(G, v)
        culoare[v] = gri
        pentru fiecare (v, u) din E
                daca culoare[u] == alb
                        dfs(u)
        push(S, v) // nodul este terminat de expandat, este pus pe stiva
```

```
        culoare[v] = negru

dfsT(G, v) – similar cu dfs(G, v): fara stiva, dar cu retinerea solutiei
```

Complexitate: O(|V| + |E|)



Algoritmul lui Tarjan

Algoritmul foloseşte o singură parcurgere DFS şi o stivă. Ideea de bază a algoritmului este că o parcurgere în adâncime porneşte dintr-un nod de start. Componentele tare conexe formează subarborii arborelui de căutare, rădăcinile cărora sunt de asemenea rădăcini pentru componentele tare conexe.

Nodurile sunt puse pe o stivă, în ordinea vizitării. Când parcurgerea termină de vizitat un subarbore, nodurile sunt scoase din stivă şi se determină pentru fiecare nod dacă este rădăcina unei component tare conexe. Dacă un nod este rădăcina unei componente, atunci el şi toate nodurile scoase din stivă înaintea lui formează acea componenta tare conexă.

Pentru a determina daca un nod este radacina unei componente conexe, calculam pentru fiecare nod:

```
idx[v]     -> nivelul / ordinea de vizitare
lowlink[v] -> min { idx[u] | u este accesibil din v, folosind doar noduri din subarborele DFS al lui v (inclusiv v) }
```

v este rădăcina unei componente tare conexe ⇔ lowlink[v] = idx[v].

```
ctc_tarjan(G = (V, E))
        index = 0
        S = stiva vida
        pentru fiecare v din V
                daca (idx[v] nu e definit) // nu a fost vizitat
                        tarjan(G, v)

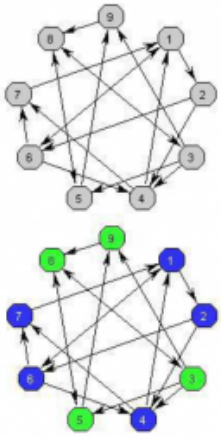
tarjan(G, v)
        idx[v] = index
        lowlink[v] = index
        index = index + 1
        push(S, v)

        pentru (v, u) din E
                daca (idx[u] nu e definit)
                        tarjan(G, u)
                        lowlink[v] = min(lowlink[v], lowlink[u])
                altfel
                        daca (u e in S)
                                lowlink[v] = min(lowlink[v], idx[u])

        daca (lowlink[v] == idx[v])
                // este v radacina unei CTC?
                print "O noua CTC: "
```

```
                repeat
                        u = pop(S)
                        print u
                until (u == v)
```

Complexitate: O(|V| + |E|)



	id	pre	low
1	1	0	10
2	1	1	10
3	2	5	10
4	1	2	10
5	2	6	10
6	1	4	10
7	1	3	10
8	2	7	10
9	2	8	10

Puncte de articulatie

Pentru determinarea punctelor de articulaţie într-un graf neorientat se foloseşte o parcurgere în adâncime modificată, reţinându-se informaţii suplimentare pentru fiecare nod. Acest algoritm a fost identificat tot de către Tarjan si este foarte similar cu algoritmul pentru determinarea CTC în grafuri orientate prezentat anterior. Fie T un arbore de adâncime descoperit de parcurgerea grafului. Atunci, un nod v este punct de articulaţie dacă:

- v este rădăcina lui T şi v are doi sau mai mulţi copii

sau

- v nu este rădăcina lui T şi are un copil u în T, astfel încât nici un nod din subarborele dominat de u nu este conectat cu un strămoş al lui v printr-o muchie înapoi (copii lui nu pot ajunge pe altă cale pe un nivel superior în arborele de adâncime).

Găsirea punctelor care se încadrează în primul caz este uşor de realizat.

Notăm:

```
idx[u] = timpul de descoperire a nodului u
low[u] = min( {idx[u]} U { idx[v] : (u, v) este o muchie înapoi } U
        { low[vi] : vi copil al lui u în arborele de adâncime} )
```

v este punct de articulaţie ⇔ low[u] ≥ idx[v], pentru cel puțin un copil u al lui v în T.

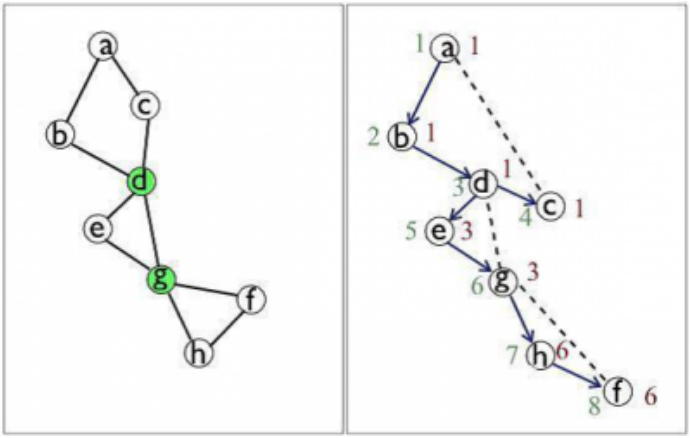
```
puncte_articulatie(G = (V, E))
        timp = 0
        pentru fiecare v din V
        daca (idx[v] nu e definit)
                dfsCV(G, v)

dfsCV(G, v)
        idx[v] = timp
        low[v] = timp
        timp = timp + 1
        copii = { } // multime vida
        pentru fiecare (v, u) din E
                daca u nu este parintele lui v
                        daca (idx[u] nu e definit)
                                // inseamna ca nodul u este nedescoperit, deci alb
                                copii = copii U {u}
                                dfsCV(G, u)
                                low[v] = min(low[v], low[u])
                        altfel
                                // inseamna ca nodul u este descoperit, deci gri, iar muchia v->u este muchie inapoi
                                low[v] = min(low[v], idx[u])

        daca v radacina arborelui
                daca |copii| >= 2
                        v este punct de articulatie
```

```
        altfel
                daca (∃u ∈ copii) astfel incat (low[u] >= idx[v])
                        v este punct de articulatie
```

Complexitate: O(|V| + |E|)



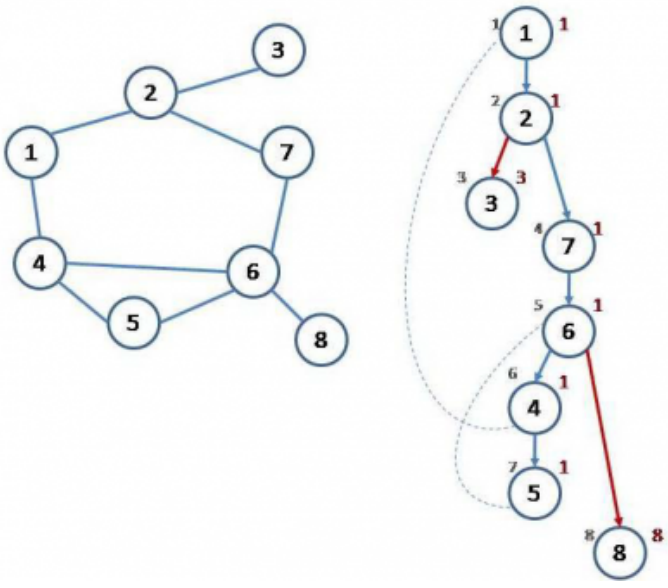
Punti

Pentru a determina muchiile critice se foloseşte tot o parcurgere în adâncime modificată, pornind de la următoarea observaţie: **muchiile critice sunt muchiile care nu apar în niciun ciclu.** Prin urmare, o muchie de întoarcere nu poate fi critică, deoarece o astfel de muchie închide întotdeauna un ciclu. Trebuie să verificăm pentru muchiile de avansare (în număr de |V| - 1) dacă fac parte dintr-un ciclu. Să considerăm că dorim să verificăm muchia de avansare (v, u).

Ne vom folosi de low[v] (definit la punctul anterior): dacă din nodul u putem să ajungem pe un nivel mai mic sau egal cu nivelul lui v, atunci muchia nu este critică, în caz contrar ea este critică.

```
dfsB(G, v, parinte)
        idx[v] = timp
        low[v] = timp
        timp = timp + 1
        pentru fiecare (v, u) din E
                daca u nu este parintele lui v
                        daca idx[u] nu este definit
                                dfsB(G, u, v)
                                low[v] = min(low[v], low[u])
                                daca (low[u] > idx[v])
                                        (v, u) este muchie critica
                        altfel
                                low[v] = min(low[v], idx[u])
```

Complexitate: O(|V| + |E|)



Componente biconexe

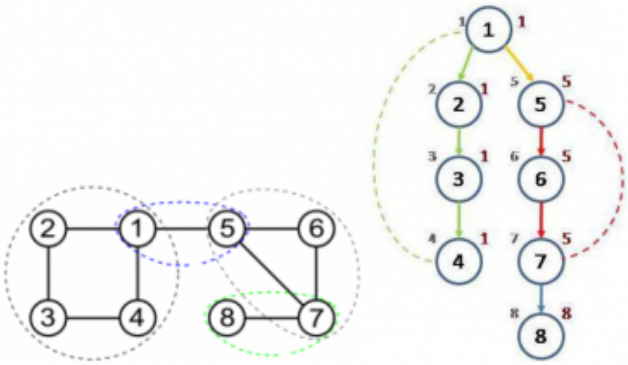
O componenta biconexă (sau biconectată) este o componentă a grafului care nu conţine puncte de articulatie. Astfel după eliminarea oricărui vârf din componenta curentă, restul vârfurilor vor rămâne conectate întrucât între oricare două vărfuri din aceeași componentă biconexă există cel puțin două căi disjuncte.

Astfel, pentru a determina componentele biconexe ale unui graf, vom adapta algoritmul de aflare a punctelor critice, reţinând şi o stivă cu toate muchiile de avansare şi de întoarcere parcurse până la un moment dat. La întâlnirea unui nod critic v se formează o nouă componentă biconexă pe care o vom determina extrăgând din stivă muchiile corespunzătoare. Nodul **v** este critic dacă am găsit un copil **u** din care nu se poate ajunge pe un nivel mai mic în arborele de adâncime pe un alt drum care foloseşte muchii de întoarcere **(low[u] >= idx[v])**. Atunci când găsim un astfel de nod **u**, toate muchiile aflate în stivă până la muchia (v, u) inclusiv formează o nouă componentă biconexă.

Nodul rădăcină trebuie tratat separat. Conform regulilor acestuia pentru a fi marcat nod critic (numărul de copii >= 2), este suficient să considerăm că fiecare copil face parte dintr-o componentă biconexă separată.

Atenție! Împărţirea în componente biconexe a unui graf neorientat reprezintă o partiție disjunctă a muchiilor grafului (împreună cu vârfurile adiacente muchiilor corespunzătoare fiecărei componente în parte). Acest lucru implică că unele vârfuri pot face parte din mai multe componente biconexe diferite. Care sunt acestea?

Complexitate: O(|V| + |E|)



Concluzii

Algoritmul de parcurgere în adâncime poate fi modificat pentru calculul componentelor tare conexe, a punctelor de articulaţie, a punţilor şi a componentelor biconexe. Complexitatea acestor algoritmi va fi cea a parcurgerii: O(|V| + |E|).

Referinte

[1] Introducere in Algoritmi, Thomas H. Cormen, Charles E. Leiserson, Ronald L. – Capitolul 23 Algoritmi elementari pe grafuri http://net.pku.edu.cn/~course/cs101/resource/Intro2Algorithm/book6/chap23.htm
[http://net.pku.edu.cn/~course/cs101/resource/Intro2Algorithm/book6/chap23.htm]

[2] Wikipedia – Algoritmul lui Tarjan http://en.wikipedia.org/wiki/Tarjan%27s_strongly_connected_components_algorithm
[http://en.wikipedia.org/wiki/Tarjan%27s_strongly_connected_components_algorithm]

[3] Wikipedia – Algoritmul lui Kosaraju http://en.wikipedia.org/wiki/Kosaraju%27s_algorithm
[http://en.wikipedia.org/wiki/Kosaraju%27s_algorithm]

[4] Wikipedia - Componente biconexe http://en.wikipedia.org/wiki/Biconnected_component
[http://en.wikipedia.org/wiki/Biconnected_component]

[5] Infoarena - Arhiva educationala - Componente tari conexe http://infoarena.ro/problema/ctc
[http://infoarena.ro/problema/ctc]

[6] Infoarena - Arhiva educationala - Componente biconexe http://infoarena.ro/problema/biconex
[http://infoarena.ro/problema/biconex]

[7] Infoarena - Arhiva educationala - 2SAT http://infoarena.ro/problema/2sat [http://infoarena.ro/problema/2sat]

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab08.zip.

Inainte de a rezolva exercitiile, asigurati-va ca ati citit si inteles toate precizarile din sectiunea Precizari laboratoare 07-12 [https://ocw.cs.pub.ro/courses/pa/skel_graph].

Prin citirea acestor precizari va asigurati ca:

- cunoasteti **conventiile** folosite
- evitati **buguri**
- evitati **depunctari** la lab/teme/test

CTC

Se da un graf **orientat** cu **n** noduri si **m** arce. Sa se gaseasca **componentele tare-conexe**.

Restrictii si precizari:

- $n <= 10^5$
- $m <= 2 * 10^5$
- timp de executie
  - C++: 1s
  - Java: 2s

Rezultatul se va returna sub forma unui vector, unde fiecare element este un vector (o CTC).

CUT VERTEX

Se da un graf **neorientat conex** cu **n** noduri si **m** muchii. Se cere sa se gaseaca toate **punctele critice**.

Restrictii si precizari:

- $n <= 10^5$
- $m <= 2 * 10^5$
- timp de executie
  - C++: 1s
  - Java: 2s

Rezultatul se va returna sub forma unui vector cu **X** elemente, unde X este numarul de puncte critice din graf.

CRITICAL EDGE

Se da un graf **neorientat conex** cu **n** noduri si **m** muchii. Se cere sa se gaseaca toate **muchiile critice**.

Restrictii si precizari:

- $n <= 10^5$
- $m <= 2 * 10^5$
- timp de executie
  - C++: 1s
  - Java: 2s

Rezultatul se va returna sub forma unui vector cu **X** elemente, unde X este numarul de muchii critice.

BONUS

Se da un graf **neorientat conex** cu **n** noduri si **m** muchii. Se cere sa se gaseaca toate **componentele biconexe**.

Pentru testare **trebuie** sa folositi problema biconex [https://infoarena.ro/problema/biconex] de pe infoarena.

Extra

Rezolvati problema retele [https://infoarena.ro/problema/retele] pe infoarena.
Rezolvati **ACASA** problema clepsidra [https://infoarena.ro/problema/clepsidra] pe infoarena.

Rezolvati **ACASA** problema course-schedule [https://leetcode.com/problems/course-schedule/description/] pe leetcode. (aplicatie tipuri de muchii)

pa/laboratoare/laborator-08.txt · Last modified: 2020/04/22 18:53 by darius.neatu

Laborator 09: Drumuri minime

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

- Înțelegerea conceptelor de cost, relaxare a unei muchii, drum minim
- Prezentarea si asimilarea algoritmilor pentru calculul drumurilor minime

Importanţă – aplicaţii practice

Algoritmii pentru determinarea drumurilor minime au multiple aplicații practice si reprezintă clasa de algoritmi pe grafuri cel mai des utilizata:

- Rutare in cadrul unei rețele (telefonice, de calculatoare etc.)
- Găsirea drumului minim dintre doua locații (Google Maps, GPS etc.)
- Stabilirea unei agende de zbor in vederea asigurării unor conexiuni optime
- Asignarea unui peer / server de fișiere in funcție de metricile definite pe fiecare linie de comunicație

Concepte

Costul unei muchii si al unui drum

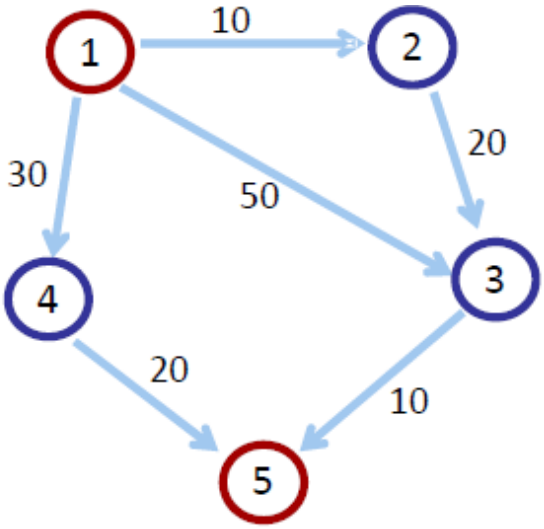
Fiind dat un graf orientat G = (V, E), se considera funcția w: E → W, numita funcție de cost, care asociază fiecărei muchii o valoare numerica. Domeniul funcției poate fi extins, pentru a include si perechile de noduri intre care nu exista muchie directa, caz in care valoarea este +∞ . Costul unui drum format din muchiile p12 p23 … p(n-1)n, având costurile w12, w23, …, w(n-1)n, este suma w = w12 + w23 + … + w(n-1)n.

In exemplul alăturat, costul drumului de la nodul 1 la 5 este:

drumul 1: w14 + w45 = 30 + 20 = 50

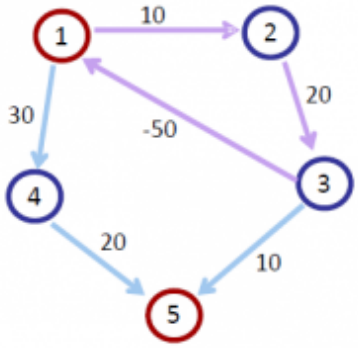
drumul 2: w12 + w23 + w35 = 10 + 20 + 10 = 40

drumul 3: w13 + w35 = 50 + 10 = 60



Drumul de cost minim

Costul minim al drumului dintre doua noduri este minimul dintre costurile drumurilor existente intre cele doua noduri. In exemplul de mai sus, drumul de cost minim de la nodul 1 la 5 este prin nodurile 2 si 3. Deși, in cele mai multe cazuri, costul este o funcție cu valori nenegative, exista situații in care un graf cu muchii de cost negativ are relevanta practica. O parte din algoritmi pot determina drumul corect de cost minim inclusiv pe astfel de grafuri. Totuși, nu are sens căutarea drumului minim in cazurile in care graful conține cicluri de cost negativ – un drum minim ar avea lungimea infinita, intrucat costul sau s-ar reduce la fiecare reparcurgere a ciclului:



In exemplul alăturat, ciclul 1 → 2 → 3 → 1 are costul -20.

drumul 1: w12 + w23 + w35 = 10 + 20 + 10 = 40

drumul 2: (w12 + w23 + w31) + w12 + w23 + w35 = -20 + 10 + 20 + 10 = 20

drumul 3: (w12 + w23 + w31) + (w12 + w23 + w31) + w12 + w23 + w35 = -20 + (-20) + 10 + 20 + 10 = 0

Relaxarea unei muchii

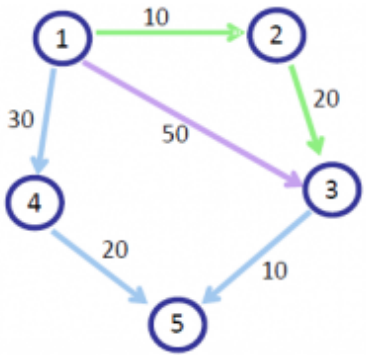
Relaxarea unei muchii v1 - v2 consta in a testa daca se poate **reduce distanta** de la **sursă** până la v2, trecând prin nodul intermediar v1 si apoi circuland pe muchia (v1,v2).

Fie w12 costul inițial al **muchiei directe** de la v1 la v2, w1u costul **drumului** de la v1 la u, si wu2 costul **drumului** de la u la v2. Daca w12 > w1u + wu2, **muchia directa** este înlocuita cu succesiunea de muchii v1 - u, u - v2. Dacă muchia nu există, se consideră **w12 = infinit**.

În acest exemplu, **muchia relaxată** este **u - v2** , întrucât aceasta reduce distanta pentru drumul de la **sursa** la v2, trecând prin v1.

In exemplul alăturat, muchia de la 1 la 3, de cost w13 = 50, poate fi relaxata la costul 30, prin nodul intermediar u = 2, fiind înlocuita cu succesiunea w12, w23.

Toți algoritmii prezentați in continuare se bazează pe relaxare pentru a determina drumul minim.



Drumuri minime de sursa unica

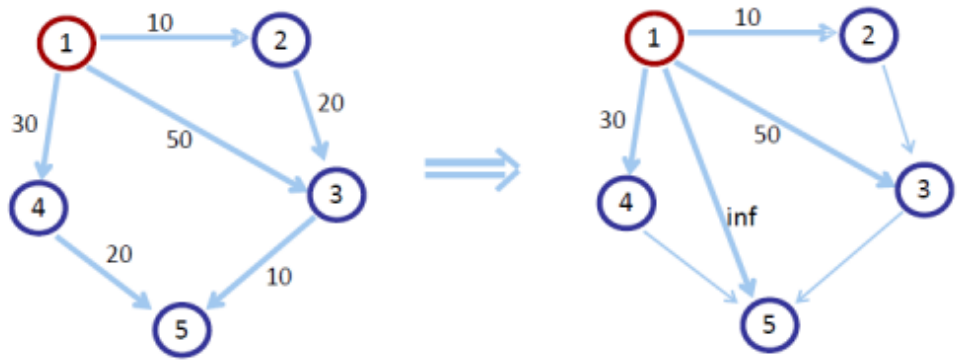
Algoritmii din aceasta secțiune determina drumul de cost minim de la un nod sursa, la restul nodurilor din graf, pe baza de relaxări repetate.

Algoritmul lui Dijkstra

Dijkstra poate fi folosit doar in grafuri care au toate muchiile nenegative.

Algoritmul este de tip Greedy:
optimul local căutat este reprezentat de costul drumului dintre nodul sursa s si un nod v. Pentru fiecare nod se retine un cost estimat d[v], inițializat la început cu costul muchiei s → v, sau cu +∞, daca nu exista muchie.

In exemplul următor, sursa s este nodul 1. Inițializarea va fi:



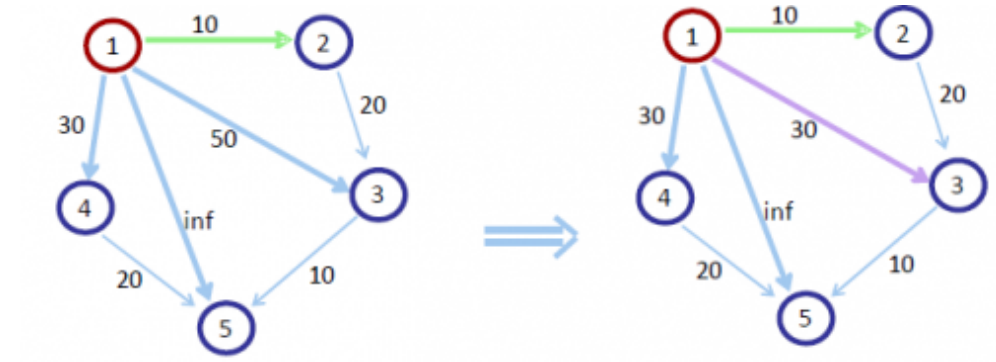
Aceste drumuri sunt îmbunătățite la fiecare pas, pe baza celorlalte costuri estimate.

Algoritmul selectează, in mod repetat, nodul u care are, la momentul respectiv, costul estimat minim (fata de nodul sursa). In continuare, se încearcă sa se relaxeze restul costurilor d[v]. Daca d[v] >= d[u] + wuv , d[v] ia valoarea d[u] + wuv.

Pentru a tine evidenta muchiilor care trebuie relaxate, se folosesc doua structuri: S (mulțimea de vârfuri deja vizitate) si Q (o coada cu priorități, in care nodurile se afla ordonate după distanta fata de sursa) din care este mereu extras nodul aflat la distanta minima. In S se afla inițial doar sursa, iar in Q doar nodurile spre care exista muchie directa de la sursa, deci care au d[nod] < +∞.

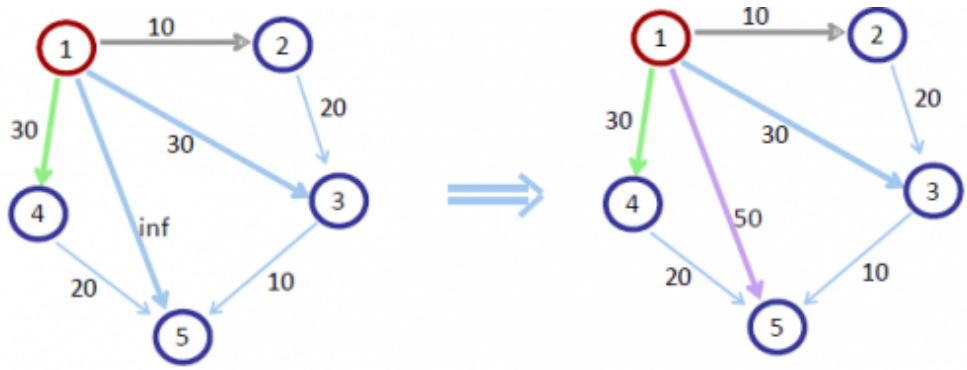
In exemplul de mai sus, vom inițializa S = {1} si Q = {2, 4, 3}.

La primul pas este selectat nodul 2, care are d[2] = 10.
Singurul nod pentru care d[nod] poate fi relaxat este 3 : d[3] = 50 > d[2] + w23 = 10 + 20 = 30



După primul pas, S = {1, 2} si Q = {4, 3}.

La următorul pas este selectat nodul 4, care are d[4] = 30.
Pe baza lui, se poate modifica d[5] : d[5] = +∞ > d[4] + w45 = 30 + 20 = 50



După al doilea pas, S = {1, 2, 4} si Q = {3, 5}.

La următorul pas este selectat nodul 3, care are d[3] = 30, si se modifica din nou d[5]: d[5] = 50 > d[3] + w35 = 30 + 10 = 40.

Algoritmul se încheie când coada Q devine vida, sau când S conține toate nodurile. Pentru a putea determina si muchiile din care este alcătuit drumul minim căutat, nu doar costul sau final, este necesar sa reținem un vector de părinți P. Pentru nodurile care au muchie directa de la sursa, P[nod] este inițializat cu sursa, pentru restul cu null.

Pseudocodul pentru determinarea drumului minim de la o sursa către celelalte noduri utilizând algoritmul lui Dijkstra este:

```
Dijkstra(sursa, dest):
introdu sursa in Q
d[sursa] = 0
d[nod] = +∞ // pentru orice nod != sursa
P[nod] = null // pentru orice nod din V

// relaxari succesive
cat timp Q nu e vida
    u = extrage_min (Q)
    selectat(u) = true
    foreach nod in vecini[u] // (*)
        /* daca drumul de la sursa la nod prin u este mai mic decat cel curent */
        daca not selectat(nod) si d[nod] > d[u] + w[u, nod]
            // actualizeaza distanta si parinte
            d[nod] = d[u] + w[u, nod]
            P[nod] = u
            /* actualizeaza pozitia nodului in coada prioritara */
            actualizeaza (Q,nod)

// gasirea drumului efectiv
Initializeaza Drum = {}
```

```
nod = P[dest]
cat timp nod != null
    insereaza nod la inceputul lui Drum
    nod = P[nod]
```

Reprezentarea grafului ca matrice de adiacenta duce la o implementare ineficienta pentru orice graf care nu este complet, datorita parcurgerii vecinilor nodului u, din linia (*), care se va executa în |V| pași pentru fiecare extragere din Q, iar pe întreg algoritmul vor rezulta |V|^2 pași. Este preferata reprezentarea grafului cu liste de adiacenta, pentru care numărul total de operații cauzate de linia (*) va fi egal cu |E|. Complexitatea algoritmului este O(|V|^2+|E|) în cazul în care coada cu priorități este implementata ca o căutare liniara. În acest caz funcția extrage_min se executa în timp O(|V|), iar actualizează(Q) in timp O(1).

O varianta mai eficienta este implementarea cozii ca heap binar. Funcția extrage_min se va executa în timp O(lg|V|); funcția actualizează(Q) se va executa tot în timp O(lg|V|), dar trebuie cunoscuta poziția cheii nod în heap, adică heapul trebuie sa fie indexat. Complexitatea obținută este O(|E|lg|V|) pentru un graf conex.

Cea mai eficienta implementare se obține folosind un heap Fibonacci pentru coada cu priorități:

Aceasta este o structura de date complexa, dezvoltata în mod special pentru optimizarea algoritmului Dijkstra, caracterizata de un timp amortizat de O(lg|V|) pentru operația extrage_min si numai O(1) pentru actualizeaza(Q). Complexitatea obținută este O(|V|lg|V| + |E|), foarte bună pentru grafuri rare.

Algoritmul Bellman – Ford

Algoritmul Bellman Ford poate fi folosit si pentru grafuri ce conțin muchii de cost negativ, dar nu poate fi folosit pentru grafuri ce conțin cicluri de cost negativ (când căutarea unui drum minim nu are sens).
Cu ajutorul sau putem afla daca un graf conține cicluri. Algoritmul folosește același mecanism de relaxare ca si Dijkstra, dar, spre deosebire de acesta, nu optimizează o soluție folosind un criteriu de optim local, ci parcurge fiecare muchie de un număr de ori egal cu numărul de noduri si încearcă sa o relaxeze de fiecare data, pentru a îmbunătăți distanta până la nodul destinație al muchiei curente.

Motivul pentru care se face acest lucru este ca drumul minim dintre sursa si orice nod destinație poate sa treacă prin maximum |V| noduri (adică toate nodurile grafului), respectiv |V|-1 muchii; prin urmare, relaxarea tuturor muchiilor de |V|-1 ori este suficienta pentru a propaga până la toate nodurile informația despre distanta minima de la sursa.

Daca, la sfârșitul acestor |E|*(|V|-1) relaxări, mai poate fi îmbunătățită o distanță, atunci graful are un ciclu de cost negativ si problema nu are soluție.

Menținând notațiile anterioare, pseudocodul algoritmului este:

```
BellmanFord(sursa):

d[sursa] = 0
d[nod] = +∞ // pentru orice nod != sursa
p[nod] = null // pentru orice nod din V

// relaxari succesive
// cum in initializare se face o relaxare (daca exista drum direct de la sursa la nod =>
// d[nod] = w[sursa, nod]) mai sunt necesare |V-1| relaxari
for i = 1 to |V|-1
    foreach (u, v) in E  // E = multimea muchiilor
        daca d[v] > d[u] + w(u,v)
            d[v] = d[u] + w(u,v)
            p[v] = u;

// daca se mai pot relaxa muchii
foreach (u, v) in E
    daca d[v] > d[u] + w(u,v)
        fail (”exista cicluri negativ”)
```

Complexitatea algoritmului este O(|E|*|V|).

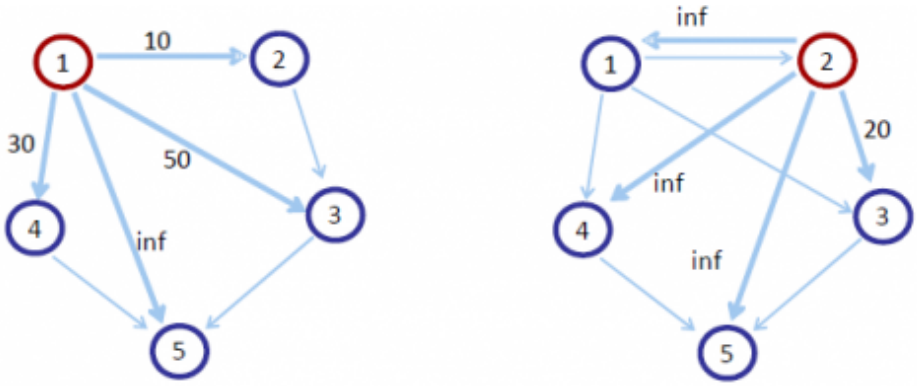
Drumuri minime intre oricare doua noduri

Floyd-Warshall

Algoritmii din aceasta secțiune determina drumul de cost minim dintre oricare doua noduri dintr-un graf. Pentru a rezolva aceasta problema s-ar putea aplica unul din algoritmii de mai sus, considerând ca sursa fiecare nod, pe rând, dar o astfel de abordare ar fi ineficienta.

Algoritmul Floyd-Warshall(intalnit si sub numele de Roy-Floyd) compara toate drumurile posibile din graf dintre fiecare 2 noduri, si poate fi utilizat si in grafuri cu muchii de cost negativ.

Estimarea drumului optim poate fi reținut intr-o structura tridimensionala d[v1, v2, k], cu semnificația – costul minim al drumului de la v1 la v2, folosind ca noduri intermediare doar noduri pana la nodul k. Daca nodurile sunt numerotate de la 1, atunci d[v1, v2, 0] reprezintă costul muchiei directe de la v1 la v2, considerând +∞ daca aceasta nu exista. Exemplu, pentru v1 = 1, respectiv 2:



Pornind cu valori ale lui k de la 1 la |V|, ne interesează să găsim cea mai scurta cale de la fiecare v1 la fiecare v2 folosind doar noduri intermedire din mulțimea {1, …, k}. De fiecare data, comparam costul deja estimat al drumului de la v1 la v2, deci d[v1, v2, k-1] obținut la pasul anterior, cu costul drumurilor de la v1 la k si de la k la v2, adică d[v1, k, k-1] + d[k, v2, k-1], obținutae la pasul anterior. Atunci, d[v1, v2, |V|] va conține costul drumului minim de la v1 la v2.

Pseudocodul acestui algoritm este:

```
FloydWarshall(G):
n = |V|
int d[n, n, n]
foreach (i, j) in (1..n,1..n)
    d[i, j, 0] = w[i,j] // costul muchiei, sau infinit
for k = 1 to n
    foreach (i,j) in (1..n,1..n)
        d[i, j, k] = min(d[i, j, k-1], d[i, k, k-1] + d[k, j, k-1])
```

Complexitatea temporala este O(|V|^3), iar cea spațială este tot O(|V|^3). O complexitate spațială cu un ordin mai mic se obține observând ca la un pas nu este nevoie decât de matricea de la pasul precedent d[i, j, k-1] si cea de la pasul curent d[i, j, k]. O observație și mai bună este că, de la un pas k-1 la k, estimările lungimilor nu pot decât sa scadă, deci putem sa lucram pe o singura matrice. Deci, spațiul de memorie necesar este de dimensiune |V|^2.

Rescris, pseudocodul algoritmului arata astfel:

```
FloydWarshall(G):
n = |V|
int d[n, n]
foreach (i, j) in (1..n,1..n)
    d[i, j] = w[i,j] // costul muchiei, sau infinit
for k = 1 to n
    foreach (i,j) in (1..n,1..n)
        d[i, j] = min(d[i, j], d[i, k] + d[k, j])
```

Pentru a determina drumul efectiv, nu doar costul acestuia, avem doua variante:

1. Se retine o structura de părinți, similara cu cea de la Dijkstra, dar, bineînțeles, bidimensionala.
2. Se folosește divide et impera astfel:

- se caută un pivot k astfel încât cost[i][j] = cost[i][k] + cost[j][k]
- se apelează funcția recursiv pentru ambele drumuri → (i,k),(k,j)
- dacă pivotul nu poate fi găsit, afișăm i
- după terminarea funcției recursie afișăm extremitatea dreapta a drumului

Cazuri speciale

1. Daca avem un graf **neorientat**, **fara cicluri** (un arbore), exista un singur drum intre oricare doua noduri, care poate fi aflat printr-o simpla parcurgere DFS. Folosind diferite preprocesari [8],[9], putem calcula distanta intre oricare doua noduri in timp constant, O(1).

2. Daca avem un graf **orientat**, **fara cicluri** (un DAG [10]), putem sa sa relaxam muchiile nodurilor, parcurgandu-le pe acestea in ordinea data de sortarea topologica. O(|V|+|E|)

3. Daca avem un graf unde toate muchiile au **cost egal**, putem afla distanta minima de la un nod sursa la orice alt nod printr-o parcurgere BFS. (de asemenea, tinand cont de faptul ca pot exista mai multe drumuri pana la un anumit nod). O(|V|+|E|)

4. Pentru grafuri orientate, **rare** (relativ putine muchii), putem folosi algoritmul lui Johnson([11]) pentru calcularea distantei minime de la un nod, la oricare alt nod. O(|V|^2log|V| + |V||E|)

Concluzii

- **Dijkstra***

– calculează drumurile minime de la o sursa către celelalte noduri
– nu poate fi folosit daca exista muchii de cost negativ
– complexitate minima O(|V|lg|V| + |E|) utilizând heapuri Fibonacci;

- **Bellman – Ford**

– calculează drumurile minime de la o sursă către celelalte noduri
– detectează existența ciclurilor de cost negativ
– complexitate O(|V| * |E|)

- **Floyd – Warshall**

– calculează drumurile minime intre oricare doua noduri din graf
– poate fi folosit in grafuri cu cicluri de cost negativ, dar nu le detectează
– complexitate O(|V|^3)

Referinţe:

[1] http://en.wikipedia.org/wiki/Dijkstra's_algorithm [http://en.wikipedia.org/wiki/Dijkstra's_algorithm]

[2] http://en.wikipedia.org/wiki/Bellman-Ford_algorithm [http://en.wikipedia.org/wiki/Bellman-Ford_algorithm]

[3] http://www.algorithmist.com/index.php/Floyd-Warshall's_Algorithm
[http://www.algorithmist.com/index.php/Floyd-Warshall's_Algorithm]

[4] http://en.wikipedia.org/wiki/Binary_heap [http://en.wikipedia.org/wiki/Binary_heap]

[5] http://en.wikipedia.org/wiki/Fibonacci_heap [http://en.wikipedia.org/wiki/Fibonacci_heap]

[6] T. Cormen, C. Leiserson, R. Rivest, C. Stein – Introducere în Algoritmi

[7] C. Giumale – Introducere în analiza algoritmilor

[8] http://en.wikipedia.org/wiki/Range_Minimum_Query [http://en.wikipedia.org/wiki/Range_Minimum_Query]

[9] http://en.wikipedia.org/wiki/Lowest_common_ancestor
[http://en.wikipedia.org/wiki/Lowest_common_ancestor]

[10] http://en.wikipedia.org/wiki/Directed_acyclic_graph [http://en.wikipedia.org/wiki/Directed_acyclic_graph]

[11] http://en.wikipedia.org/wiki/Johnson%27s_algorithm [http://en.wikipedia.org/wiki/Johnson%27s_algorithm]

Resurse

- Varianta PDF a laboratorului
  [http://cs.curs.pub.ro/2015/pluginfile.php/32415/mod_resource/content/1/Lab%208%20Drumuri%20minime.pdf]

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab09.zip.

Inainte de a rezolva exercitiile, asigurati-va ca ati citit si inteles toate precizarile din sectiunea Precizari laboratoare 07-12 [https://ocw.cs.pub.ro/courses/pa/skel_graph].

Prin citirea acestor precizari va asigurati ca:

- cunoasteti **conventiile** folosite
- evitati **buguri**
- evitati **depunctari** la lab/teme/test

Dijkstra

Se da un graf **orientat** cu **n** noduri si **m** arce. Graful are pe arce **costuri pozitive**.

Folositi **Dijkstra** pentru a gasi **costul minim** (**lungimea minima**) a unui drum de la o sursa data (**source**) la toate celelalte n - 1 noduri din graf.

Costul / lungimea unui drum este suma costurilor/lungimilor arcelor care compun drumul.

Restrictii si precizari:

- $n <= 50.000$
- $m <= 2.5 * 10^5$
- $0 <= c <= 20.000$, unde c este costul/lungimea unui arc
- timp de executie
  - C++: 1s
  - Java: 2s

Rezultatul se va returna sub forma unui vector **d** cu **n + 1** elemente.

Conventie:

- **d[node]** = costul minim / lungimea minima a unui drum de la **source** la nodul **node**
- **d[source] = 0**
- **d[node] = -1** , daca nu se poate ajunge de la **source** la **node**

d[0] nu este folosit, deci ca fi initializat cu 0! (am pastrat indexarea nodurilor de la 1)

Bellman-Ford

Se da un graf **orientat conex** cu **n** noduri si **m** arce. Graful are pe arce **costuri pozitive sau negative**.

Folositi **Bellman-Ford** pentru a gasi **costul minim** (**lungimea minima**) a unui drum de la o sursa data (**source**) la toate celelalte n - 1 noduri din graf. In caz ca se va detecta un ciclu de cost negativ, se va semnala acest lucru.

Costul / lungimea unui drum este suma costurilor/lungimilor arcelor care compun drumul.

Restrictii si precizari:

- $n <= 50.000$
- $m <= 2.5 * 10^5$
- $-1.000 <= c <= +1.000$, unde c este costul/lungimea unui arc
- timp de executie
    - C++: 1s
    - Java: 2s
- Pentru punctaj maxim, implementarea din laborator trebuie sa treaca toate testele, cu exceptia testelor 8 si 9, pe care va lua TLE. Pentru cei curiosi, exista si o implementare mai eficienta a algoritmului, oarecum similara cu cea de la Dijkstra (pentru mai multe detalii: https://infoarena.ro/problema/bellmanford [https://infoarena.ro/problema/bellmanford])

Rezultatul se va returna sub forma unui vector **d** cu **n + 1** elemente.

Conventie:

- **d[node]** = costul minim / lungimea minima a unui drum de la **source** la nodul **node**
- **d[source] = 0**
- **d[node] = -1** , daca nu se poate ajunge de la **source** la **node**

d[0] nu este folosit, deci ca fi initializat cu 0! (am pastrat indexarea nodurilor de la 1)

ATENTIE!!! Este posibil ca un astfel de graf sa aiba ciclu de cost negativ. In cazul detectarii unui ciclu de cost negativ, functia voastra va returna un vector gol! (**std::vector<int>() / {}** sau **new ArrayList<Integer>()**).

RoyFloyd

Se da un graf **orientat** cu **n** noduri. Graful are **costuri pozitive** pe arce.

Se da **matricea ponderilor** , se cere **matricea drumurilor minime**.

Restrictii si precizari:

- $n <= 100$
- $0 <= c <= 1.000$, unde c este costul unui arc
- daca **nu exista muchie** intre o pereche de noduri x si y, distanta de la nodul x la nodul y din **matricea ponderilor** va fi 0
- daca dupa aplicarea algoritmului **nu se gaseste drum** pentru o pereche de noduri x si y, se va considera **distanta** dintre ele egala cu 0 (se stocheaza in **matricea distantelor** valoarea 0)
- drumul de la nodul i la nodul i are lungime 0 (prin conventie)
- timp de executie
    - C++: 1s
    - Java: 2s

Rezultatul se va **stoca** in matricea **d** declarata in schelet! Algoritmul vostru trebuie doar sa o populeze corect, tinand cont ca nodurile sunt indexate de la 1.

BONUS

Pentru exercitiul cu Dijkstra, **reconstituiti** drumul de lungime minima **source** la celelalte noduri din graf.

Extra

Rezolvati problema rfinv [https://infoarena.ro/problema/rfinv] pe infoarena.
Rezolvati problema coach [https://infoarena.ro/problema/coach] pe infoarena.
Rezolvati problema rf [https://infoarena.ro/problema/rf] pe infoarena.
Rezolvati problema TODO [https://infoarena.ro/problema/TODO] pe infoarena.

pa/laboratoare/laborator-09.txt · Last modified: 2020/05/05 19:12 by darius.neatu

Laborator 10 - Arbori Minimi de Acoperire

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

- Însuşirea conceptului de arbore minim de acoperire;
- Înţelegerea modului de funcţionare a algoritmilor de determinare a unui arbore minim de acoperire prezentaţi;
- Aplicarea algoritmilor în rezolvarea problemelor;

Importanţă – aplicaţii practice

Găsirea unui arbore minim de acoperire pentru un graf are aplicaţii în domenii cât se poate de variate:

- Reţele (de calculatoare, telefonie, cablu TV, electricitate, drumuri): se doreşte interconectarea mai multor puncte, cu un cost redus şi atunci este utilă cunoaşterea arborelui care conectează toate punctele, cu cel mai mic cost posibil. STP(Spanning Tree Protocol) este un protocol de rutare care previne apariţia buclelor într-un LAN, şi se bazează pe crearea unui arbore de acoperire. Singurele legături active sunt cele care apar în acest arbore, iar astfel se evită buclele.
- Segmentarea imaginilor: împărţirea unei imagini în regiuni de pixeli cu proprietăţi asemănătoare. E utilă mai apoi în analiza medicală a unei zone afectate de o tumoare de exemplu.
- Algoritmi de aproximare pt probleme NP-dure: problema comis-voiajorului, arbori Steiner.
- Clustering: pentru detectarea de clustere cu forme neregulate [8], [9].

Descrierea problemei şi a rezolvărilor

Dându-se un graf conex neorientat G =(V, E), se numeşte arbore de acoperire al lui G un subgraf G'=(V, E') care conţine toate vârfurile grafului G şi o submulţime minimă de muchii E'⊆ E cu proprietatea că uneşte toate vârfurile şi nu conţine cicluri. Cum G' este conex şi aciclic, el este arbore. Pentru un graf oarecare, există mai mulţi arbori de acoperire.

Dacă asociem o matrice de costuri, w, pentru muchiile din G, fiecare arbore de acoperire va avea asociat un cost egal cu suma costurilor muchiilor conţinute. Un arbore care are costul asociat mai mic sau egal cu costul oricărui alt arbore de acoperire se numeşte arbore minim de acoperire (minimum spanning tree) al grafului G. Un graf poate avea mai mulţi arbori minimi de acoperire. Dacă toate costurile muchiilor sunt diferite, există un singur AMA. Primul algoritm pentru determinarea unui arbore minim de acoperire a fost scris în 1926 de Otakar Boruvka. În prezent, cei mai folosiţi algoritmi sunt Prim şi Kruskal. Toţi trei sunt algoritmi greedy, şi rulează în timp polinomial. La fiecare pas, pentru a construi arborele se alege cea mai bună variantă posibilă la momentul respectiv. Generic, algoritmul de determinare a unui AMA se poate scrie astfel:

```
ArboreMinimDeAcoperire(G(V, E), c)
        MuchiiAMA = ∅;
        while (MuchiiAMA nu reprezintă muchiile unui arbore minim de acoperire)
                găsește o muchie (u, v) care este sigură pentru MuchiiAMA;
                MuchiiAMA = MuchiiAMA ∪ {(u, v)};
        return MuchiiAMA;
```

O muchie sigură este o muchie care se poate adăuga unei submulţimi de muchii ale unui arbore minim de acoperire, astfel încât noua mulţime obţinută să aparţină tot unui arbore minim de acoperire. Iniţial, MuchiiAMA este o mulţime vidă. La fiecare pas, se adaugă câte o muchie sigură, deci MuchiiAMA rămâne o submulţime a unui AMA. În consecinţă, la sfarşitul rulării algoritmului (când muchiile din mulţime unesc toate nodurile din graf), MuchiiAMA va conţine de fapt arborele minim de acoperire dorit.

Algoritmul Kruskal

Algoritmul a fost dezvoltat în 1956 de Joseph Kruskal. Determinarea arborelui minim de acoperire se face prin reuniuni de subarbori minimi de acoperire. Iniţial, se consideră că fiecare nod din graf este un arbore. Apoi, la fiecare pas se selectează muchia de cost minim care uneşte doi subarbori disjuncţi, şi se realizează unirea celor doi subarbori. Muchia respectivă se adaugă la mulţimea MuchiiAMA, care la sfârşit va conţine chiar muchiile din arborele minim de acoperire.

Pseudocod

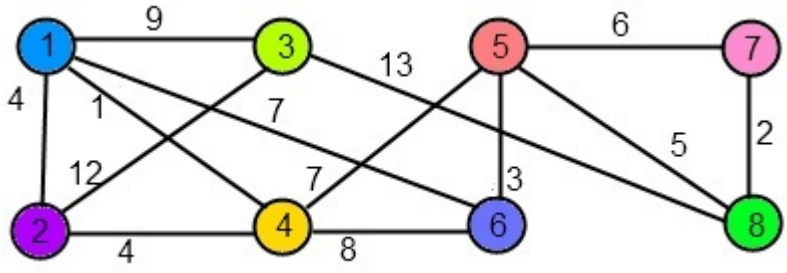
```
Kruskal(G(V, E), w)
MuchiiAMA <- Ø;
for each v in V do
        MakeSet(v);     //fiecare nod e un arbore diferit
sort(E);                //sortează muchiile în ordine crescătoare a costului
for each (u,v) in E do
        if (FindSet(u) != FindSet(v)) then      //capetele muchiei fac parte //din subarbori disjuncţi
                MuchiiAMA = MuchiiAMA ∪ {(u, v)};       //adaugă muchia la arbore
                Union(u, v);    //unește subarborii corespunzători lui u și v
return MuchiiAMA;
```

Bucla principală for poate fi înlocuită cu o buclă while, în care se verifică dacă în MuchiiAMA există mai puţin de |V| - 1 muchii, pentru că orice arbore de acoperire are |V| - 1 muchii, iar la fiecare pas se adaugă o muchie sigură.

Exemplu de rulare

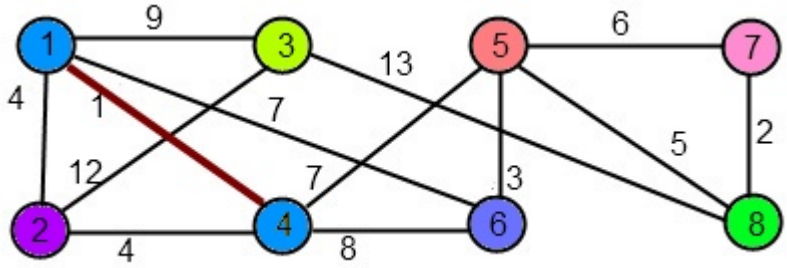
Se consideră graful din figura următoare:

Fiecare subarbore va fi colorat diferit. Cum iniţial fiecare nod reprezintă un subarbore, nodurile au culori diferite. Pe măsură ce subarborii sunt uniţi, nodurile aparţinând aceluiaşi subarbore vor fi coloraţi identic. Costurile muchiilor sunt sortate în ordine crescătoare.



Pas 1

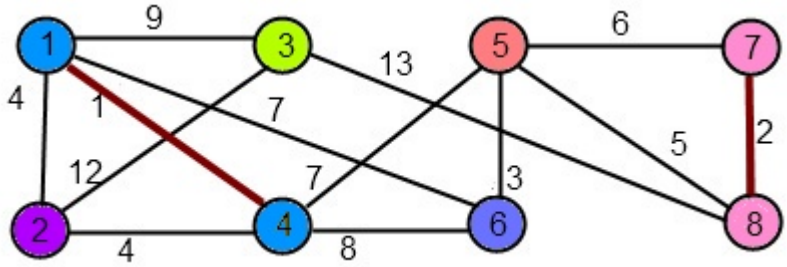
Se alege prima muchie, (1,4). Se observă că uneşte subarborii {1} şi {4}, deci muchia e adăugată la MuchiiAMA, iar cei doi subarbori se unesc.



MuchiiAMA = {(1,4)}.

Pas 2

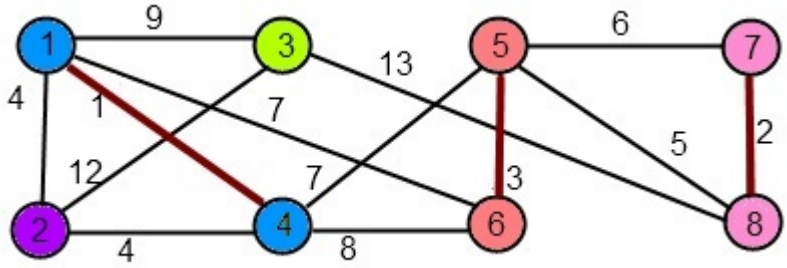
Următoarea muchie este (7,8), care uneşte {7} şi {8}. Se adaugă la MuchiiAMA şi se unesc cei doi subarbori.



MuchiiAMA = {(1,4),(7,8)}.

Pas 3

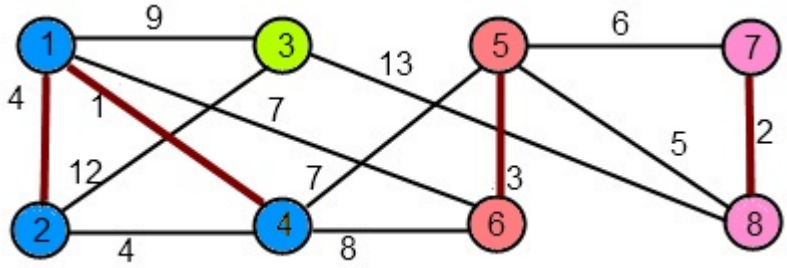
Următoarea muchie este (5,6), care uneşte {5} şi {6}. Se adaugă la MuchiiAMA şi se unesc cei doi subarbori.



MuchiiAMA = {(1,4),(7,8),(5,6)}.

Pas 4

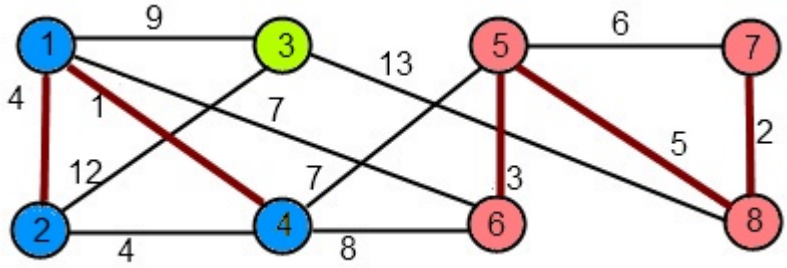
Următorul cost este 4. Se observă că muchiile (1,2) şi (2,4) au costul 4 şi unesc {2} cu {1,4}. Se adaugă la MuchiiAMA una dintre cele două muchii, fie ea (1,2), şi se unesc cei doi subarbori. Alegerea muchiei (2,4) va duce la găsirea unui alt AMA. [Am spus anterior că un graf poate avea mai mulţi arbori minimi de acoperire, cu acelaşi cost, dacă există muchii diferite cu acelaşi cost.]



MuchiiAMA = {(1,4),(7,8),(5,6),(1,2)}.

Pas 5

Următoarea muchie de cost minim este (5,8), care uneşte {5,6} şi {7,8}. Se adaugă la MuchiiAMA şi se unesc cei doi subarbori, rezultând {5,6,7,8}.



MuchiiAMA = {(1,4),(7,8),(5,6),(1,2), (5,8)}.

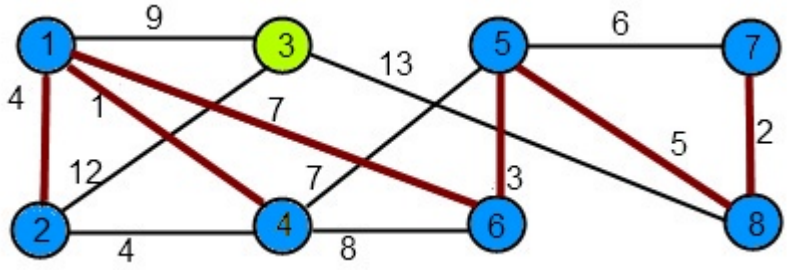
Pas 6

Muchia (5,7), care are cel mai mic cost actual, are ambele extremităţi în subarborele {5,6,7,8}. În consecinţă, nu se efectuează nicio schimbare.

MuchiiAMA = {(1,4),(7,8),(5,6),(1,2),(5,8)}.

Pas 7

Următorul cost este 7. Se observă că muchiile (1,6) şi (4,5) au costul 7 şi unesc subarborii {1,2,4} şi {5,6,7,8}. Se adaugă la MuchiiAMA (1,6), şi se unesc cei doi subarbori. Alegerea muchiei (4,5) va duce la găsirea unui alt AMA.



MuchiiAMA = {(1,4),(7,8),(5,6),(1,2),(5,8),(1,6)}.

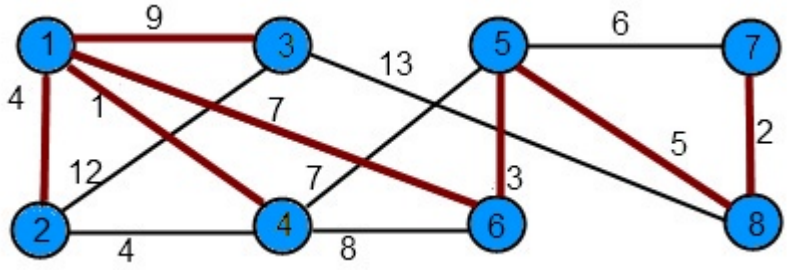
Pas 8

Muchia (4,6) de cost 8 are capetele în acelaşi subarbore, deci nu se produc schimări.

MuchiiAMA = {(1,4),(7,8),(5,6),(1,2),(5,8),(1,6)}.

Pas 9

Muchia (1,3) de cost 9 uneşte cei doi subarbori rămaşi, {1,2,4,5,6,7,8} şi {3}. Deci după unire obţinem un singur arbore. (1,3) se adaugă la MuchiiAMA, care va conţine acum 7 muchii, iar algoritmul se opreşte.



Arborele minim de acoperire obţinut este {(1,4),(7,8),(5,6),(1,2),(5,8),(1,6), (1,3)}.Costul său se calculează însumând costurile tuturor muchiilor:

Cost(MuchiiAMA) = 1 + 2 + 3 + 4 + 5 + 7 + 9 = 31

Alţi arbori minimi de acoperire pentru exemplul propus sunt: * {(1,4),(7,8),(5,6),(1,2),(5,8),(4,5), (1,3)} * {(1,4),(7,8),(5,6),(2,4),(5,8),(1,6), (1,3)} * {(1,4),(7,8),(5,6),(2,4),(5,8),(4,5), (1,3)}.

Pentru alte exemple explicate consultaţi [2], [3] şi [5].

Complexitate

Timpul de execuţie depinde de implementarea structurilor de date pentru mulţimi disjuncte. Vom presupune că se foloseşte o pădure cu mulţimi disjuncte [2][11][12]. Iniţializarea se face într-un timp O(|V|). Sortarea muchiilor în funcţie de cost se face în O(|E|log|E|). În bucla principală se execută |E| operaţii care presupun două operaţii de găsire a subarborilor din care fac parte extremităţile muchiilor şi eventual o reuniune a acestor arbori, într-un timp O(|E|log|E|).

Deci, complexitatea totală este: O(|V|) + O(|E|log|E|) + O(|E|log|E|) = O(|E|log|E|).

Cum |E| ⇐ |V|^2, şi O(log(|V|^2)) = O(2*log(|V|)) = O(log(|V|)), rezultă o complexitate O(|E|log|V|).

Algoritmul Prim

Algoritmul a fost prima oară dezvoltat în 1930 de matematicianul ceh Vojtěch Jarnik, şi independent în 1957 de informaticianul Robert Prim, al cărui nume l-a luat. Algoritmul consideră iniţial că fiecare nod este un subarbore independent, ca şi Kruskal. Însă spre deosebire de acesta, nu se construiesc mai mulţi subarbori care se unesc şi în final ajung să formeze AMA, ci există un arbore principal, iar la fiecare pas se adaugă acestuia muchia cu cel mai mic cost care uneşte un nod din arbore cu un nod din afara sa. Nodul rădăcină al arborelui principal se alege arbitrar. Când s-au adăugat muchii care ajung în toate nodurile grafului, s-a obţinut AMA dorit. Abordarea seamănă cu algoritmul Dijkstra de găsire a drumului minim între două noduri ale unui graf.

Pentru o implementare eficientă, următoarea muchie de adăugat la arbore trebuie să fie uşor de selectat. Vârfurile care nu sunt în arbore trebuie sortate în funcţie de distanţa până la acesta (de fapt costul minim al unei muchii care leagă nodul dat de un nod din interiorul arborelui). Se poate folosi pentru aceasta o structură de heap. Presupunând că (u, v) este muchia de cost minim care uneşte nodul u cu un nod v din arbore, se vor reţine două informaţii:

- d[u] = w[u,v] distanţa de la u la arbore
- p[u] = v predecesorul lui u în drumul minim de la arbore la u.

La fiecare pas se va selecta nodul u cel mai apropiat de arborele principal, reunind apoi arborele principal cu subarborele corespunzător nodului selectat. Se verifică apoi dacă există noduri mai apropiate de u decât de nodurile care erau anterior în arbore, caz în care trebuie modificate distanţele dar şi predecesorul. Modificarea unei distanţe impune şi refacerea structurii de heap.

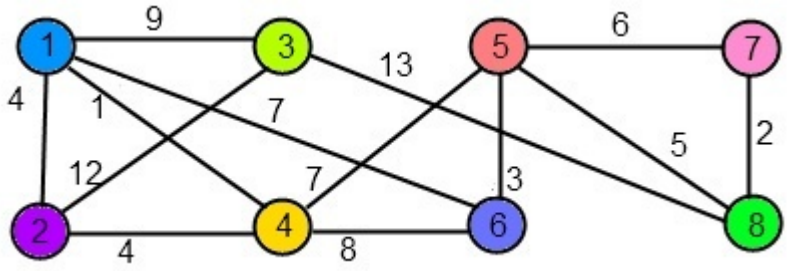
Pseudocod

```
Prim(G(V,E), w, root)
1.      MuchiiAMA <- Ø;
2.      for each u in V do
3.              d[u] = INF;     //inițial distanțele sunt infinit
4.              p[u] = NIL;     //și nu există predecesori
5.      d[root] = 0;    //distanța de la rădăcină la arbore e 0
6.      H = Heap(V,d);  //se construiește heap-ul
7.      while (H not empty) do  //cât timp mai sunt noduri neadăugate
8.              u = GetMin(H);  //se selectează cel mai apropiat nod u
9.              MuchiiAMA = MuchiiAMA ∪ {(u, p[u])};//se adaugă muchia care unește u cu un nod din arborele principal
10.             for each v in Adj(u) do
                //pentru toate nodurile adiacente lui u se verifică dacă
                //trebuie făcute modificări
11.                     if w[u][v] < d[v] then
12.                             d[v] = w[u][v];
13.                             p[v] = u;
14.                             Heapify(v, H);  //refacerea structurii de heap
15.     MuchiiAMA = MuchiiAMA \ {(root, p[root])};
16.     return MuchiiAMA;
```

Exemplu de rulare

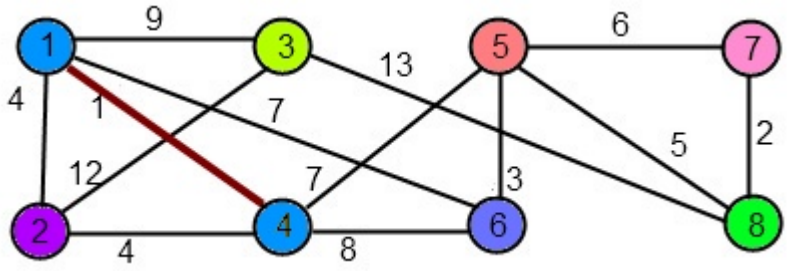
Se consideră graful folosit pentru exemplificarea algoritmului Kruskal.

Iniţial fiecare nod reprezintă un arbore independent, şi are o culoare unică. Se alege ca rădăcină a arborelui principal nodul 1.



Pas 1

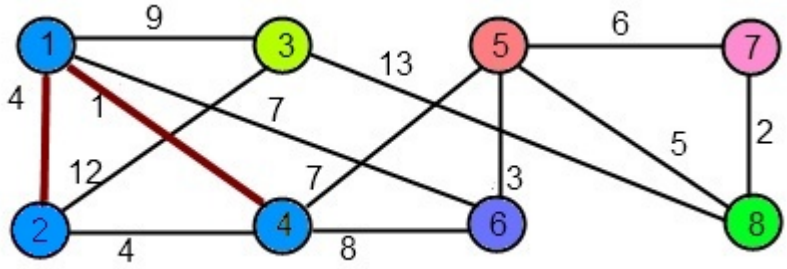
Se alege muchia de cost minim care uneşte rădăcina cu un alt nod: (1,4) de cost 1 si se adaugă arborelui principal.



MuchiiAMA = {(1,4)}.

Pas 2

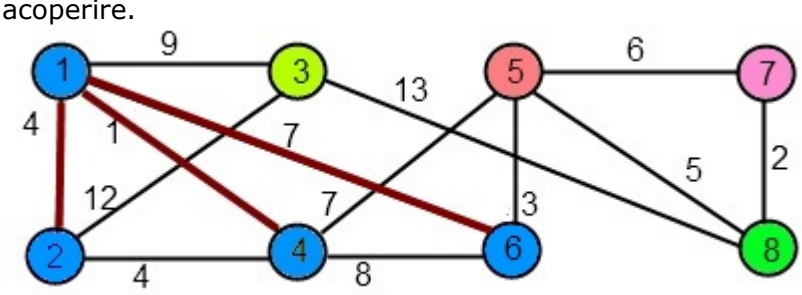
Se alege muchia de cost minim care uneşte 1 sau 4 cu un alt nod. (1,2) şi (2,4) au ambele costul 4. Se alege una dintre ele, fie ea (1,2).



MuchiiAMA = {(1,4),(1,2)}.

Pas 3

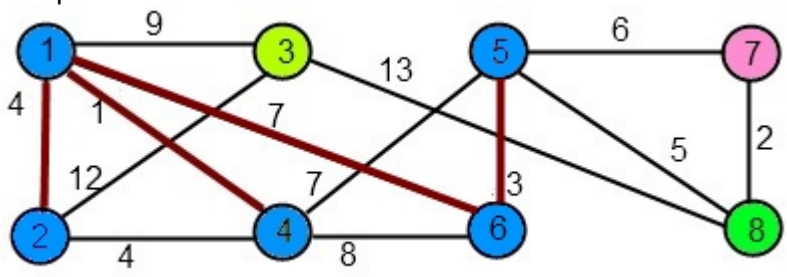
Se alege următoarea muchie care uneşte {1,4,2} cu alt nod. (1,6) şi (4,5) au acelaşi cost, dar adaugă arborelui noduri diferite. La acest pas se selectează de exemplu (1,6). Selectând la acest pas (4,5) s-ar obţine un alt arbore de acoperire.



MuchiiAMA = {(1,4),(1,2),(1,6)}.

Pas 4

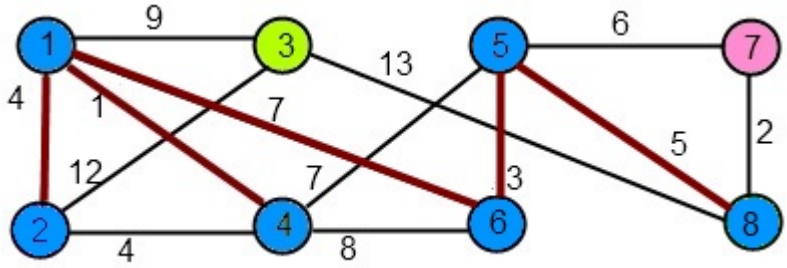
Se caută muchia de cost minim care unește {1,4,2,6} cu un alt nod, și se găsește (6,5) de cost 3. Nodul 5 va fi adăugat arborelui principal.



MuchiiAMA = {(1,4),(1,2),(1,6),(6,5)}.

Pas 5

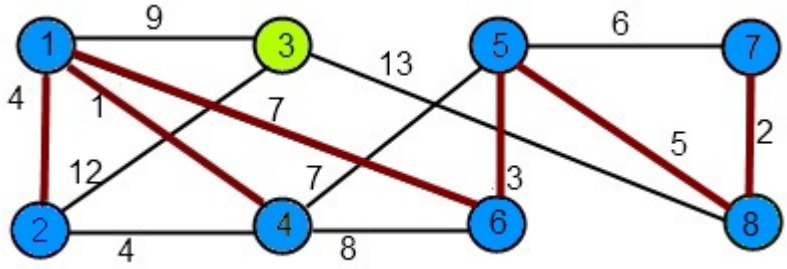
Se alege muchia de cost minim care unește {1,4,2,6,5} cu un alt nod. (5,8) de cost 5 se adaugă listei de muchii.



MuchiiAMA = {(1,4),(1,2),(1,6),(5,6),(5,8)}

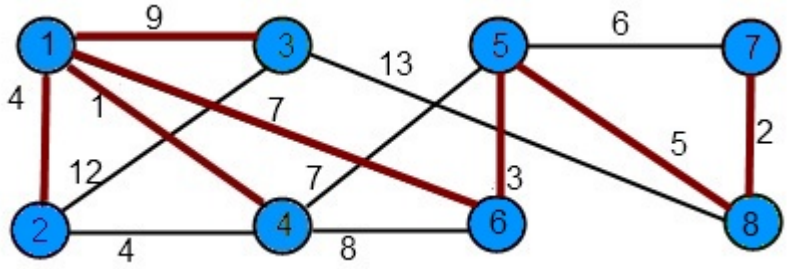
Pas 6

Se alege muchia de cost minim (8,7) care unește {1,4,2,6,5,8} cu nodul 7.



MuchiiAMA = {(1,4),(1,2),(1,6),(5,6),(5,8),(8,7)}.

Pas 7



Se alege muchia (1,3) de cost 9, care unește arborele principal cu ultimul nod ramas, și se adaugă mulțimii de muchii. Cum toate nodurile sunt acoperite, am obținut un arbore minim de acoperire.

MuchiiAMA = {(1,4),(1,2),(1,6),(5,6),(5,8),(8,7),(1,3)}.

Cost(MuchiiAMA) = 1 + 4 + 7 + 3 + 5 + 2 + 9 = 31

Alți arbori minimi de acoperire pentru exemplul propus se pot obține alegând diferit muchiile cu același cost (vezi pașii 2 și 3) .

Pentru alte exemple explicate consultaţi [2], [4] şi [6].

Complexitate

Iniţializările se fac în O(|V|). Bucla principală while se execută de |V| ori. Procedura GetMin() are nevoie de un timp de ordinul O(lg|V|), deci toate apelurile vor dura O(|V|lg|V|). Bucla for este executată în total de O(|E|) ori, deoarece suma tuturor listelor de adiacenţă este 2|E|. Modificarea distanţei, a predecesorului, şi refacerea heapului se execută într-un timp de O(1), O(1) şi respectiv O(lg|V|). Deci în total bucla interioară for durează O(|E|lg|V|).

În consecinţă, timpul total de rulare este O(|V|lg|V|+|E|lg|V|), adică O(|E|lg|V|). Aceeaşi complexitate s-a obţinut şi pentru algoritmul Kruskal. Totuşi, timpul de execuţie al algoritmului Prin se poate îmbunătăţi până la O(|E|+|V|lg|V|), folosind heap-uri Fibonacci.

Concluzii

Un arbore minim de acoperire al unui graf este un arbore care conţine toate nodurile, şi în plus acestea sunt conectate prin muchii care asigură un cost total minim. Determinarea unui arbore minim de acoperire pentru un graf este o problemă cu aplicaţii în foarte multe domenii: reţele, clustering, prelucrare de imagini. Cei mai cunoscuţi algoritmi, Prim şi Kruskal, rezolvă problema în timp polinomial. Performanţa algoritmilor depinde de modul de reprezentare a structurilor de date folosite.

Referinţe

[1] – http://en.wikipedia.org/wiki/Minimum_spanning_tree
[http://en.wikipedia.org/wiki/Minimum_spanning_tree]

[2] – T. Cormen, C. Leiserson, R. Rivest, C. Stein – Introducere în Algoritmi, cap. 24

[3] – http://en.wikipedia.org/wiki/Kruskal%27s_algorithm
[http://en.wikipedia.org/wiki/Kruskal%27s_algorithm]

[4] – http://en.wikipedia.org/wiki/Prim%27s_algorithm [http://en.wikipedia.org/wiki/Prim%27s_algorithm]

[5] – http://w3.cs.upt.ro/~calin/resources/sdaa/kruskal.ppt
[http://w3.cs.upt.ro/~calin/resources/sdaa/kruskal.ppt]

[6] – http://www.cs.upt.ro/~calin/resources/sdaa/prim.ppt
[http://www.cs.upt.ro/~calin/resources/sdaa/prim.ppt]

[7] – http://www.cs.princeton.edu/~wayne/kleinberg-tardos/04mst.pdf
[http://www.cs.princeton.edu/~wayne/kleinberg-tardos/04mst.pdf]

[8] – http://hc.ims.u-tokyo.ac.jp/JSBi/journal/GIW01/GIW01F03.pdf [http://hc.ims.u-tokyo.ac.jp/JSBi/journal/GIW01/GIW01F03.pdf]

[9] – http://www4.ncsu.edu/~zjorgen/ictai06.pdf [http://www4.ncsu.edu/~zjorgen/ictai06.pdf]

[10] – C. Giumale – Introducere în Analiza Algoritmilor, cap.5.5

[11] - Păduri de mulțimi disjuncte - Infoarena [https://www.infoarena.ro/problema/disjoint]

[12] - Păduri de mulțimi disjuncte - Wikipedia [https://en.wikipedia.org/wiki/Disjoint-set_data_structure]

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab10.zip.

Inainte de a rezolva exercitiile, asigurati-va ca ati citit si inteles toate precizarile din sectiunea Precizari laboratoare 07-12 [https://ocw.cs.pub.ro/courses/pa/skel_graph].

Prin citirea acestor precizari va asigurati ca:

- cunoasteti **conventiile** folosite
- evitati **buguri**
- evitati **depunctari** la lab/teme/test

Kruskal

Se da un graf **neorientat conex** cu **n** noduri si **m** muchii. Folositi algoritmul lui Kruskal pentru a calcula costul minim al unui arbore de acoperire.

Restrictii si precizari:

- $n <= 2 * 10^5$
- $m <= 4 * 10^5$
- $-10^3 <= c <= 10^3$, unde c este costul unei muchii

Prim

Aceeasi cerinta si aceleasi restrictii ca mai sus, doar ca se cere gasirea costului minim al unui arbore de acoperire folosind algoritmul lui Prim.

pa/laboratoare/laborator-10.txt · Last modified: 2020/04/24 10:12 by darius.neatu

Laborator 11 - Flux Maxim

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

- formalizarea noțiunilor de rețea de transport și flux în rețea
- prezentarea unei metode de rezolvare a problemei de flux maxim
- analiza unei implementări a metodei oferite

Importanţă – aplicaţii practice

Un graf orientat poate fi utilizat pentru modelarea unui proces de transport într-o rețea între un producător *s* și un consumator *t*. Destinația nu poate consuma mai mult decât se produce, iar cantitatea trimisă pe o cale nu poate depăși capacitatea sa de transport.

Rețelele de transport pot modela curgerea lichidului în sisteme cu țevi, deplasarea pieselor pe benzi rulante, deplasarea curentului prin rețele electrice, transmiterea informațiilor prin rețele de comunicare etc.

Descrierea problemei şi a rezolvărilor

O problemă des întâlnită într-o rețea de transport este cea a găsirii fluxului maxim posibil prin arcele rețelei astfel încât:

1. să nu fie depășite capacitățile arcelor

2. fluxul să se conserve în drumul său de la *s* la *t*

Definiție 1

O rețea de transport este un graf orientat $G = (V,E)$ cu proprietățile:

1. există două noduri speciale în V: *s* este nodul sursă (sau producătorul) și *t* este nodul terminal (sau consumatorul).

2. este definită o funcție totală de capacitate $c: V \times V \rightarrow R+$ astfel încât:

- $c(u,v) = 0$ dacă $(u,v) \notin E$
- $c(u,v) \geq 0$ daca $(u,v) \in E$

3. pentru orice nod $v \in V \setminus \{s,t\}$ există cel puțin o cale $s \rightarrow v \rightarrow t$.

Definiție 2

Numim flux în rețeaua $G = (V,E)$ o funcție totală $f: V \times V \rightarrow R$ cu proprietățile:

1. Restricție de capacitate:

- $f(u,v) \leq c(u,v), \forall (u,v) \in V$ - fluxul printr-un arc nu poate depăși capacitatea acestuia

2. Antisimetrie:

- $f(u,v) = -f(v,u)$, $\forall u \in V, \forall v \in V$

3. Conservarea fluxului:

- $\Sigma f(u,v) = 0, \forall u \in V \setminus \{s,t\}, v \in V$

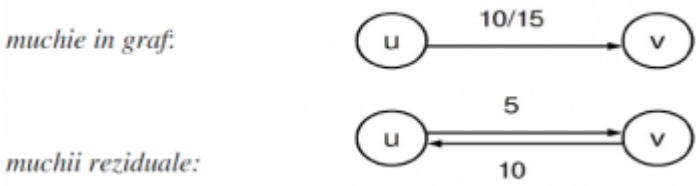
Un flux negativ de la u la v este unul virtual, el nu reprezintă un transport efectiv, ci doar sugerează că există un transport fizic de la v la u (este o convenție asemănătoare cu cea făcută pentru intensitățile curenților într-o rețea electrică). Ultima proprietate ne spune că la trecerea printr-un nod fluxul se conservă: suma fluxurilor ce intră într-un nod este 0 (ținând cont de convenția de semn stabilită).

Numim **capacitate reziduală** a unui arc

$c_f(u,v) = c(u,v) - f(u,v)$

și o interpretăm ca fiind cantitatea de flux adițional care poate fi transportat de la *u* la *v*, fără a depăși capacitatea $c(u,v)$.

Exemplu:



Daca avem arcul $(u,v) \in V$ cu $c(u,v) = 15$ și $f(u,v) = 10$, se pot transporta $c_f(u,v) = 5$ unități suplimentare fără a încălca restricția de capacitate. Dar, conform definiției, deși arcul $(v,u) \notin V$ vom avea totuși o capacitate reziduală $c_f(v,u) = c(v,u) - f(v,u) = 0 - (-10) = 10$: as putea transporta 10 unități în sens opus care să le anuleze pe cele 10 ale fluxului direct pe muchia (u,v).

Definiție 3

Fie o rețea de flux $G = (V,E)$, iar *f* fluxul prin G. Numim rețea reziduală a lui G, indusă de *f*, o rețea de flux notată cu $G_f = (V, E_f)$, astfel încât

- $E_f = \{(u,v) \in V \mid c_f(u,v) = c(u,v) - f(u,v) > 0\}$

Este important de observat că E_f și E pot fi disjuncte: un arc rezidual (u,v) apare în rețeaua reziduală doar dacă capacitatea sa este strict pozitivă (ceea ce nu implică existența arcului în rețeaua originală).

Un drum de ameliorare este o cale $(u1, u2, \ldots, uk)$, unde $u1 = s$ și $uk = t$, în graful rezidual cu $c_f(ui, ui+1) > 0, \forall i=1,2,\ldots,k-1$. Practic, un drum de ameliorare va reprezenta o cale în graf prin care se mai poate pompa flux adițional de la sursa la destinație.

Așa cum era de intuit, capacitatea reziduală a unui drum de ameliorare p este cantitatea maximă de flux ce se poate transporta de-a lungul lui:

- $c_f(p) = \min\{c_f(u,v) \mid (u,v) \in p\}$

Acum că am introdus noțiunile necesare pentru formalizarea problemei de flux maxim într-un graf, putem să prezentăm și cea mai utilizată metodă de rezolvare.

Algoritmul Ford-Fulkerson

Aceasta este o metodă iterativă de găsire a fluxului maxim într-un graf care pleacă de la ideea: cât timp mai există un drum de ameliorare (o cale de la sursă la destinație) pot pompa pe această cale un flux suplimentar egal cu capacitatea reziduală a căii.

Acest algoritm reprezintă mai mult un șablon de rezolvare pentru că nu detaliază modul în care se alege drumul de ameliorare din rețeaua reziduală.

```
Ford_Fulkerson
    Input G(V,E), s, t
    Output |fmax|
    |fmax| ← 0
    f{u,v) ← 0, ∀(u,v) ∈ V×V
    while ∃a path p(s ---> t) in Gf such that cf(u,v) > 0, ∀(u,v) ∈ p
        find cf(p) = min{cf(u,v) | cf(u,v) ∈ p}
        |fmax| += cf(p)
        for-each (u,v) ∈ p
            f(u,v) ← f(u,v) + cf(p)
            f(v,u) ← –f(u,v)
    return |fmax|
```

Complexitatea va fi $O(E * f max)$ pentru că în ciclul while putem găsi, în cel mai rău caz, doar cai care duc la creșterea fluxului cu doar o unitate la fiecare pas.

Ne punem acum problema dacă acest algoritm este corect, dacă la final vom avea cu adevărat fluxul maxim posibil prin graf. Corectitudinea algoritmului derivă imediat din teorema **Flux maxim - tăietura minimă**.

Numim o **tăietură a unui graf** o partiție (S,T) a nodurilor sale cu proprietatea $s \in S$ si $t \in T$.

Teorema flux maxim – tăietura minimă:

Pentru o rețea de flux $G(V,E)$ următoarele afirmații sunt echivalente:

1. f este fluxul maxim în G

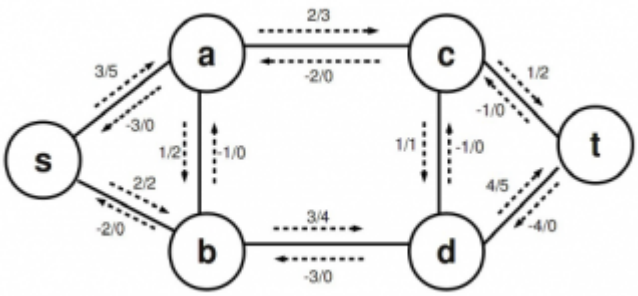
2. Rețeaua reziduală G_f nu conține drumuri de ameliorare

3. Există o tăietură (S,T) a lui G astfel încât fluxul net prin tăietură este egal cu capacitatea acelei tăieturi.

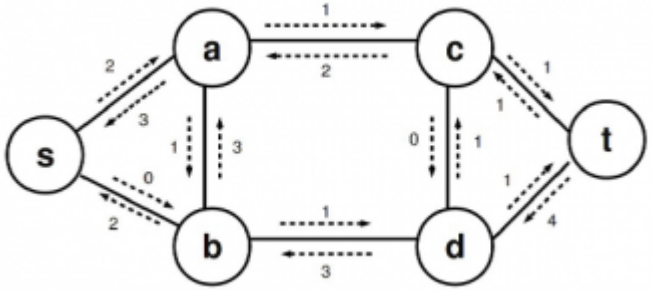
Obs: Prin orice tăietură fluxul este egal cu cel maxim pentru că nu există o altă cale pe care ar putea ajunge flux de la sursă la destinație și care să nu treacă prin tăietură (ar încălca tocmai definiția ei); sau, altfel spus, valoarea unui flux într-o rețea este dată de fluxul oricărei tăieturi. Astfel, fluxul total va fi mărginit de cea mai mică capacitate a unei tăieturi. Dacă este îndeplinit punctul 3. al teoremei atunci știm că acea tăietură nu poate fi decât una de capacitate minimă. Ultima incercare de a gasi o cale de la sursa la drena va rezulta in gasirea doar a elementelor marginite de o astfel de taietura.

Exemplu:

Rețeaua inițială:



Rețeaua reziduală:



Observăm că deși muchia `(d,c)` are capacitate 0 în rețeaua originală (ea nu ar putea transporta flux) în rețeaua reziduală avem `c(d,c) = 1` ceea ce îi permite să facă parte din drumul de ameliorare `p1 = (s, a, b, d, c, t)` de capacitate c_f`(p1) = 1`, astfel că adăugarea acestui flux suplimentar pe muchia `(d,c)` nu va duce la încălcarea restricției de capacitate; sau am fi putut alege drumul `p2=(s, a, c, t)` cu c_f`(p2) = 1` sau `p3 = (s,a,b,d,t)` cu c_f`(p3) = 1`. Performanța algoritmului ține de modul în care va fi ales drumul de ameliorare, se poate întâmpla ca pentru `|fmax|` de valoarea mare o alegere nepotrivită să ducă la timpi de execuție foarte mari.

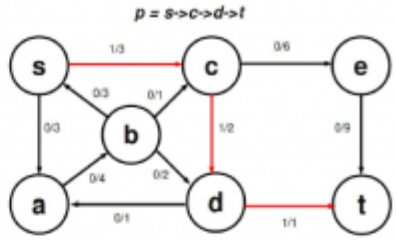
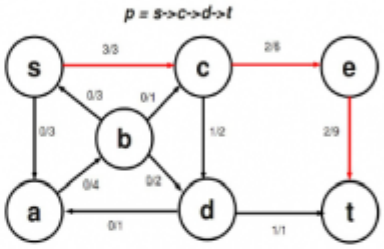
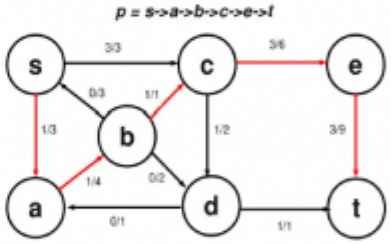
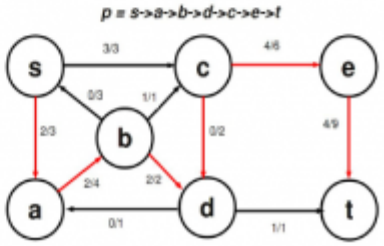
Implementarea Edmonds-Karp

Așa cum am văzut, algoritmul Ford-Fulkerson nu definește o metodă de alegere a drumului de ameliorare pe baza căruia se modifică fluxul în graf. Implementarea Edmonds-Karp alege întotdeauna cea mai scurtă cale folosind o căutare în lățime în graful rezidual unde fiecare arc are ponderea 1. Se poate demonstra că lungimea căilor găsite astfel crește monoton cu fiecare nouă ameliorare.

```
Edmonds-Karp
    Input G(V,E), s, t
    Output |fmax|
    |fmax| ← 0
    f(u,v) ← 0, ∀(u,v) ∈ V×V
    while true
        p(s ⇢ t) ← BFS(Gf,s,t)
        if not ∃p(s ⇢ t)
            break;
        find cf(p) = min{cf(u,v) | cf(u,v) ∈ p}
        |fmax| += cf(p)
        for-each (u,v) ∈ p
            f(u,v) ← f(u,v) + cf(p)
            f(v,u) ← –f(u,v)
    return |fmax|
```

Plecând de la ideea că drumurile de ameliorare găsite au lungimi din ce în ce mai mari se poate arăta că în această implementare fluxul se mărește de cel mult `O(V*E)` ori ([1], pag.513).

Complexitatea algoritmului va fi $O(V*E^2)$. Să luăm un exemplu de rulare al acestui algoritm. Vom considera starea rețelei după ce a fost găsita prima cale de pompare flux(inițial toate arcele sunt etichetate cu 0/0 conform notației stabilite):

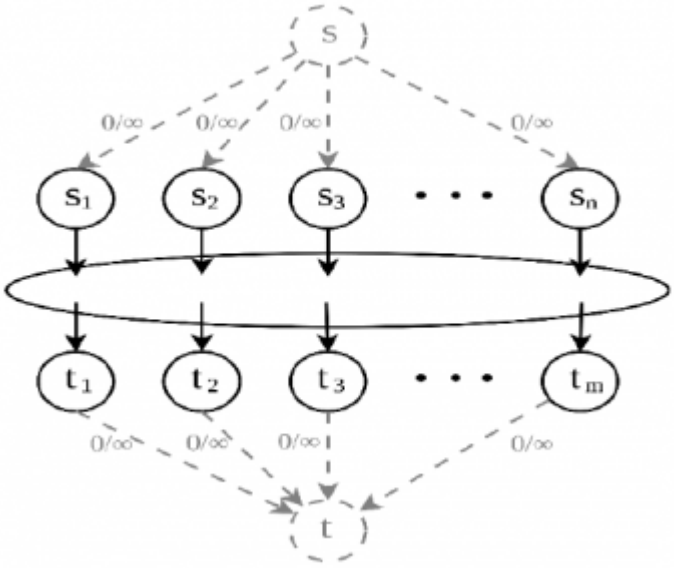
Capacitatea reziduala a caii de ameliorare	Graful si calea de ameliorare gasita
$c_f(p)=min(c_f(s,c),c_f(c,d),c_f(d,t))=$ $=min(3-0,2-0,1-0)=min(3,2,1)=1$	
$c_f(p)=min(c_f(s,c),c_f(c,e),c_f(e,t))=$ $=min(3-1,6-0,9-0)=min(2,6,9)=2$	
$c_f(p)=min(c_f(s,a),c_f(a,b),c_f(b,c),$ $c_f(c,e),c_f(e,t))=$ $=min(3-0,4-0,1-0,6-2,9-2)=$ $=min(3,4,1,4,7)=1$	
$c_f(p)=min(c_f(s,a),c_f(a,b),c_f(b,d),$ $c_f(d,c),c_f(c,e),c_f(e,t))=$ $=min(3-1,4-1,2-0,0-(-1),6-3,9-3)=$ $=min(2,3,2,1,3,6)=1$	

Obs: In cazul ultimului drum de ameliorare găsit in exemplul dat, se observa ca deși nu exista muchia `(d,c)` se simulează un flux pe aceasta printr-unul negativ in sens opus.

Variații ale problemei clasice

In rețelele clasice studiate pana acum aveam o sursa unica care putea produce oricât, destinația unica consuma oricât, orice nod intermediar conserva fluxul, iar singura constrângere a muchiilor era limitarea superioara a fluxului prin capacitate. Sa vedem cum putem generaliza aceste condiții si daca rețelele obținute ar putea fi reduse la una clasica.

Surse si destinații multiple



Se observă ca putem reduce acest graf la cel cunoscut prin adăugarea unei meta-surse legată de sursele mici prin muchii de capacitate nelimitata si analog un meta-terminal cu aceleași proprietăți. Global comportamentul rețelei de flux nu se va schimba.

Cuplaj bipartit maxim

Fiind dat un graf neorientat `G = (V,E)`, un cuplaj este o submultime de muchii `M` inclusa in `E` astfel incat pentru toate varfurile `v ∈ V`, exista cel mult o muchie in `M` incidenta in `v`. Spunem ca un varf `v ∈ M` este cuplat de cuplajul `M` daca exista o muchie in `M` incidenta in `v`. Un cuplaj maxim este un cuplaj de cardinalitate maxima. In cazul grafurilor bipartite, multimea de varfuri poate fi partitionata in `V = L U R`, unde `L` si `R` sunt disjuncte si toate muchiile din `E` sunt intre `L` si `R`. Problema poate fi rezolvata cu ajutorul notiunii de flux, construind reteaua de transport `G' = (V',E')` pentru graful bipartit `G`. Vom alege sursa *s* si destinatia *t* ca fiind noi varfuri care nu sunt in `V` si vom construi `V' = V U {s,t}`. Arcele orientate ale lui `G'` sunt date de:

- `E' = { (s, u) : u ∈ L } U { (u, v) : u ∈ L, v ∈ R si (u, v) ∈ E} U { (v, t) : v ∈ R}`

Pentru a completa constructia, vom atribui fiecarei muchii din `E'` capacitatea unitate.

Rețea cu noduri ce nu conservă fluxul

Spre deosebire de *s* si *t* care produc/consumă oricât, un nod intermediar ar putea produce sau consuma o cantitate constanta de flux la trecerea prin el. In acest caz vom avea un invariant la nivel de nod care ia forma:

- $f_{in} - f_{out} = di$, unde `di` este cantitatea produsă suplimentar (`> 0`) sau solicitată (`< 0`) de un nod.

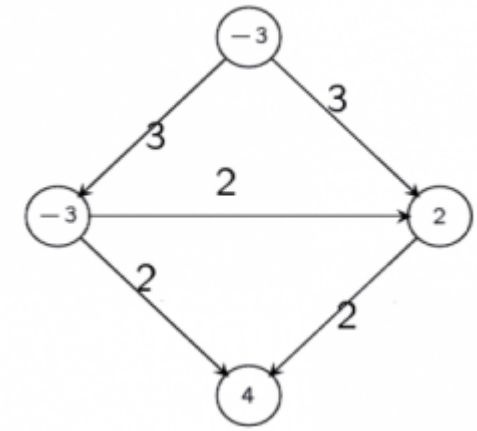
Am putea transforma egalitatea in:

- $(f_{in} + |di|) - f_{out} = 0$, dacă `di < 0`
- $f_{in} - (f_{out} + |di|) = 0$, daca `di > 0`

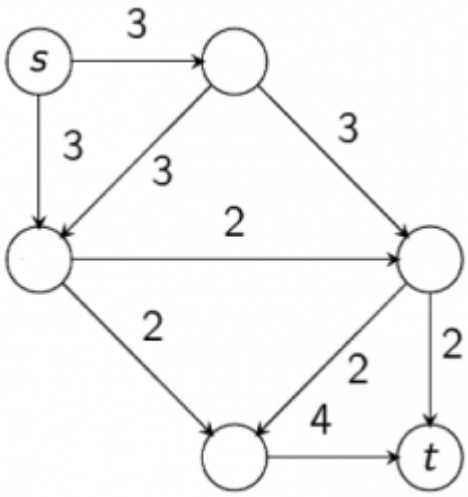
Altfel spus, un nod ce consumă flux poate fi transformat intr-unul ce conservă fluxul și are un in-arc adițional de capacitate `|di|`, iar unul ce produce flux va avea un out-arc de aceeași capacitate.

La nivelul unei rețele întregi se adaugă un nod sursă cu muchii către toate nodurile ce consumau flux si un nod destinație dinspre toate nodurile ce produceau flux.

Exemplu:



se va transforma in:

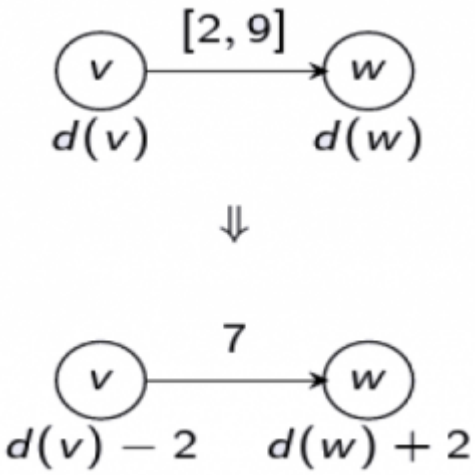


Rețea cu limite inferioare de capacitate

Există cazuri în care aș vrea ca datele de pe muchiile rețelei sa facă parte dintr-un anumit interval `[inf, sup]`. Pe o astfel de muchie fluxul trebuie să respecte inegalitatea `inf ≤ f ≤ sup`

Plecând tot de la condițiile de conservare la nivel de nod putem translata intervalul `[inf, sup]` în `[0, sup-inf]` și să considerăm că nodul sursă a consumat `inf` unități de flux iar nodul destinație a produs `inf` unități. Din exterior entitatea alcătuită din doua noduri și o muchie este văzută ca acționând în același fel asupra fluxului ce o traversează.

Iată un exemplu de transformare:



Bineînțeles că toate aceste transformări pot fi combinate pentru a se ajunge la rețeaua de flux clasică.

Concluzii și observații

Laboratorul de față s-a vrut a fi doar o introducere în domeniul fluxurilor într-un graf – modalitate de a reprezenta probleme de circulație a materialelor atât de frecvent întâlnite. In [1] și [2] găsiți și alți algoritmi interesanți împreună cu studiul complexității lor (algoritmul de pompare preflux, algoritmul „mutare-in-fata"). Spre exemplu, cel mai bun algoritm în prezent pentru cuplajul bipartit maxim se executa in $O(\sqrt{V}*E)$.

Referinte

[1] Introducere in algoritmi, Thomas H. Cormen, Charles E. Leiserson, Ronald R. Rivest – Capitolul VI Algoritmi pe grafuri: Flux maxim

[2] Introducere in analiza algoritmilor, Cristian A. Giumale – Cap. V Algoritmi pr grafuri: Fluxuri maxime intr-un graf

[3] Un articol de la MIT: A Labeling Algorithm for the maximum flow network problem [http://web.mit.edu/15.053/www/AMP-Appendix-C.pdf]

[4] Resurse wiki – Edmonds Karp [http://en.wikipedia.org/wiki/Edmonds%E2%80%93Karp_algorithm] Max Flow - Min Cut Theorem [http://en.wikipedia.org/wiki/Max-flow_min-cut_theorem]

[5] Aplicatii flux maxim [http://www.cs.princeton.edu/~wayne/cs423/lectures/max-flow-applications-4up.pdf]

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab11.zip.

Inainte de a rezolva exercitiile, asigurati-va ca ati citit si inteles toate precizarile din sectiunea Precizari laboratoare 07-12 [https://ocw.cs.pub.ro/courses/pa/skel_graph].

Prin citirea acestor precizari va asigurati ca:

- cunoasteti **conventiile** folosite
- evitati **buguri**
- evitati **depunctari** la lab/teme/test

Flux maxim

Se da un graf **orientat** cu **n** noduri si **m** arce. Graful are pe arce **capacitati pozitive**.

Folositi **Edmonds-Karp** pentru a gasi **fluxul maxim** intre nodul sursa **1** si nodul destinatie **n** in graful dat.

Restrictii si precizari:

- $n <= 1.000$
- $m <= 5.000$
- $1 <= c <= 110.000$, unde c este capacitatea unui arc
- Nu exista arc de la **n** la **1**.
- timp de executie
    - C++: 1s
    - Java: 2s

Rezultatul se va returna sub forma unui singur numar **F**, reprezentand fluxul maxim ce poate fi trimis prin retea.

BONUS

Rezolvati problema harta [https://infoarena.ro/problema/harta] pe infoarena.

pa/laboratoare/laborator-11.txt · Last modified: 2020/05/30 19:30 by darius.neatu

Laborator 12 : Algoritmi euristici de explorare a grafurilor. A*

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

In cadrul acestui laborator se va discuta despre modelarea problemelor sub forma grafurilor de stari si despre algoritmi specializati in gasirea solutiilor pentru acest tip de grafuri. De asemenea, se vor discuta modalitatile care pot fi folosite in analiza complexitatii si pe baza acestor metode se vor prezenta avantajele si limitarile acestei clase de algoritmi.

Importanţă – aplicaţii practice

Algoritmii de cautare euristica sunt folositi in cazurile care implica gasirea unor solutii pentru probleme pentru care fie nu exista un model matematic de rezolvare directa, fie acest model este prea complicat pentru a fi implementat. In acest caz e necesara o explorare a spatiului starilor problemei pentru gasirea unui raspuns. Intrucat o mare parte dintre problemele din viata reala pornesc de la aceste premise, gama de aplicatii a algoritmilor euristici este destul de larga. Proiectarea agentilor inteligenti [1], probleme de planificare, proiectare circuitelor VLSI [3], robotica, cautare web, algoritmi de aproximare pentru probleme NP-Complete [10], teoria jocurilor sunt doar cateva dintre domeniile in care cautarea informata este utilizata.

Descrierea problemei şi a rezolvărilor

Prezentare generală a problemei

Primul pas in rezolvarea unei probleme folosind algoritmi euristici de explorare este definirea exacta a problemei, prin tuplul (Si, O, Sf) – stare initiala, operatori, stari finale. Incercam sa cunoastem urmatorii parametri:

- **Starea initiala a problemei** – reprezinta configuratia de plecare.
- **Functie de expandare a nodurilor** – in cazul general este o lista de perechi (actiune, stare_rezultat). Astfel, pentru fiecare stare se enumera toate actiunile posibile precum si starea care va rezulta in urma aplicarii respectivei actiuni.
- **Predicat pentru starea finala** – functie care intoarce adevarat daca o stare este stare scop si fals altfel
- **Functie de cost** – atribuie o valoare numerica fiecarei cai generate in procesul de explorare. De obicei se foloseste o functie de cost pentru fiecare actiune/tranzitie, atribuind, astfel, o valoare fiecarui arc din graful starilor.

In functie de reprezentarea problemei, sarcina algoritmilor de cautare este de a gasi o cale din starea initiala intr-o stare scop. Daca algoritmul gaseste o solutie atunci cand multimea solutiilor este nevida spunem ca algoritmul este complet. Daca algoritmul gaseste si calea de cost minim catre starea finala spunem ca algoritmul este optim.

In principiu, orice algoritm pe grafuri discutat in laboratoarele si cursurile anterioare poate fi utilizat pentru gasirea solutiei unei probleme astfel definite. In practica, insa, multi dintre acesti algoritmi nu sunt utilizati

in acest context, fie pentru ca exploreaza mult prea multe noduri, fie pentru ca nu garanteaza o solutie pentru grafuri definite implicit (prin stare initiala si functie de expandare).

Algoritmii euristici de cautare sunt algoritmi care lucreaza pe grafuri definite ca mai sus si care folosesc o informatie suplimentara, necontinuta in definirea problemei, prin care se accelereaza procesul de gasirea a unei solutii. In cadrul explorarii starilor fiecare algoritm genereaza un arbore, care in radacina va contine starea initiala. Fiecare nod al arborelui va contine urmatorele informatii:

1. **Starea continuta** - stare(nod)
2. **Parintele nodului** – ɲ(nod)
3. **Cost cale** – costul drumului de la starea initiala pana la nod – g(nod)

De asemenea, pentru fiecare nod definim si o functie de evaluare **f** care indica cat de promitator este un nod in perspectiva gasirii unui drum catre solutie. (De obicei, cu cat **f** este mai mic, cu atat nodul este mai promitator). Am mentionat mai sus ca algoritmii de cautare euristica utilizeaza o informatie suplimentara referitoare la gasirea solutiei problemei. Aceasta informatie este reprezentata de o functie **h**, unde h(nod) reprezinta drumul estimat de la nod la cea mai apropiata stare solutie. Functia h poate fi definita in orice mod, existand o singura constrangere:

```
h(n)=0 pt ∀n,solutie(n)=adevarat
```

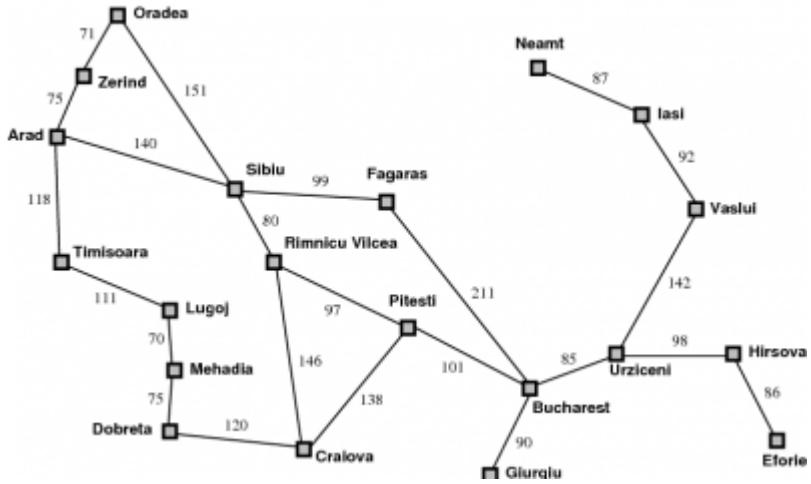
Algoritmi de cautare informata

Intrucat functioneaza prin alegerea, la fiecare pas, a nodului pentru care f(nod) este minim, algoritmii prezentati mai jos fac parte din clasa algoritmilor de cautare informata (nodul cel mai promitator este explorat mereu primul). Best-First tine cont doar de istoric (informatii sigure), pe cand A* estimeaza costul pana la gasirea unei solutii. De notat ca BFS si DFS (desi sunt algoritmi de cautare neinformata) sunt particularizari ale Best-First:

- pentru BFS: f = adancime (S)
- pentru DFS: f = - adancime (S)

Vom prezenta in continuare cativa dintre cei mai importanti algoritmi de cautare euristica. Vom folosi pentru exemplificare, urmatoarea problema: data fiind o harta rutiera a Romaniei, sa se gaseasca o cale (de preferinta de cost minim) intre Arad si Bucuresti.

Pentru aceasta problema starea initiala indica faptul ca ne aflam in orasul Arad, starea finala este data de predicatul Oras_curent == Bucuresti, functia de expandare intoarce toate orasele in care putem ajunge dintr-un oras dat, iar functia de cost indica numarul de km al fiecarui drum intre doua orase, presupunand ca viteza de deplasare este constanta. Ca euristica vom utiliza pentru fiecare oras distanta geometrica(in linie dreapta) pana la Bucuresti.



Greedy Best-First

In cazul acestui algoritm se considera ca nodul care merita sa fie expandat in pasul urmator este cel mai apropiat de solutie. Deci, in acest caz, avem:

f(n) = h(n), pt. oricare n

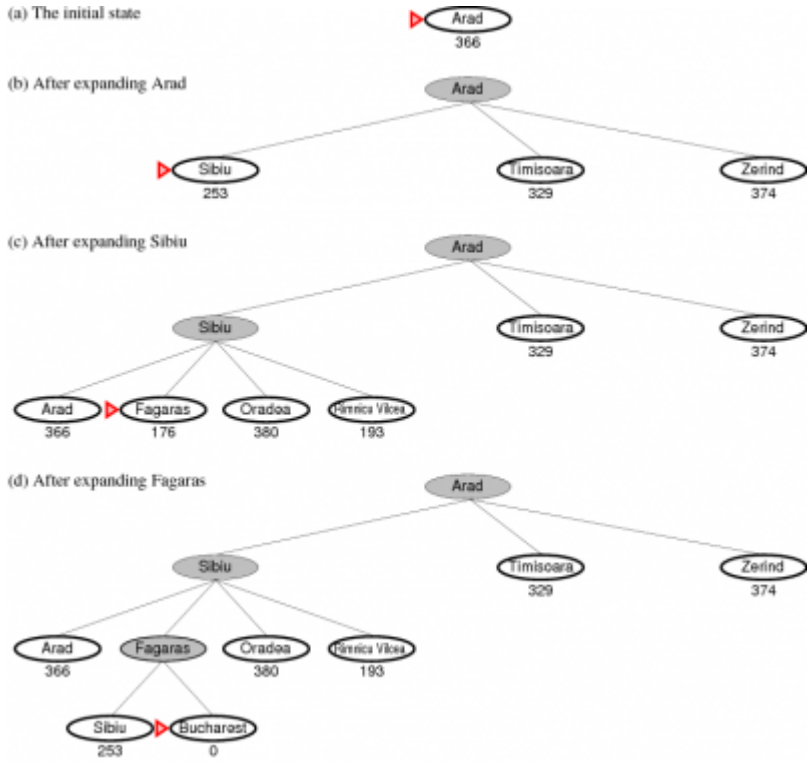
Pseudocodul acestui algoritm:

```
Greedy Best-First(sinitial , expand, h, solution)
    closed ← {}
    n ← new-node()
    state(n) ← sinitial
    π(n) ← nil
    open ← { n }
    // Bucla principala
    repeat
        if open = ∅ then return failure
        n ← get_best(open) with f(n) = h(n) = min
        open ← open - {n}
        if solution(state(n)) then return build-path(n)
        else if n not in closed then
            closed ← closed U {n}
            for each s in expand(n)
                n' ← new-node()
                state(n') ← s
                π(n') ← n
                open ← open U {n'}
            end-for
    end-repeat
```

In cadrul algoritmului se folosesc doua multimi: closed – indica nodurile deja explorate si expandate si open – nodurile descoperite dar neexpandate. Open este intializata cu nodul corespunzator starii intiale. La fiecare pas al algoritmului este ales din open nodul cu valoarea f(n) = h(n) cea mai mica (din acest motiv e de preferat ca open sa fie implementata ca o coada cu prioritati). Daca nodul se dovedeste a fi o solutie a problemei atunci este intoarsa ca rezultat calea de la starea initiala pana la nod (mergand recursiv din parinte in parinte). Daca nodul nu a fost deja explorat atunci se expandeaza iar nodurile corespunzatoare starilor rezultate sunt introduse in multimea open. Daca multimea open ramane fara elemente atunci nu exista niciun drum catre solutie si algoritmul intoarce esec.

Greedy Best-First urmareste mereu solutia care pare cea mai aproape de destinatie. Din acest motiv nu se vor analiza stari care desi par mai departate de solutie produc o cale catre solutie mai scurta (vezi exemplul de rulare). De asemenea, intrucat nodurile din closed nu sunt niciodata reexplorate se va gasi calea cea mai scurta catre scop doar daca se intampla ca aceasta cale sa fie analizata inaintea altor cai catre aceiasi stare scop. Din acest motiv, algoritmul nu este optim. De asemenea, pentru grafuri infinite e posibil ca algoritmul sa ruleze la infinit chiar daca exista o solutie. Rezulta ca algoritmul nu indeplineste nici conditia de completitudine.

In figura de mai jos se prezinta rularea algoritmului Greedy Best-First pe exemplul dat mai sus. In primul pas algoritmul expandeaza nodul Arad, iar ca nod urmator de explorat se alege Sibiu, intrucat are valoarea h(n) minima. Se alege in contiuare Fagaras dupa care urmeaza Bucuresti, care este un nod final. Se observa insa ca acest drum nu este minimal. Desi Fagaras este mai aproape ca distanta geometrica de Bucuresti, in momentul in care starea curenta este Sibiu alegerea optimala este Ramnicu-Valcea. In continuare ar fi urmat Pitesti si apoi Bucuresti obtinandu-se un drum cu 32 km mai scurt.



A* (A star)

A* reprezinta cel mai cunoscut algoritm de cautare euristica. El foloseste, de asemenea o politica Best-First, insa nu sufera de aceleasi defecte pe care le are Greedy Best-First definit mai sus. Acest lucru este realizat prin definirea functiei de evaluare astfel:

```
f(n) = g(n) + h(n) , pt. oricare n ∈ Noduri
```

A* evalueaza nodurile combinand distanta deja parcursa pana la nod cu distanta estimata pana la cea mai apropiata stare scop. Cu alte cuvinte, pentru un nod n oarecare, **f(n) reprezinta costul estimat al celei mai bune solutii care trece prin n**. Aceasta strategie se dovedeste a fi completa si optimala daca euristica h(n) este admisibila:

```
0 ≤ h(n) ≤ h*(n) , pt. oricare n ∈ Noduri
```

unde h*(n) este distanta exacta de la nodul n la cea mai apropiata solutie. Cu alte cuvinte A* intoarce mereu solutia optima daca o solutie exista atat timp cat ramanem optimisti si nu supraestimam distanta pana la solutie. Daca h(n) nu este admisibila o solutie va fi in continuare gasita, dar nu se garanteaza optimalitatea. De asemenea, pentru a ne asigura ca vom gasi drumul optim catre o solutie chiar daca acest drum nu este analizat primul, A* permite scoaterea nodurilor din closed si reintroducerea lor in open daca o cale mai buna pentru un nod din closed (g(n) mai mic) a fost gasita.

Algoritmul evolueaza in felul urmator: initial se introduce in multimea open (organizata ca o coada de prioritati dupa f(n)) nodul corespunzator starii initiale. **La fiecare pas se extrage din open nodul cu f(n) minim**. Daca se dovedeste ca nodul n contine chiar o stare scop atunci se intoarce calea de la starea initiala pana la nodul n. Altfel, daca nodul nu a fost explorat deja se expandeaza. Pentru fiecare stare rezultata, daca nu a fost generata de alt nod inca (nu este nici in open nici in closed) atunci se introduce in open. Daca exista un nod corespunzator starii generate in open sau closed se verifica daca nu cumva nodul curent produce o cale mai scurta catre s. Daca acest lucru se intampla se seteaza nodul curent ca parinte al nodului starii s si **se corecteaza distanta g**. Aceasta corectare implica reevaluarea tuturor cailor care trec prin nodul lui s, deci acest nod va trebui reintrodus in open in cazul in care era inclus in closed.

Pseudocodul pentru A* este prezentat in continuare:

```
// s_i      = stare initiala
// expand   = functie de expandare care returneaza toti succesorii unei stari
// h        = functie euristica (h(n) = aproximarea drumului minim de la n la o destinatie)
// is_goal  = is_goal(n) returneaza true daca n este o destinatie (stare goal/finala)
// w        = w[x][y] reprezinta costul muchiei directe intre starile x si y
A*(s_i, expand, h, is_goal)
        //initializari
    closed ← {}     // multimea nodurilor expandate
    open   ← { }    // multimea nodurilor in curs de explorare

    // pentru fiecare stare s
    for each s
        g(s) ← Infinity // initializare distante
        π(s) ← Infinity // initializare parinti

    // pentru s_i avem distanta 0 si parinte NULL
    g(s_i) ← 0
    π(s_i) ← NULL
    open   ← open U { s_i }

    // parcurgerea continua cat timp avem noduri in open
    while !open.empty()
        // extrage nodul s cu f(s) minim
        s ← get_best(open) with minimum f(s) = g(s) + h(s)
        open ← open - { s }

        // daca s este stare finala
        if is_goal(s)
            // reconstituie drumul de la s la s_i
            return build_path(s, s_i)

        // daca nodul nu este deja expandat
        if s not in closed
            // il adaug in closed
            closed ← closed U { s }

            // pentru fiecare copil c al starii s
            for each c in expand(s)
                if g(s) + w[s][c] < g(c)
                    // actualizez distanta si parintele
                    g(c) ← g(s) + w[s][c]
                    π(c) ← s

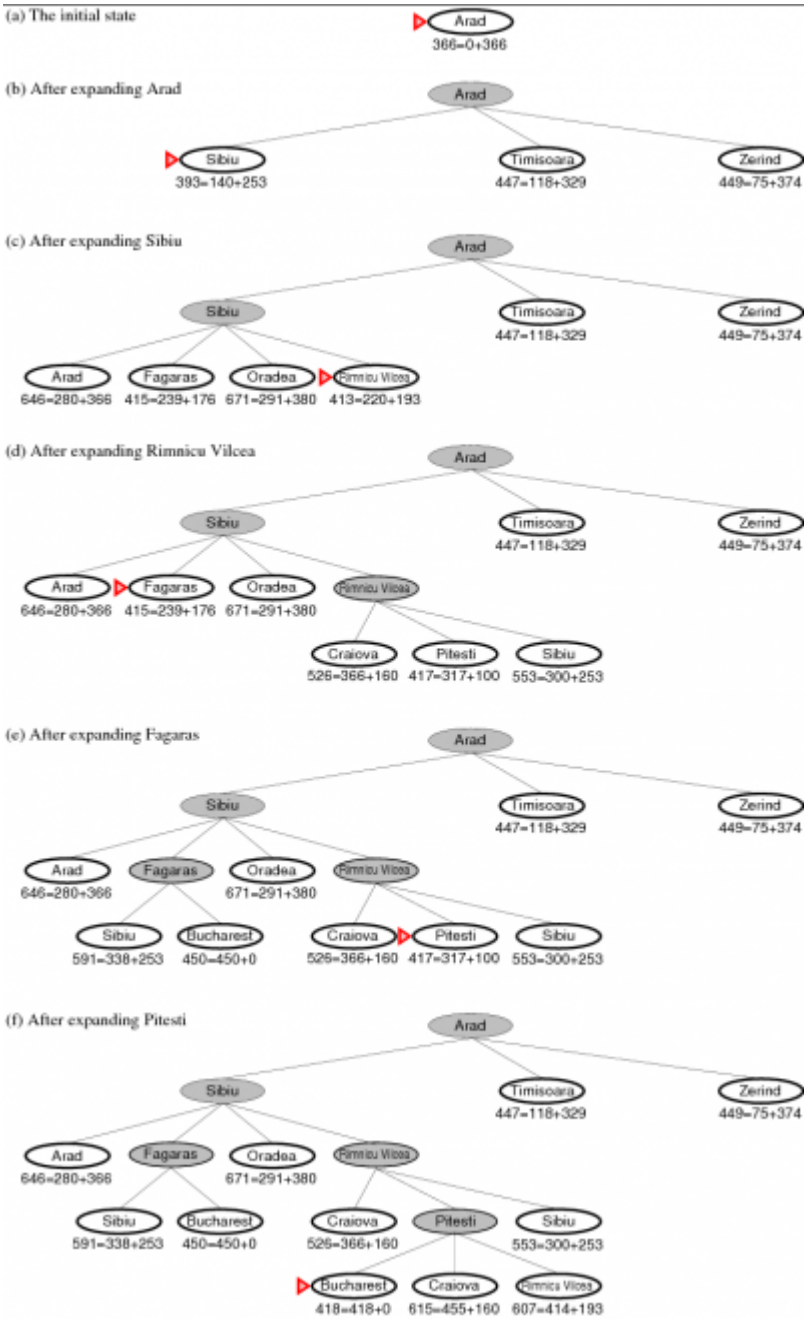
                    if not (c in closed U open) // daca nodul c nu a mai fost intalnit
                        open   ← open U { c }   // c intra in open
                    else if c in closed         // daca nodul c a mai fost expandat
                        closed ← closed – { c } // c trece din closed in open
                        open   ← open U { c }

    // nu a fost gasita o cale de la s_i la o stare goal s
    return failure
```

Algoritmul prezentat mai sus va intoarce calea optima catre solutie, daca o solutie exista. Singurul incovenient fata de Greedy Best-First este ca sunt necesare reevaluarile nodurilor din closed. Si aceasta problema poate fi rezolvata daca impunem o conditie mai tare asupra euristicii h, si anume ca euristica sa fie **consistenta** (sau **monotona**):

```
h(n) ≤ h(n') + cost(n,n') , pt. oricare n' ∈ expand(n)
```

Daca o functie este consistenta atunci ea este si admisibila. Daca euristica h indeplineste si aceasta conditie atunci algoritmul A* este asemanator cu Greedy Best-First cu modificarea ca functia de evaluare este f = g + h, in loc de f = h. In imaginea de mai jos se prezinta rularea algoritmului pe exemplul laboratorului. Se observa ca euristica aleasa (distanta in linie dreapta) este consistenta si deci admisibila. Se observa ca in pasul (e), dupa expandarea nodului Fagaras desi exista o solutie in multimea open aceasta nu este aleasa pentru explorare. Se va alege Pitesti, intrucat f(nod(Bucuresti)) = 450 > f(nod(Pitesti)) = 417, semnificatia acestei inegalitati fiind ca e posibil sa existe prin Pitesti un drum mai bun catre Bucuresti decat cel descoperit pana acum.



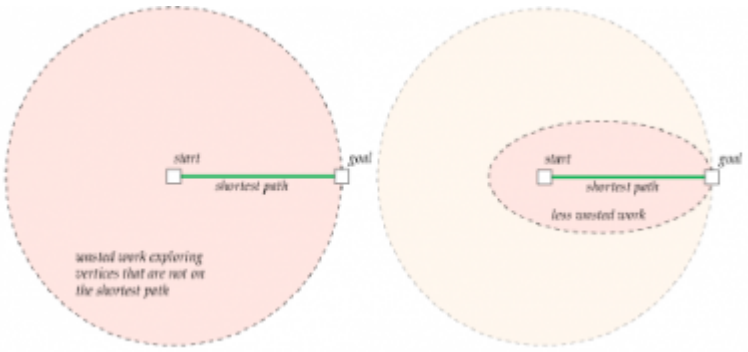
Complexitate algoritmului A*

Pe langa proprietatile de completitudine si optimalitate A* mai are o calitate care il face atragator: pentru o euristica data orice algoritm de cautare complet si optim va explora cel putin la fel de multe noduri ca A*, ceea ce inseamna ca A* este optimal din punctul de vedere al eficientei. In practica, insa, A* poate fi de multe ori imposibil de rulat datorita dimensiunii prea mari a spatiului de cautare. Singura modalitate prin care spatiul de cautare poate fi redus este prin gasirea unei euristici foarte bune – cu cat euristica este mai apropiata de distanta reala fata de stare solutie cu atat spatiul de cautare este mai strans (vezi figura de mai jos). S-a demonstrat ca spatiul de cautare incepe sa creasca exponential daca eroarea euristicii fata de distanta reala pana la solutie nu are o crestere subexponentiala:

```
|h(n) - h*(n)| ≤ O(logh*(n))
```

Din pacate, in majoritatea cazurilor, eroarea creste liniar cu distanta pana la solutie ceea ce face ca A* sa devina un algoritm mare consumator de timp, dar mai ales de memorie. Intrucat in procesul de cautare se retin toate nodurile deja explorate (multimea closed), in cazul unei dimensiuni mari a spatiului de cautare cantitatea de memorie alocata cautarii este in cele din urma epuizata. Pentru acest incovenient au fost

gasite mai multe solutii. Una dintre acestea este utilizarea unor euristici care sunt mai stranse de distanta reala pana la starea scop, desi nu sunt admisibile. Se obtin solutii mai rapid, dar nu se mai garanteaza optimalitatea acestora. Folosim aceasta metoda cand ne intereseaza mai mult sa gasim o solutie repede, indiferent de optimalitatea ei.



Volumul spatiului de cautare in functie de euristica aleasa

Alte abordari presupun sacrificarea timpului de executie pentru a margini spatiul de memorie utilizat [3]. Aceste alternative sunt prezentate in continuare.

IDA*, RBFS, MA*

IDA*

Iterative deepening A* [5] utilizeaza conceptul de adancire iterativa[1][8] in procesul de explorare a nodurilor – la un anumit pas se vor explora doar noduri pentru care functia de evaluare are o valoare mai mica decat o limita data, limita care este incrementata treptat pana se gaseste o solutie. IDA* nu mai necesita utilizarea unor multimi pentru retinerea nodurilor explorate si se comporta bine in cazul in care toate actiunile au acelasi cost. Din pacate, devine ineficient in momentul in care costurile sunt variabile.

RBFS

Recursive Best-First Search [6] functioneaza intr-un mod asemanator cu DFS, explorand la fiecare pas nodul cel mai promitator fara a retine informatii despre nodurile deja explorate. Spre deosebire de DFS, **se retine in orice moment cea mai buna alternativa sub forma unui pointer catre un nod neexplorat**. Daca valoarea functiei de evaluare (f = g + h) pentru nodul curent devine mai mare decat valoarea caii alternative, drumul curent este abandonat si se incepe explorarea nodului retinut ca alternativa. RBFS are avantajul ca spatiul de memorie creste doar liniar in raport cu lungimea caii analizate. Marele dezavantaj este ca se ajunge de cele mai multe ori la reexpandari si reexplorari repetate ale acelorasi noduri, lucru care poate fi dezavantajos mai ales daca exista multe cai prin care se ajunge la aceiasi stare sau functia expand este costisitoare computational (vezi problema deplasarii unui robot intr-un mediu real). RBFS este optimal daca euristica folosita este admisibila.

MA*

Memory-bounded A* este o varianta a A* in care se limiteaza cantitatea de memorie folosita pentru retinerea nodurilor. Exista doua versiuni – MA* [7]si SMA* [4] (Simple Memory-Bounded A*), ambele bazandu-se pe acelasi principiu. SMA* ruleaza similar cu A* pana in momentul in care cantitatea de memorie devine insuficienta. In acest moment spatiul de memorie necesar adaugarii unui nod nou este obtinut prin stergerea celui mai putin promitator nod deja explorat. In cazul in care exista o egalitate in privinta valorii functiei de evaluare se sterge nodul cel mai vechi. Pentru a evita posibilitatea in care nodul sters este totusi un nod care conduce la o cale optimala catre solutie valoarea f a nodului sters este retinuta la nodul parinte intr-un mod asemanator felului in care in RBFS se retine cea mai buna alternativa la nodul curent. Si in acest caz vor exista reexplorari de noduri, dar acest lucru se va intampla doar cand toate caile mai promitatoare vor esua. Desi exista probleme in care aceste regenerari de noduri au o frecventa care face ca algoritmul sa devina intractabil, MA* si SMA* asigura un compromis bun intre timpul de executie si limitarile de memorie.

Concluzii și observații

Deseori singura modalitate de rezolvare a problemelor dificile este de e explora graful starilor acelei probleme. Algoritmii clasici pe grafuri nu sunt intotdeauna potriviti pentru cautarea in aceste grafuri fie pentru ca nu garanteaza un rezultat fie pentru ca sunt ineficienti. Algoritmii euristici sunt algoritmi care exploreaza astfel de grafuri folosindu-se de o informatie suplimentara despre modul in care se poate ajunge la o stare scop mai rapid. A* este, teoretic, cel mai eficient algoritm de explorare euristica. In practica, insa, pentru probleme foarte dificile, A* implica un consum prea mare de memorie. In acest caz se folosesc variante de algoritmi care incearca sa minimizeze consumul de memorie in defavoarea timpului de executie.

Referinte

[1] S. Russel, P. Norvig - Artificial Intelligence: A Modern Approach - Prentice Hall, 2nd Edition – cap. 4

[2] C.A Giumale – Introducere in Analiza Algoritmilor, cap. 7

[3] Mehul Shah - Algorithms in the real world (Course Notes) - Introduction to VLSI routing [http://www.cs.cmu.edu/afs/cs/project/pscico-guyb/294/class-notes/all/18.ps]

[4] S. Russel - Efficient memory-bounded search methods [http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.54.6489&rep=rep1&type=pdf]

[5] R. Korf - Depth-first iterative-deepening: an optimal admissible tree search

[6] Recursive Best-First Search - Prezentare [http://www.ailab.si/gregorl/Students/mui/Recursive%20Best-First%20Search.ppt]

[7] P. Chakrabati, S. Ghosh, A. Acharya, S. DeSarkar – Heuristic search in restricted memory

[8] Wikipedia - Iterative deepening [http://en.wikipedia.org/wiki/Iterative_deepening_depth-first_search]

[9] A. E. Prieditis - Machine Discovery of Effective Admissible Heuristics [http://dli.iiit.ac.in/ijcai/IJCAI-91-VOL2/PDF/017.pdf]

[10] Wikipedia - Travelling salesman problem - Heuristic and approximation algorithms [http://en.wikipedia.org/wiki/Travelling_salesman_problem#Heuristic_and_approximation_algorithms]

[11] Wikipedia – A* [http://en.wikipedia.org/wiki/A*_search_algorithm]

[12] Tutorial A* [http://www.policyalmanac.org/games/aStarTutorial.htm]

[13] H. Kaindl, A. Khorsand - Memory Bounded Bidirectional Search, AAAI-94 Proceedings [http://aaaipress.org/Papers/AAAI/1994/AAAI94-209.pdf]

Probleme

În acest laborator vom rezolva o singură problemă: PACMAN [https://www.hackerrank.com/challenges/pacman-astar/problem] pe hackerrank. Puteți porni implementarea pe hackerrank de la următorul schelet: skel-lab12.zip.

Bonus

Problema misionarilor şi a canibalilor

Trei misionari şi trei canibali ajung la malul estic al unui râu. Aici se află o barcă cu două locuri cu care se poate traversa râul (râul nu se poate traversa înot). Dacă pe unul dintre maluri numărul de canibali este

mai mare decât numărul de misionari, atunci misionarii de pe acel mal vor fi mâncaţi de canibali. Se cere sa determinaţi cum pot trece toţi râul fără ca misionarii să fie mâncaţi de canibali.

Să se rezolve această problemă folosind algoritmul A* cu:

a) O euristica care nu este admisibilă

h(S) = nE(S)

b) Doua euristici admisibile

h1(S) = nE(S)/2

h2(S) =

* nE(S) + 1 daca barca pe malul vestic si nE(S) diferit de 0

* nE(S) - 1 daca barca pe malul estic si nE(S) diferit de 0

* 0 altfel

nE - numar oameni est

Comparati numarul de pasi necesar pentru a ajunge la solutie in functie de euristica

pa/laboratoare/laborator-12.txt · Last modified: 2020/05/31 20:27 by darius.neatu